

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
РАЗДЕЛ 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ	12
1.1. Характеристика предметной области	12
1.1.1. Общая характеристика и основные понятия предметной области	12
1.1.2. Анализ типовых сценариев деятельности целевой аудитории	13
1.1.3. Нормативная база предметной области	14
1.1.4. Выводы по результатам анализа предметной области.....	15
1.2. Исследование существующих программных решений.....	15
1.2.1. Критерии сравнения программных продуктов	15
1.2.2. Обзор и анализ аналога № 1: CoinKeeper	16
1.2.3. Обзор и анализ аналога № 2: ZenMoney	16
1.2.4. Обзор и анализ аналога № 3: Money Lover.....	16
1.2.5. Обзор и анализ аналога № 4: GnuCash.....	16
1.2.6. Сравнительная характеристика и определение недостатков существующих решений.....	17
1.2.7. Обоснование актуальности и конкурентных преимуществ собственной разработки.....	18
1.3. Выявление проблем и постановка задачи на разработку.....	18
1.3.1. Формулировка проблемной ситуации.....	18
1.3.2. Определение цели и задач создания программного продукта	18
1.3.3. Описание концепции будущего решения (ТО-ВЕ)	19
1.3.4. Определение границ проекта и ограничений	19
1.4. Моделирование и спецификация требований	19

1.4.1. Определение акторов и построение диаграммы вариантов использования	19
1.4.2. Детальное описание функциональных требований (спецификация прецедентов)	22
1.4.3. Формулировка нефункциональных требований	24
1.4.3.1. Требования к производительности.....	24
1.4.3.2. Требования к безопасности и защите данных.....	24
1.4.3.3. Требования к надёжности	25
1.4.3.4. Требования к интерфейсу и удобству использования (UI/UX).....	25
1.4.3.5. Требования к технологическому стеку (предварительные ограничения)	25
1.5. Моделирование предметной области (концептуальное проектирование данных).....	25
1.5.1. Выявление основных сущностей предметной области.....	25
1.5.2. Определение атрибутов и связей между сущностями	26
1.5.3. Построение концептуальной модели данных (ER-диаграмма).....	26
ВЫВОДЫ ПО РАЗДЕЛУ 1	27
РАЗДЕЛ 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	29
2.1. Выбор и обоснование архитектурного стиля	29
2.1.1. Анализ альтернатив	29
2.1.2. Критерии выбора и обоснование	29
2.2. Моделирование архитектуры с использованием подхода C4	30
2.2.1. Диаграмма контекста системы (уровень 1)	30
2.2.2. Диаграмма контейнеров (уровень 2).....	32
2.2.3. Диаграмма компонентов (уровень 3)	33

2.2.4. Проектирование REST API	34
2.3. Проектирование базы данных.....	35
2.3.1. Концептуальная и логическая модели данных	35
2.3.2. Физическая модель данных и обоснование выбора СУБД.....	36
2.4. Проектирование пользовательского интерфейса.....	37
2.4.1. Прототипы экранных форм.....	37
2.4.2. Описание навигации и пользовательских сценариев	38
2.5. Детальное проектирование (уровень кода)	39
2.5.1. Диаграмма классов ключевого компонента	39
2.5.2. Диаграмма последовательности для ключевого сценария	40
2.5.3. Ключевые архитектурные решения	40
2.5.4. Применение шаблонов проектирования (GoF и смежные)	41
2.6. Обоснование выбора стека технологий	42
ВЫВОДЫ ПО РАЗДЕЛУ 2	42
РАЗДЕЛ 3. КОНСТРУИРОВАНИЕ (РЕАЛИЗАЦИЯ) ПРОГРАММНОГО ПРОДУКТА	44
3.1. Организация процесса разработки	44
3.1.1. Выбор и обоснование методологии разработки	44
3.1.2. Инструменты управления задачами	44
3.1.3. Организация репозитория и системы контроля версий	44
3.1.4. Стратегия ветвления	45
3.1.5. Правила оформления коммитов	46
3.2. Соответствие реализации спроектированной архитектуре	46
3.2.1. Сопоставление компонентов архитектуры и модулей реализации	46
3.2.2. Организация файловой структуры проекта.....	46

3.2.3. Конфигурационные файлы и управление настройками.....	47
3.3. Реализация ключевых компонентов.....	47
3.3.1. Сервис проводки балансов (ledger)	47
3.3.2. Сервис курсов валют (cbr_rates)	48
3.3.3. Движок повторяющихся операций (recurrence).....	48
3.3.4. Слой доступа к данным	49
3.4. Демонстрация применения принципов качественного кода	49
3.4.1. Принципы SOLID	49
3.4.2. Применение шаблонов проектирования (GoF и смежные)	50
3.4.3. Принципы чистого кода (Clean Code).....	50
3.5. Интеграция с внешними сервисами и библиотеками.....	51
3.5.1. Интеграция с сервисом курсов ЦБ РФ.....	51
3.5.2. Использование сторонних библиотек	51
3.6. Обеспечение безопасности и обработка ошибок.....	51
3.6.1. Аутентификация и авторизация.....	51
3.6.2. Валидация входных данных.....	52
3.6.3. Логирование.....	52
3.6.4. Обработка исключительных ситуаций	52
3.7. Версионность и управление конфигурацией	53
3.7.1. Версионирование продукта.....	53
3.7.2. Анализ истории репозитория.....	53
3.7.3. Управление зависимостями	53
3.7.4. Управление конфигурацией окружения	53
ВЫВОДЫ ПО РАЗДЕЛУ 3	54

РАЗДЕЛ 4. ТЕСТИРОВАНИЕ, ОБЕСПЕЧЕНИЕ КАЧЕСТВА И СОПРОВОЖДЕНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	55
4.1. Стратегия тестирования программного продукта	55
4.1.1. Цели и задачи тестирования.....	55
4.1.2. Критерии начала и окончания тестирования	55
4.1.3. Выбор уровней тестирования	55
4.1.4. Выбор типов тестирования	56
4.1.5. Инструментарий тестирования	56
4.1.6. Организация тестовой среды	56
4.2. Модульное тестирование.....	56
4.2.1. Фреймворк и организация	56
4.2.2. Объекты модульного тестирования	57
4.2.3. Анализ покрытия кода	57
4.3. Интеграционное, системное и нагрузочное тестирование	58
4.3.1. Интеграционное тестирование	58
4.3.2. Системное тестирование	59
4.3.3. Нагрузочное тестирование	59
4.3.4. UI/UX и тестирование совместимости.....	61
4.3.5. Тестирование безопасности	61
4.4. Результаты тестирования и анализ дефектов	62
4.4.1. Сводные результаты тестирования	62
4.4.2. Классификация дефектов	62
4.4.3. Таблица найденных и исправленных дефектов	62
4.4.4. Динамика обнаружения и исправления дефектов	63
4.4.5. Неисправленные дефекты и ограничения	63

4.4.6. Выводы о качестве программного продукта.....	64
4.5. Подготовка к сопровождению (развёртывание и документация).....	64
4.5.1. Требования к окружению эксплуатации.....	64
4.5.2. Процесс развёртывания	64
4.5.3. Пользовательская документация	64
4.5.4. Техническая документация	65
4.5.5. План сопровождения.....	65
4.5.6. Готовность продукта к внедрению	65
ВЫВОДЫ ПО РАЗДЕЛУ 4	65
РАЗДЕЛ 5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА	67
5.1. Планирование ресурсов и расчёт трудоёмкости разработки.....	67
5.1.1. Состав работ и этапов разработки.....	67
5.1.2. Метод оценки трудоёмкости.....	67
5.1.3. Расчёт трудоёмкости.....	67
5.1.4. Итоговая трудоёмкость.....	69
5.1.5. Календарное планирование.....	69
5.2. Расчёт себестоимости разработки	69
5.2.1. Состав статей калькуляции	69
5.2.2. Затраты на оплату труда.....	69
5.2.3. Отчисления на социальные нужды	70
5.2.4. Затраты на электроэнергию.....	70
5.2.5. Амортизация вычислительной техники.....	70
5.2.6. Материальные затраты	70
5.2.7. Накладные расходы.....	70
5.2.8. Смета затрат на разработку	70

5.3. Оценка экономической эффективности проекта	71
5.3.1. Модель экономического эффекта.....	71
5.3.2. Годовой экономический эффект.....	71
5.3.3. Показатели эффективности инвестиций.....	71
5.4. Основные технико-экономические показатели проекта	73
5.5. Обоснование экономической целесообразности	73
ВЫВОДЫ ПО РАЗДЕЛУ 5	74
РАЗДЕЛ 6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА ПРИ ЭКСПЛУАТАЦИИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	75
6.1. Анализ опасных и вредных факторов на рабочем месте	75
6.2. Мероприятия по обеспечению безопасности труда	76
6.2.1. Организация рабочего места.....	76
6.2.2. Микроклимат	76
6.2.3. Расчёт искусственного освещения	77
6.2.4. Режим труда и отдыха	77
6.3. Электробезопасность и пожарная безопасность.....	78
6.3.1. Электробезопасность	78
6.3.2. Пожарная безопасность	78
ВЫВОДЫ ПО РАЗДЕЛУ 6	79
ЗАКЛЮЧЕНИЕ	80
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	83
Нормативные правовые акты и стандарты.....	83
Литература	84
Электронные ресурсы	85
ПРИЛОЖЕНИЕ А. ЛИСТИНГИ ПРОГРАММНОГО КОДА	87

ПРИЛОЖЕНИЕ Б. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	110
ПРИЛОЖЕНИЕ В. СНИМКИ ЭКРАНОВ ПРИЛОЖЕНИЯ	115

ВВЕДЕНИЕ

Актуальность темы. Управление личными финансами становится всё более значимым навыком в условиях преобладания безналичных расчётов и роста числа финансовых операций у частных лиц. При этом для российского пользователя сократился выбор удобных инструментов учёта: ряд иностранных сервисов прекратил работу или ушёл с рынка (в частности, по открытым данным, в 2024 году закрыт популярный сервис Mint), а оставшиеся коммерческие приложения либо ограничивают бесплатные тарифы, либо хранят чувствительные финансовые данные на внешней инфраструктуре. Последнее обостряет вопрос приватности: финансовая информация относится к персональным данным, и их обработка на чужих серверах противоречит интересам пользователя и требованиям законодательства РФ. Дополнительную актуальность придаёт распространённость валютных сбережений и доходов, что требует мультивалютного учёта по официальным курсам. Перечисленное обосновывает потребность в бесплатном, приватном (self-hosted) и мультивалютном инструменте учёта личных финансов с открытым исходным кодом.

Цель работы — автоматизация рутинных процессов ведения личных финансов (повторяющихся операций, мультивалютного пересчёта по курсам Центрального банка Российской Федерации, контроля бюджетов и формирования отчётности) посредством разработки self-hosted веб-приложения, соответствующего требованиям законодательства о персональных данных.

Задачи работы. Для достижения цели решены следующие задачи:

1. провести анализ предметной области и существующих программных решений (аналогов);
2. сформировать и документировать функциональные и нефункциональные требования к программному продукту, построить модель вариантов использования;

3. спроектировать архитектуру приложения (клиент-серверную, с применением методологии C4) и REST API;
4. спроектировать модель данных и обосновать выбор системы управления базами данных;
5. реализовать серверную часть на FastAPI с асинхронным доступом к данным;
6. реализовать клиентскую часть (одностраничное веб-приложение) на React;
7. реализовать интеграцию с сервисом курсов валют ЦБ РФ;
8. обеспечить безопасность приложения (хеширование паролей, аутентификация, изоляция данных) и соответствие 152-ФЗ;
9. провести тестирование (модульное, интеграционное, системное, нагрузочное) и оценку качества продукта;
10. выполнить технико-экономическое обоснование разработки;
11. рассмотреть вопросы безопасности жизнедеятельности при эксплуатации программного обеспечения.

Объект исследования — процесс учёта личных финансов физического лица.

Предмет исследования — программные средства и алгоритмы автоматизации учёта и анализа личных финансов.

Методы исследования. В работе использованы: анализ научно-технической литературы и сравнительный анализ аналогов; объектно-ориентированное проектирование и UML-моделирование (диаграммы вариантов использования, классов, последовательности, модель C4); реляционное моделирование баз данных и нормализация; методы безопасной разработки (рекомендации OWASP); экспериментальный метод (модульное, интеграционное, системное и нагрузочное тестирование).

Практическая значимость. Результатом работы является готовое к развёртыванию веб-приложение FinTrack с открытым исходным кодом и открытым REST API. Self-hosted-модель обеспечивает хранение данных под

контролем пользователя и соответствие требованиям 152-ФЗ. Продукт применим для персонального учёта финансов и в малых семейных «бухгалтериях», а открытость кода и API допускает его дальнейшую доработку и интеграцию.

Структура работы. Пояснительная записка состоит из введения, шести разделов, заключения, списка использованных источников и приложений.

- В первом разделе проведён анализ предметной области и аналогов, сформированы требования к ПО, построены модель вариантов использования и концептуальная модель данных.

- Во втором разделе спроектирована архитектура приложения по методологии C4, спроектированы база данных и REST API, обоснован выбор технологического стека.

- В третьем разделе описано конструирование продукта: организация разработки и контроля версий, реализация ключевых компонентов, соблюдение принципов качественного кода (SOLID, шаблоны проектирования), обеспечение безопасности.

- В четвёртом разделе изложены стратегия и результаты тестирования (включая нагрузочное), анализ дефектов и мероприятия по подготовке к сопровождению.

- В пятом разделе выполнено технико-экономическое обоснование разработки.

- В шестом разделе рассмотрены вопросы безопасности жизнедеятельности при эксплуатации программного обеспечения.

В заключении приведены итоги работы и направления её дальнейшего развития. В приложениях размещены листинги программного кода, схема базы данных, снимки экранов приложения и сопутствующие документы.

РАЗДЕЛ 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

1.1. Характеристика предметной области

1.1.1. Общая характеристика и основные понятия предметной области

Предметной областью настоящей работы является учёт личных финансов, систематическая регистрация доходов и расходов физического лица, контроль остатков по счетам, планирование бюджета и анализ структуры трат. В отличие от корпоративного бухгалтерского учёта, личный финансовый учёт не регламентируется стандартами отчётности, ориентирован на одного владельца и преследует прикладную цель — помочь человеку понимать, куда уходят его деньги, и принимать обоснованные финансовые решения.

Ключевые понятия предметной области, на которых строится разрабатываемое приложение:

- Счёт (account) — источник или хранилище средств пользователя. В FinTrack поддерживается шесть типов счетов: банковская карта, наличные, накопительный счёт, кредитная карта, электронный кошелёк и «прочее». Каждый счёт ведётся в собственной валюте.

- Операция (transaction) — атомарное финансовое событие одного из трёх видов: доход (поступление средств), расход (списание) и перевод между собственными счетами пользователя.

- Категория (category) — классификатор операций с разделением на доходные и расходные и древовидной иерархией (например, «Продукты» → «Бакалея»).

- Бюджет (budget) — заданный пользователем лимит расходов по выбранной категории на конкретный календарный месяц.

– Повторяющаяся операция (recurring transaction) — правило автоматического создания операций по расписанию (заработная плата, подписки, аренда).

– Мультивалютность — возможность вести счета в разных валютах с пересчётом сумм в единую валюту по официальным курсам Центрального банка Российской Федерации.

– Источник курсов валют — ежедневная XML-выгрузка ЦБ РФ (https://www.cbr.ru/scripts/XML_daily.asp), публикуемая в рабочие дни.

Место разрабатываемого ПО в структуре предметной области — это персональный инструмент аналитического учёта: он не подключается к банковским системам напрямую, а агрегирует и анализирует данные, которые вводит сам пользователь (либо которые порождаются по заданным им правилам повторения), предоставляя наглядную картину личного финансового состояния.

1.1.2. Анализ типовых сценариев деятельности целевой аудитории

Целевая аудитория продукта разделена на три сегмента.

1. Молодой специалист (22–35 лет). Использует приложение ежедневно для быстрого ввода операций и еженедельной сверки фактических трат с запланированным бюджетом. Ключевая ценность — скорость ввода и наглядность.

2. Семья с совместным бюджетом. Ведёт общий учёт расходов по категориям, планирует крупные покупки и контролирует месячные лимиты. Ключевая ценность — категоризация и бюджетирование.

3. Фрилансер с нерегулярным и валютным доходом. Получает доход в разных валютах (RUB, USD, EUR), нуждается в мультивалютном учёте, отчётах по произвольным периодам и выгрузке данных. Ключевая ценность — мультивалютность и экспорт.

Типовые проблемы («боли») целевой аудитории, выявленные при анализе:

- учёт в электронных таблицах (Excel, Google Sheets) трудоёмок, не автоматизирует пересчёт валют и не даёт удобной визуализации;
- облачные сервисы личных финансов хранят чувствительные данные на чужой инфраструктуре, что вызывает опасения за приватность;
- бесплатные тарифы коммерческих приложений сильно ограничены по функциональности (число счетов, период истории, отчёты), а полный доступ требует подписки.

На основе сегментов сформулированы пользовательские истории (user stories), например:

- «Как молодой специалист, я хочу за несколько секунд внести расход с указанием категории, чтобы не откладывать учёт на потом».
- «Как фрилансер, я хочу видеть суммарный капитал во всех валютах, пересчитанный в рубли по курсу ЦБ, чтобы понимать своё реальное финансовое положение».
- «Как член семьи, я хочу задать на месяц лимит по категории „Развлечения“ и видеть процент его расходования, чтобы не выйти за рамки бюджета».

1.1.3. Нормативная база предметной области

Поскольку финансовая информация физического лица относится к персональным данным, разработка и эксплуатация приложения регулируются рядом нормативных актов:

- Федеральный закон № 152-ФЗ «О персональных данных» [1] — определяет требования к обработке и защите персональных данных, включая право субъекта на уточнение, блокирование или уничтожение своих данных (ст. 14) и обязанность оператора обеспечить их удаление (ст. 21);
- Постановление Правительства РФ № 1119 от 01.11.2012 «Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных» [5] — устанавливает уровни

защищённости персональных данных и требования к средствам защиты (шифрование канала, хеширование паролей, разграничение доступа);

– Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и о защите информации» [2] — регулирует обработку информации, что повышает актуальность self-hosted-решений с хранением данных на стороне пользователя;

– Постановление Правительства РФ № 1236 от 16.11.2015 [6] — Реестр российского программного обеспечения, в контексте импортозамещения.

1.1.4. Выводы по результатам анализа предметной области

Анализ предметной области «учёт личных финансов» показал, что её ключевыми особенностями являются ориентация на единственного владельца, отсутствие жёсткой нормативной регламентации формата учёта и высокая чувствительность обрабатываемых данных. Выделены три сегмента целевой аудитории с общими потребностями в простом, наглядном, мультивалютном и приватном инструменте. Установлено, что обрабатываемые данные относятся к персональным, что обуславливает требования к их защите и обосновывает целесообразность self-hosted-модели развёртывания.

1.2. Исследование существующих программных решений

1.2.1. Критерии сравнения программных продуктов

Для сравнительного анализа аналогов выбраны критерии, отражающие потребности целевой аудитории и концепцию разрабатываемого продукта:

1. веб-доступ из браузера;
2. наличие бесплатного полнофункционального тарифа;
3. мультивалютность с курсами ЦБ РФ;
4. возможность self-hosted-развёртывания;
5. наличие открытого REST API;
6. хранение данных на территории РФ;
7. импорт/экспорт данных (CSV);

8. открытость исходного кода.

1.2.2. Обзор и анализ аналога № 1: CoinKeeper

CoinKeeper (Россия, развивается с 2012 г.) — преимущественно мобильное приложение для учёта личных финансов с наглядным интерфейсом «перетаскивания» средств между конвертами. Веб-версия доступна по подписке. Бесплатный тариф существенно ограничен; платный составляет порядка 399 ₽/мес. Недостатки: отсутствует self-hosted-режим, нет открытого API и открытого исходного кода.

1.2.3. Обзор и анализ аналога № 2: ZenMoney

ZenMoney (Россия, с 2010 г.) — кроссплатформенное решение (мобильное, веб, десктоп). Сильная сторона — автоматический импорт операций из более чем 60 банков. Модель распространения — подписка (порядка 599 ₽/мес.). Недостатки: отсутствует открытый API, нет self-hosted-режима и открытого исходного кода; полноценное использование требует платной подписки.

1.2.4. Обзор и анализ аналога № 3: Money Lover

Money Lover (Вьетнам, Finsify) — мобильное и веб-приложение с бесплатным тарифом, ограниченным пятью кошельками и содержащим рекламу. Недостатки: серверы расположены вне РФ, что не соответствует требованиям к хранению персональных данных; по открытым данным, с 2022 г. приложение недоступно в российском сегменте App Store.

1.2.5. Обзор и анализ аналога № 4: GnuCash

GnuCash (международный проект, развивается с 1998 г.) — свободное (лицензия GPL) настольное приложение для учёта финансов с двойной записью и поддержкой мультивалютности. Сильные стороны в контексте поставленной задачи: полностью бесплатный и открытый исходный код, данные хранятся локально у пользователя (полный контроль приватности), есть импорт/экспорт

в распространённых форматах. Недостатки: это настольное приложение без веб-доступа и без открытого REST API; курсы валют не привязаны к ЦБ РФ; интерфейс ориентирован на бухгалтерскую двойную запись и сложен для неподготовленного пользователя. GnuCash включён в обзор как ближайший по идеологии аналог (открытость и контроль над данными), что делает сравнение с ним показательным.

1.2.6. Сравнительная характеристика и определение недостатков существующих решений

Результаты сравнения аналогов и разрабатываемого продукта по выбранным критериям сведены в таблицу 1.1.

Таблица 1.1 – Сравнительная характеристика аналогов и FinTrack

Критерий	CoinKeeper	ZenMoney	Money Lover	GnuCash	FinTrack
Веб-доступ из браузера	По подписке	Да	Да	Нет	Да
Бесплатный полный тариф	Нет	Нет	Частично	Да	Да
Мультивалютность по курсам ЦБ РФ	Частично	Да	Нет	Нет	Да
Self-hosted-развёртывание / локальное хранение	Нет	Нет	Нет	Да	Да
Открытый REST API	Нет	Нет	Нет	Нет	Да
Хранение данных в РФ	Да	Да	Нет	Да	Да (у пользователя)
Импорт/экспорт CSV	Частично	Да	Да	Да	Частично (только экспорт)
Открытый исходный код	Нет	Нет	Нет	Да	Да

Сравнение показывает, что ни один из рассмотренных аналогов не сочетает одновременно бесплатный полный функционал, веб-доступ, self-hosted-развёртывание, открытый API, мультивалютность по курсам ЦБ РФ и хранение данных под контролем пользователя. Ближе всех по идеологии открытости и контроля над данными — GnuCash, однако он лишён веб-доступа, открытого API и интеграции с курсами ЦБ РФ.

1.2.7. Обоснование актуальности и конкурентных преимуществ собственной разработки

На основании выявленных недостатков аналогов определены конкурентные преимущества FinTrack:

- веб-доступ из любого браузера, без платных тарифов и рекламных вставок;
- self-hosted-модель — данные хранятся на сервере пользователя, что обеспечивает приватность и соответствие требованиям 152-ФЗ;
- открытый REST API с автоматической документацией OpenAPI 3.0 (Swagger UI) — возможность интеграции и создания альтернативных клиентов;
- прямая интеграция с XML-сервисом ЦБ РФ — мультивалютный учёт по официальным курсам;
- открытый исходный код — возможность доработки и аудита.

1.3. Выявление проблем и постановка задачи на разработку

1.3.1. Формулировка проблемной ситуации

Российскому пользователю недоступно программное решение для учёта личных финансов, которое одновременно: предоставляется бесплатно и в полном объёме, работает в браузере, хранит данные под контролем самого пользователя, поддерживает мультивалютный учёт по официальным курсам и соответствует требованиям законодательства РФ о персональных данных.

1.3.2. Определение цели и задач создания программного продукта

Цель разработки — автоматизация рутинных процессов учёта и анализа личных финансов посредством создания веб-приложения FinTrack, поддерживающего мультивалютные счета, категоризированный учёт операций, бюджетирование, повторяющиеся операции и отчётность, с возможностью self-hosted-развёртывания и соблюдением требований 152-ФЗ.

Для достижения цели продукт должен решать следующие основные задачи (функции): регистрация и аутентификация пользователя; ведение счетов

в разных валютах; учёт операций трёх видов с категоризацией; переводы между счетами, включая кросс-валютные; бюджетирование по категориям; автоматизация повторяющихся операций; получение курсов валют ЦБ РФ и пересчёт капитала; формирование сводных отчётов и дашборда; экспорт данных; защита персональных данных и право на их удаление.

1.3.3. Описание концепции будущего решения (ТО-ВЕ)

Целевой пользовательский сценарий: пользователь регистрируется и входит в систему → создаёт счета в нужных валютах → ведёт операции (доход, расход, перевод, в том числе кросс-валютный) → настраивает бюджеты и правила повторяющихся операций → анализирует финансовое состояние на дашборде и в отчётах → при необходимости экспортирует историю операций в CSV. Все данные хранятся на сервере, развёрнутом самим пользователем.

1.3.4. Определение границ проекта и ограничений

Включено в проект: серверная часть (backend) на FastAPI, одностраничное веб-приложение (SPA) на React, реляционная СУБД PostgreSQL, миграции схемы (Alembic), контейнеризация (Docker).

Исключено из проекта (вне границ MVP): мобильное приложение; прямая интеграция с банковскими системами (open banking); инвестиционный и кредитный учёт; многопользовательский режим с ролевой моделью. Перечисленные возможности отнесены к направлениям дальнейшего развития (см. заключение).

1.4. Моделирование и спецификация требований

1.4.1. Определение акторов и построение диаграммы вариантов использования

В системе выделены три актора:

- Пользователь — аутентифицированное физическое лицо; в self-hosted-установке он же является владельцем сервера. Это единственная роль: ролевая

модель «администратор/модератор» сознательно не вводится, поскольку инструмент персональный.

- Сервис курсов ЦБ РФ — внешний XML-фид, поставляющий официальные курсы валют (вспомогательный актер).

- Почтовый сервер (SMTP) — внешняя система доставки писем подтверждения адреса и сброса пароля (вспомогательный актер).

Взаимодействие акторов с системой и состав вариантов использования представлены на диаграмме вариантов использования (рисунок 1.1).

Диаграмма вариантов использования системы FinTrack

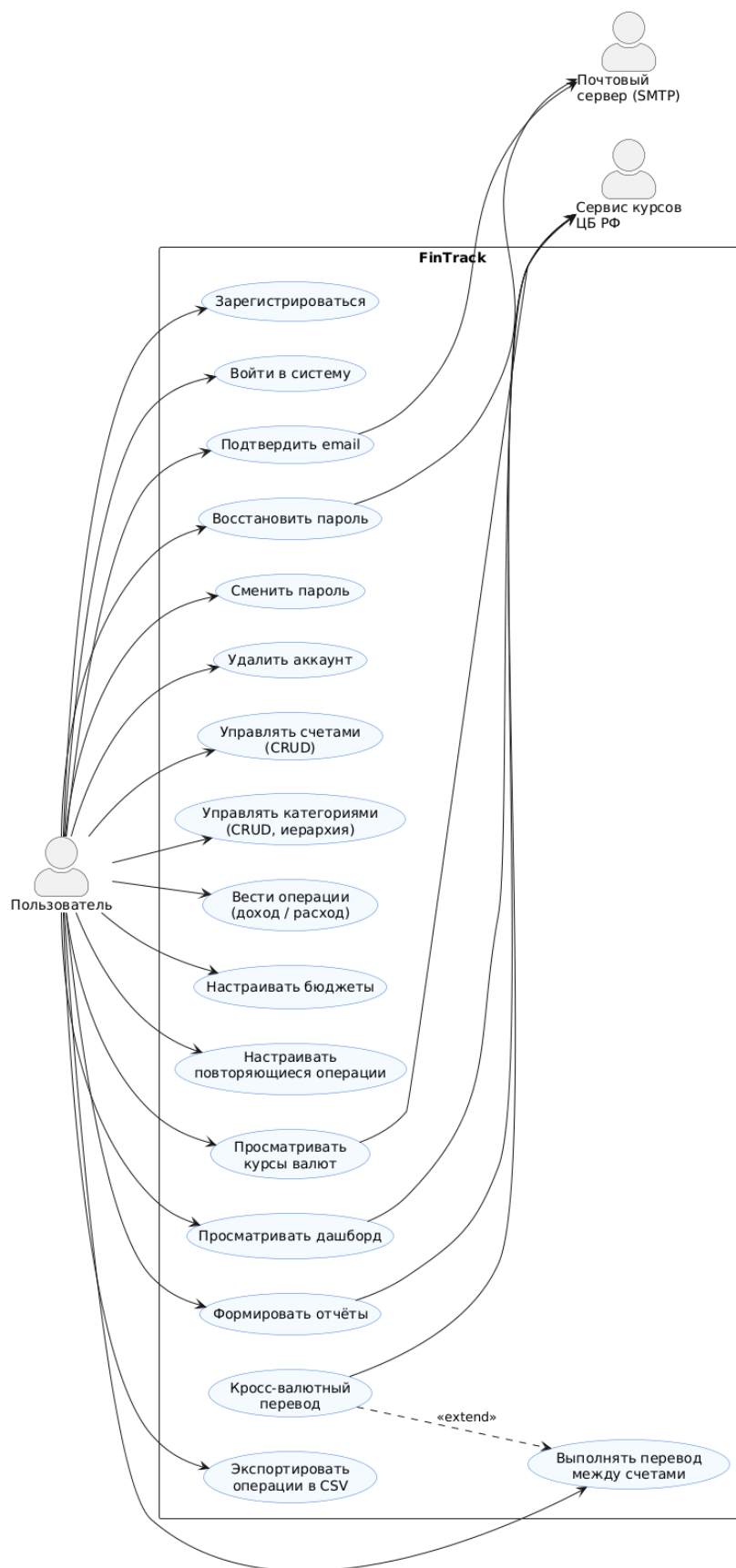


Рисунок 1.1 – Диаграмма вариантов использования системы FinTrack

1.4.2. Детальное описание функциональных требований (спецификация прецедентов)

Для ключевых вариантов использования приведено детальное описание по стандартному шаблону (таблицы 1.2–1.4).

Таблица 1.2 – Спецификация прецедента UC-1 «Добавить расход»

Элемент	Описание
Название	Добавить расход
Актор	Пользователь
Предусловия	Пользователь авторизован и имеет хотя бы один счёт
Основной поток	1. Пользователь открывает форму операции. 2. Выбирает тип «Расход», счёт, сумму, категорию и дату. 3. Подтверждает. 4. Система создаёт операцию и атомарно уменьшает баланс счёта
Альтернативные потоки	A1. Сумма $\leq 0 \rightarrow$ ошибка валидации (HTTP 422). A2. Дата операции раньше «даты начального остатка» счёта $\rightarrow$ операция сохраняется в истории, но баланс не изменяется
Постусловие	Операция сохранена в истории, баланс счёта уменьшен

Таблица 1.3 – Спецификация прецедента UC-2 «Выполнить кросс-валютный перевод»

Элемент	Описание
Название	Выполнить кросс-валютный перевод
Актор	Пользователь
Предусловия	У пользователя есть два счёта в разных валютах
Основной поток	1. Пользователь выбирает тип «Перевод», счёт-источник и счёт-получатель. 2. Вводит сумму списания. 3. Система предзаполняет сумму зачисления по курсу ЦБ РФ. 4. Пользователь при необходимости корректирует её (банковский курс отличается от курса ЦБ). 5. Подтверждает. 6. Система атомарно списывает сумму с источника и зачисляет сумму на получателя
Альтернативные потоки	A1. Валюты счетов различаются, а сумма зачисления не указана $\rightarrow$ ошибка (HTTP 400). A2. Курс ЦБ недоступен $\rightarrow$ используется последний известный курс
Постусловие	Балансы обоих счетов обновлены согласованно в рамках одной транзакции БД

Таблица 1.4 – Спецификация прецедента UC-3 «Настроить бюджет на категорию»

Элемент	Описание
Название	Настроить бюджет на категорию
Актор	Пользователь
Предусловия	Существует хотя бы одна расходная категория
Основной поток	1. Пользователь задаёт категорию, месяц и сумму лимита. 2. Система сохраняет бюджет и отслеживает сумму расходов по категории (с пересчётом в рубли). 3. Отображает статус: «в норме» / «предупреждение» / «превышен»
Альтернативные потоки	A1. На категорию в этом месяце уже задан бюджет → ошибка (HTTP 409)
Постусловие	Бюджет отображается на странице бюджетов и на дашборде с индикатором прогресса

Полный перечень функциональных требований, согласованных с реализацией и привязанных к вариантам использования, приведён в таблице 1.5.

Таблица 1.5 – Функциональные требования

№	Требование
ФТ-01	Регистрация и вход по адресу email и паролю, выдача JWT-токена
ФТ-02	Подтверждение email, сброс и смена пароля по одноразовым ссылкам
ФТ-03	Управление счетами (шесть типов, любая валюта): создание, просмотр, изменение, удаление
ФТ-04	Учёт по модели «начальный остаток + движения»: баланс вычисляется от начального остатка и операций
ФТ-05	Управление категориями (доходные/расходные, иерархия): полный CRUD
ФТ-06	Учёт операций трёх видов (доход, расход, перевод) с фильтрами и поиском по заметке
ФТ-07	Кросс-валютные переводы (списание в валюте источника, зачисление в валюте получателя, предзаполнение по курсу ЦБ)
ФТ-08	Бюджеты по категориям на месяц с расчётом прогресса и статуса (норма/предупреждение/превышение)
ФТ-09	Повторяющиеся операции (правила и движок до-генерации без планировщика)

№	Требование
ФТ-10	Получение курсов валют ЦБ РФ (схема cache-aside), расчёт общего капитала в рублях
ФТ-11	Сводные отчёты: доходы/расходы по периодам, структура категорий, динамика капитала
ФТ-12	Дашборд: общий капитал, расходы за месяц, топ категорий, прогресс бюджетов
ФТ-13	Экспорт операций в CSV с учётом фильтров
ФТ-14	Тёмная тема оформления (автоматически и вручную)
ФТ-15	Право на забвение: самостоятельное удаление учётной записи со всеми данными (152-ФЗ, ст. 14 и ст. 21)

1.4.3. Формулировка нефункциональных требований

Нефункциональные требования сформулированы в соответствии с рекомендациями IEEE Std 830-1998 [9] и сгруппированы по категориям.

1.4.3.1. Требования к производительности

– НФТ-01: время отклика API при чтении списка операций при типовом объёме данных и нагрузке 20 одновременных пользователей не превышает 200 мс по 95-му перцентилю (целевая медиана — не более 50 мс);

– НФТ-02: формирование сводных отчётов и дашборда выполняется не дольше 300 мс по 95-му перцентилю за счёт агрегаций на стороне СУБД; доля ошибок под нагрузкой — 0 %.

1.4.3.2. Требования к безопасности и защите данных

– НФТ-03: пароли хранятся в виде хеша по алгоритму Argon2id (без хранения в открытом виде);

– НФТ-04: аутентификация на основе JWT (HS256), секрет не короче 32 байт, срок жизни токена — 7 суток;

– НФТ-05: изоляция данных между пользователями — проверка владельца ресурса с ответом 404 при обращении к чужому ресурсу («защита от IDOR (Insecure Direct Object Reference)» без раскрытия факта существования);

– НФТ-06: шифрование канала TLS terminates обратным прокси при production-развёртывании; параметризованные ORM-запросы (защита от SQL-инъекций); экранирование вывода во фронтенде (защита от XSS); передача токена в заголовке Authorization (снижение рисков CSRF).

1.4.3.3. Требования к надёжности

– НФТ-07: денежные операции атомарны — создание операции и обновление баланса(ов) выполняются в одной транзакции БД; для перевода атомарно обновляются два счёта;

– НФТ-08: денежные суммы хранятся в типе NUMERIC(15,2) и обрабатываются как Decimal, что исключает погрешности чисел с плавающей точкой.

1.4.3.4. Требования к интерфейсу и удобству использования (UI/UX)

– НФТ-09: одностраничное приложение (SPA) с адаптивной вёрсткой; поддержка современных браузеров (на движках Chromium и Firefox);

– НФТ-10: самодокументируемое API — спецификация OpenAPI 3.0 и интерактивный Swagger UI.

1.4.3.5. Требования к технологическому стеку (предварительные ограничения)

– НФТ-11: серверная часть — Python 3.13, FastAPI, SQLAlchemy 2.0 (асинхронный режим), Alembic, PostgreSQL 15; клиентская часть — React 19, TypeScript 6, Vite, Mantine 9; развёртывание — Docker.

1.5. Моделирование предметной области (концептуальное проектирование данных)

1.5.1. Выявление основных сущностей предметной области

На основе анализа предметной области и сформированных требований выделено восемь сущностей:

1. Пользователь (users) — владелец учётной записи;

2. Счёт (accounts) — счёт пользователя в определённой валюте;
3. Категория (categories) — иерархический классификатор операций;
4. Операция (transactions) — доход, расход или перевод;
5. Бюджет (budgets) — месячный лимит по категории;
6. Повторяющаяся операция (recurring_transactions) — шаблон автоповтора;
7. Курс валюты ЦБ (exchange_rates) — справочник официальных курсов;
8. Токен письма (email_tokens) — одноразовый токен подтверждения/сброса.

1.5.2. Определение атрибутов и связей между сущностями

Пользователь является владельцем счетов, категорий, операций, бюджетов, повторяющихся операций и токенов (связи «один ко многим»). Категория допускает иерархию через self-ссылку на родительскую категорию. Операция связана со счётом-источником (обязательно) и, для перевода, со счётом-получателем (опционально), а также, опционально, с категорией. Бюджет привязан к расходной категории. Повторяющаяся операция ссылается на счёт (и счёт-получатель для перевода) и опционально на категорию. Справочник курсов ЦБ не привязан к пользователю и используется для пересчёта валют по буквенному коду.

Отдельной таблицы-справочника валют не вводится: код валюты хранится строкой по стандарту ISO 4217 непосредственно в счёте и операции. Это сознательное упрощение модели для персонального масштаба (обосновано в разделе 2).

1.5.3. Построение концептуальной модели данных (ER-диаграмма)

Концептуальная модель данных, отражающая выделенные сущности и связи между ними, представлена на рисунке 1.2.

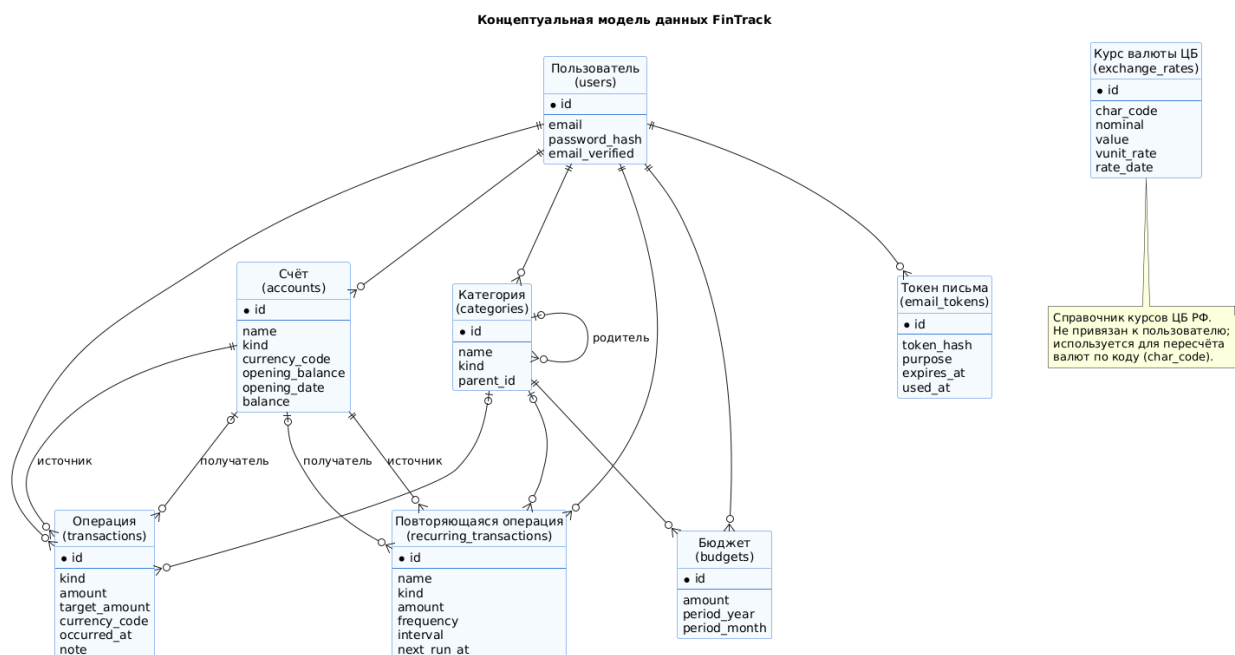


Рисунок 1.2 – Концептуальная модель данных FinTrack

ВЫВОДЫ ПО РАЗДЕЛУ 1

В результате выполнения первого раздела проведён комплексный анализ предметной области «учёт личных финансов» и сформированы требования к программному продукту.

1. Проанализирована предметная область: определены ключевые понятия (счёт, операция, категория, бюджет, повторяющаяся операция, мультивалютность) и выделены три сегмента целевой аудитории (молодой специалист, семья, фрилансер) с общей потребностью в простом, мультивалютном и приватном инструменте учёта.

2. Обзор существующих решений (CoinKeeper, ZenMoney, Money Lover, GnuCash) показал, что ни один аналог не сочетает бесплатный полный функционал, веб-доступ, self-hosted-режим, открытый API и хранение данных под контролем пользователя, что обосновывает актуальность и целесообразность собственной разработки.

3. Сформулированы и согласованы с реализацией 15 функциональных и 11 нефункциональных требований; определены акторы и построена диаграмма вариантов использования; для ключевых прецедентов («Добавить расход»,

«Кросс-валютный перевод», «Настроить бюджет») приведены детальные спецификации.

4. Построена концептуальная модель данных из восьми сущностей, являющаяся основой для проектирования базы данных в разделе 2.

Таким образом, сформированная спецификация требований и концептуальная модель данных являются достаточной основой для перехода к этапу проектирования архитектуры программного обеспечения; задачи аналитического этапа выполнены в полном объёме.

РАЗДЕЛ 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

На основе требований, сформированных в разделе 1, в настоящем разделе проектируется архитектурный каркас системы FinTrack. Изложение ведётся от общего к частному с применением методологии C4 model [38]: от диаграммы контекста до диаграмм компонентов и детального проектирования на уровне классов.

2.1. Выбор и обоснование архитектурного стиля

2.1.1. Анализ альтернатив

Для веб-приложения учёта личных финансов рассмотрены следующие архитектурные стили (таблица 2.1).

Таблица 2.1 – Сравнение архитектурных стилей применительно к проекту

Стиль	Сильные стороны	Почему не выбран / выбран
Монолитное серверное приложение (SSR, шаблоны/HTMX)	Простота, единая кодовая база	Жёстко связывает представление и логику; затрудняет открытый API и потенциальный мобильный клиент
Микросервисная архитектура	Независимое развёртывание, масштабирование частей	Избыточна для одного разработчика и персонального масштаба; высокая инфраструктурная сложность
Сервис-ориентированная / событийная	Слабая связанность, асинхронность	Не оправдана объёмом задач; усложняет разработку и отладку
Клиент-серверная трёхуровневая (SPA + REST API + СУБД)	Чёткое разделение слоёв, переиспользуемый API, простое развёртывание	Выбрана — см. обоснование ниже

2.1.2. Критерии выбора и обоснование

Критериями выбора служили: ожидаемая нагрузка (персональный масштаб), размер команды (один разработчик), требование наличия открытого REST API (НФТ и концепция продукта), возможность подключения в будущем

альтернативных клиентов (в т.ч. мобильного), простота self-hosted-развёртывания.

Выбрана клиент-серверная трёхуровневая архитектура с разделением на три слоя:

- слой представления — одностраничное приложение (SPA) на React, исполняемое в браузере пользователя;
- слой бизнес-логики — серверное приложение (REST API) на FastAPI, реализующее всю предметную логику; взаимодействие — stateless через HTTP/JSON с JWT-аутентификацией;
- слой данных — реляционная СУБД PostgreSQL.

Такое разделение обеспечивает чёткую границу между фронтендом и бэкендом, делает API самостоятельным продуктом (его можно использовать сторонними клиентами) и упрощает сопровождение. REST выбран как наиболее распространённый и самодокументируемый (OpenAPI) стиль взаимодействия; версионирование API заложено префиксом /api/v1.

2.2. Моделирование архитектуры с использованием подхода C4

2.2.1. Диаграмма контекста системы (уровень 1)

На верхнем уровне система FinTrack рассматривается как «чёрный ящик», взаимодействующий с пользователем и двумя внешними системами — сервисом курсов ЦБ РФ и почтовым сервером (рисунок 2.1).

Диаграмма контекста системы FinTrack (C4: Level 1)



Рисунок 2.1 – Диаграмма контекста системы FinTrack (C4, уровень 1)

Пользователь работает с системой через браузер по протоколу HTTPS. Система обращается к сервису курсов ЦБ РФ за официальными курсами валют (HTTPS, XML) и к почтовому серверу за доставкой писем подтверждения адреса и сброса пароля (SMTP).

2.2.2. Диаграмма контейнеров (уровень 2)

Декомпозиция системы на отдельно развёртываемые единицы (контейнеры) представлена на рисунке 2.2.

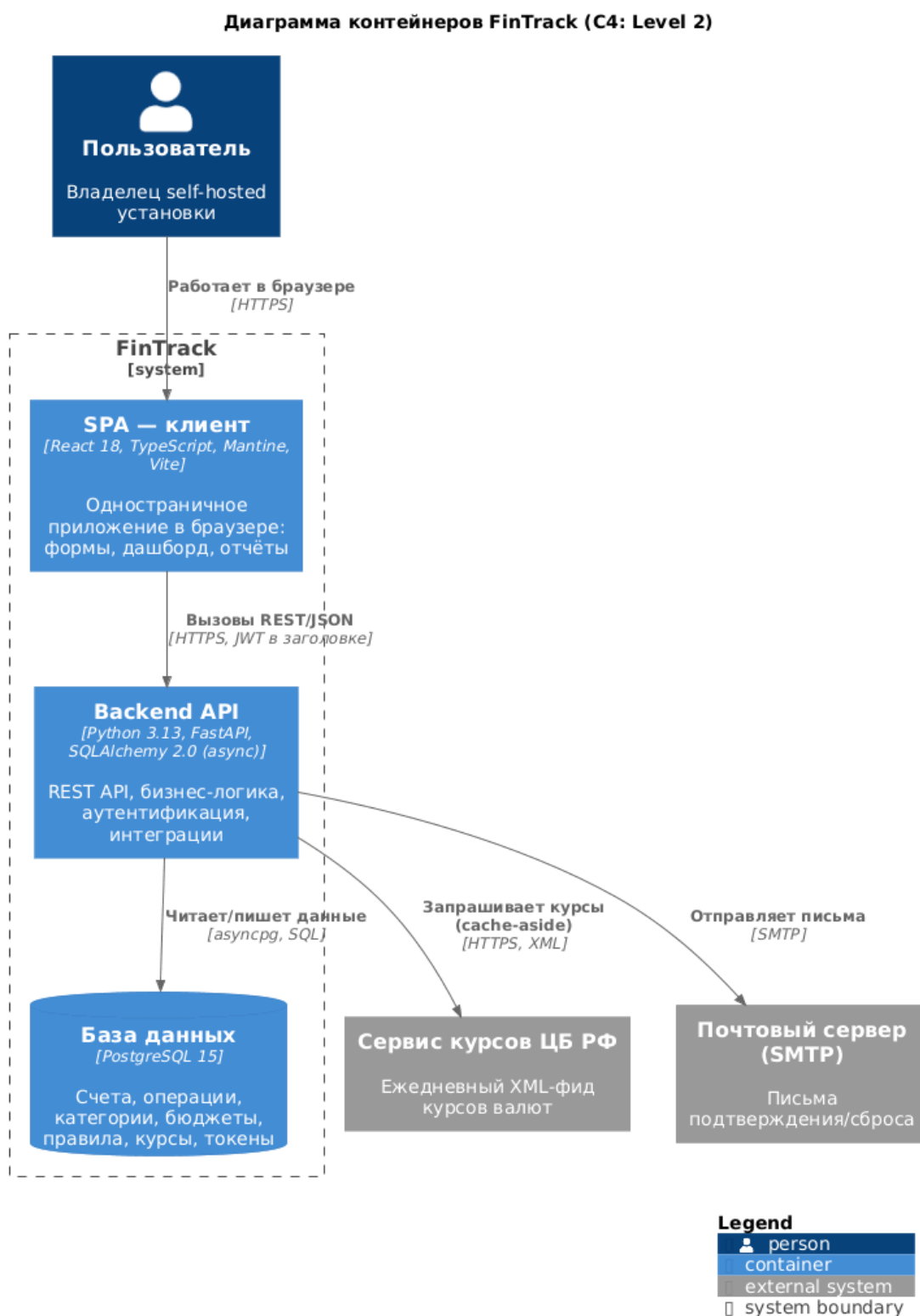


Рисунок 2.2 – Диаграмма контейнеров FinTrack (C4, уровень 2)

Система состоит из трёх контейнеров:

- SPA-клиент (React 19 [28], TypeScript [29], Mantine [30], Vite) — пользовательский интерфейс в браузере;
- Backend API (Python 3.13, FastAPI [25], SQLAlchemy 2.0 async [26]) — REST API и бизнес-логика;
- База данных (PostgreSQL 15 [27]) — постоянное хранилище.

SPA взаимодействует с Backend API по HTTPS (REST/JSON, JWT в заголовке Authorization); API обращается к базе данных через асинхронный драйвер asynсpg, а к внешним системам — по HTTPS (курсы ЦБ) и SMTP (почта).

2.2.3. Диаграмма компонентов (уровень 3)

Внутреннее устройство контейнера Backend API показано на рисунке 2.3.

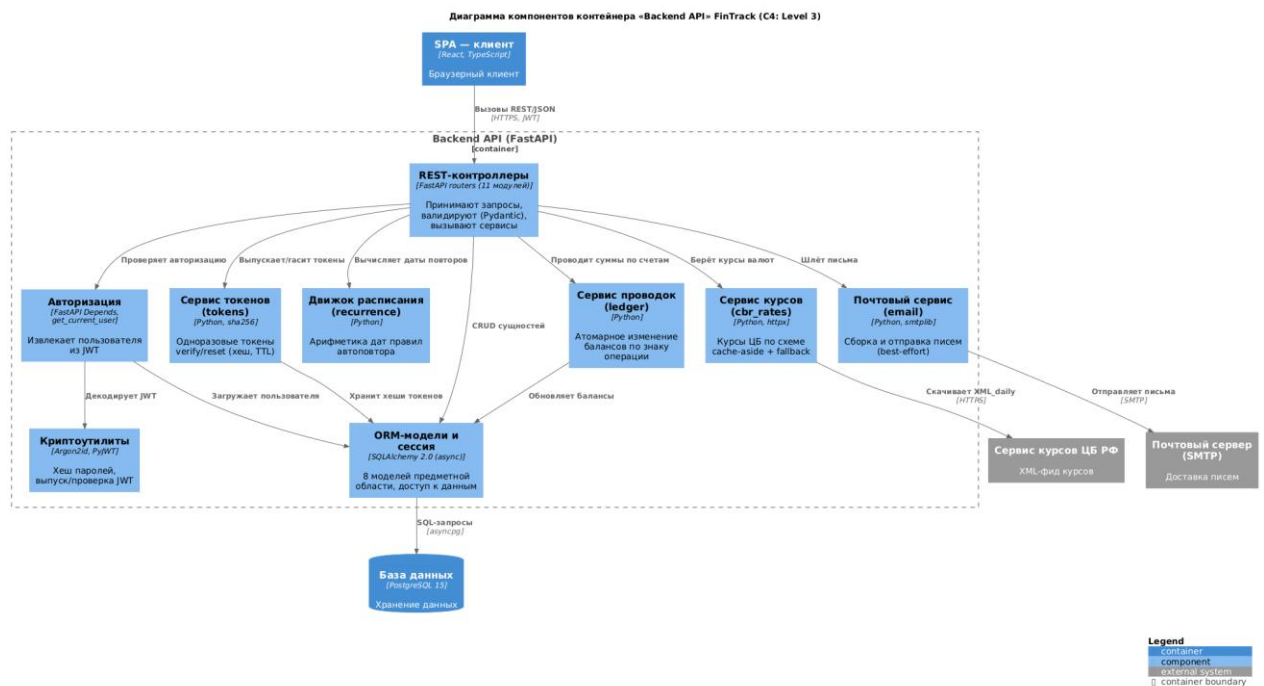


Рисунок 2.3 – Диаграмма компонентов контейнера «Backend API» (C4, уровень 3)

Backend API организован по слоям:

- REST-контроллеры (11 модулей-роутеров) принимают HTTP-запросы, валидируют входные данные средствами Pydantic и делегируют работу сервисам (пример — листинг A.4, приложение A);

- компонент авторизации (`get_current_user`) извлекает пользователя из JWT;
- сервисный слой: `ledger` (проводка балансов), `recurrence` (расписание повторяющихся операций), `cbr_rates` (курсы валют по схеме `cache-aside`), `email` (отправка писем), `tokens` (одноразовые токены);
- криптоутилиты (`security`) — хеширование паролей `Argon2id` и работа с JWT;
- слой данных — ORM-модели `SQLAlchemy` и сессия, обеспечивающие доступ к БД.

2.2.4. Проектирование REST API

Спроектированный REST API объединяет ресурсы предметной области; все защищённые маршруты требуют JWT-аутентификации. Сводка ресурсов приведена в таблице 2.2.

Таблица 2.2 – Ресурсы REST API (всего 39 эндпоинтов)

Префикс	Назначение	Кол-во методов
/api/v1/auth	Регистрация, вход, подтверждение/сброс/смена пароля	7
/api/v1/users	Профиль (/me) и удаление учётной записи	2
/api/v1/accounts	CRUD счетов	5
/api/v1/categories	CRUD категорий (иерархия)	5
/api/v1/transactions	CRUD операций, фильтры, кросс-валютный перевод	5
/api/v1/budgets	Бюджеты по категориям на месяц	4
/api/v1/dashboard	Сводка для главной страницы (/summary)	1
/api/v1/reports	Сводные отчёты (/overview)	1
/api/v1/recurring-transactions	Правила автоповтора и движок (/run)	6
/api/v1/export	Экспорт операций в CSV	1
/api/v1/rates	Курсы валют ЦБ РФ	2

Спецификация API публикуется в формате OpenAPI 3.0 [36] и доступна через интерактивный Swagger UI (см. раздел 4 и приложения).

2.3. Проектирование базы данных

2.3.1. Концептуальная и логическая модели данных

Концептуальная модель данных, отражающая восемь сущностей предметной области и связи между ними, разработана в разделе 1 (рисунок 1.2). На её основе построена логическая модель: определены атрибуты, первичные и внешние ключи, выполнена нормализация до третьей нормальной формы (3НФ). Сводка таблиц логической модели приведена в таблице 2.3.

Таблица 2.3 – Логическая модель данных (основные поля)

Таблица	Ключевые атрибуты	Связи (внешние ключи)
users	id (PK), email (UNIQUE), password_hash, email_verified	—
accounts	id (PK), name, kind, currency_code, opening_balance, opening_date, balance	owner_id → users
categories	id (PK), name, kind, parent_id	owner_id → users; parent_id → categories
transactions	id (PK), kind, amount, target_amount, currency_code, occurred_at, note	owner_id → users; account_id → accounts; transfer_account_id → accounts; category_id → categories
budgets	id (PK), amount, period_year, period_month	owner_id → users; category_id → categories
recurring_transactions	id (PK), name, kind, amount, frequency, interval, start_at, end_at, next_run_at, is_active	owner_id → users; account_id, transfer_account_id → accounts; category_id → categories
exchange_rates	id (PK), char_code, nominal, value, vunit_rate, rate_date	— (справочник)
email_tokens	id (PK), token_hash, purpose, expires_at, used_at	owner_id → users

Модель соответствует 3НФ: каждая таблица имеет первичный ключ, неключевые атрибуты зависят только от первичного ключа, транзитивные зависимости устранены. Сознательно допущены две денормализации, обоснованные практическими соображениями:

1. Код валюты хранится строкой ISO 4217 (currency_code) непосредственно в счёте и операции, без отдельной таблицы-справочника валют — упрощение модели и запросов для персонального масштаба;
2. Текущий баланс счёта (balance) хранится как кешированное производное значение (равное opening_balance плюс сумма движений) — денормализация ради скорости чтения страницы счетов; целостность кеша поддерживается при всех изменениях операций.

2.3.2. Физическая модель данных и обоснование выбора СУБД

Физическая модель учитывает особенности конкретной СУБД: типы данных (NUMERIC(15,2) для денег, TIMESTAMPTZ для дат, VARCHAR с ограничениями), индексы (по owner_id, occurred_at, внешним ключам), ограничения целостности (CHECK, UNIQUE, в т.ч. уникальный индекс с опцией nulls_not_distinct). Физическая модель сгенерирована из реальной схемы базы данных (рисунок 2.4); полный DDL-скрипт приведён в приложении А (листинг А.1).

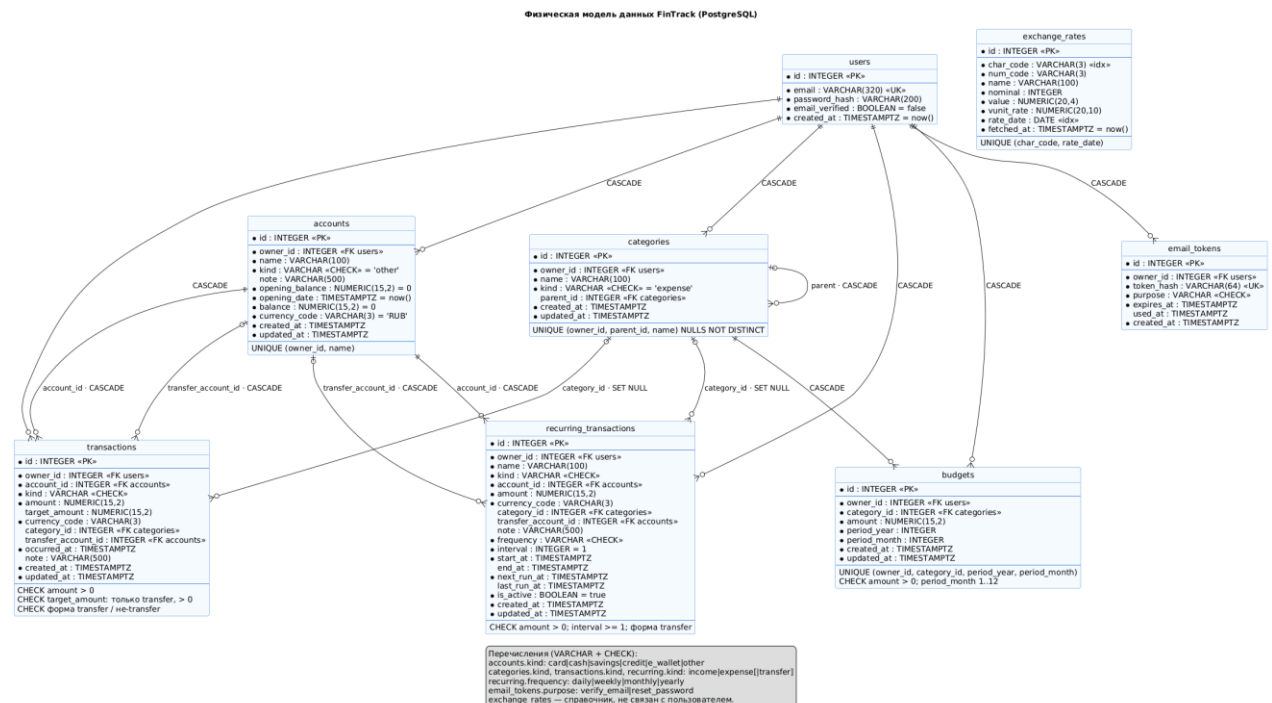


Рисунок 2.4 – Физическая модель данных FinTrack

Обоснование выбора СУБД. В качестве СУБД выбрана PostgreSQL 15. Ключевые аргументы:

- ACID-транзакции — критичны для финансовой логики (атомарность обновления балансов при переводе между счетами);
- тип `'NUMERIC'` обеспечивает точное представление денежных сумм без погрешностей чисел с плавающей точкой;
- богатый набор ограничений целостности (CHECK, UNIQUE, частичные индексы, опция `nulls_not_distinct` в версии 15) — позволяет вынести инварианты на уровень БД как «последнюю линию обороны»;
- зрелость, надёжность и открытая лицензия.

Нереляционные СУБД (например, MongoDB) отклонены: предметная область сильно реляционная (счета, операции, категории, бюджеты связаны внешними ключами), а требования к целостности и транзакционности — первостепенны.

2.4. Проектирование пользовательского интерфейса

2.4.1. Прототипы экранных форм

Поскольку приложение реализовано в полном объёме, в качестве прототипов экранных форм используются снимки экрана работающего приложения. Ключевые экраны — дашборд, список счетов, форма операции, страница отчётов — приведены в приложении (Приложение В). Ниже показаны основные из них (рисунки 2.5–2.6).

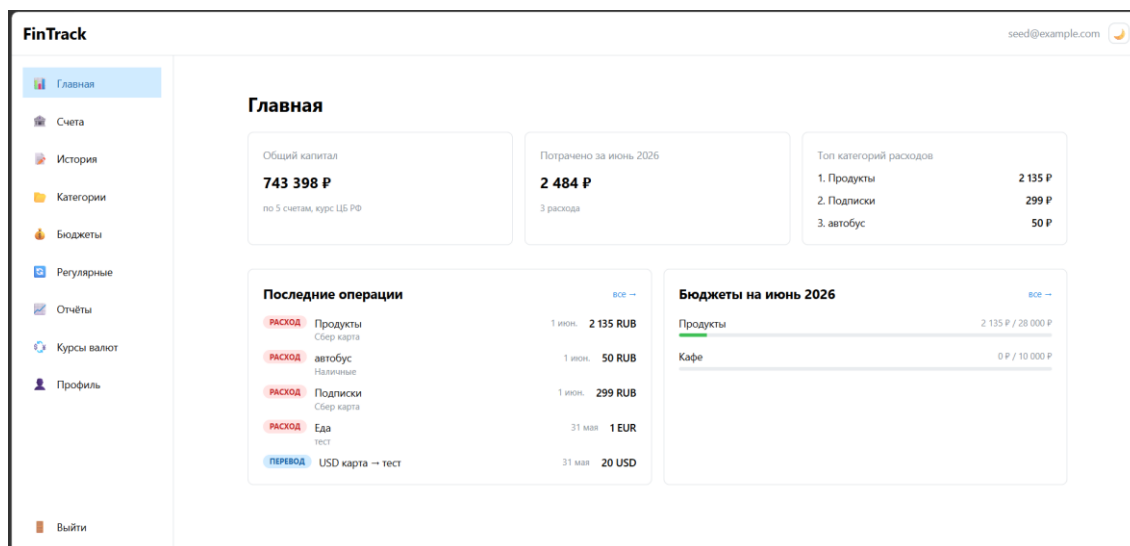


Рисунок 2.5 – Главная страница (дашборд): общий капитал, расходы за месяц, бюджеты

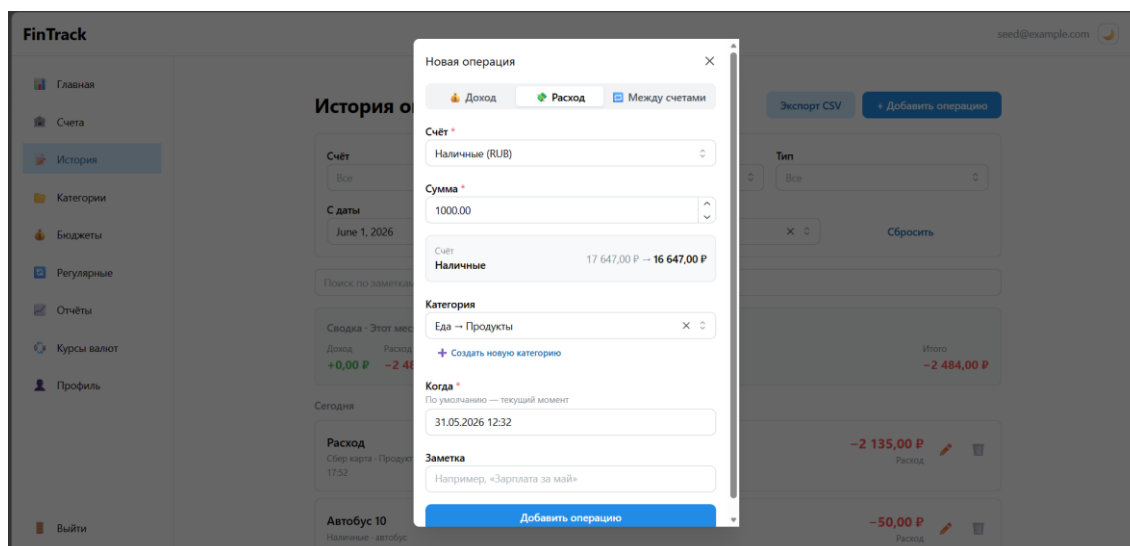


Рисунок 2.6 – Форма создания операции с предпросмотром изменения балансов

2.4.2. Описание навигации и пользовательских сценариев

Навигация организована вокруг бокового меню с разделами: «Главная», «Счета», «История», «Категории», «Бюджеты», «Регулярные», «Отчёты», «Курсы валют», «Профиль». Типовой пользовательский поток соответствует концепции TO-BE (раздел 1.3.3): после входа пользователь попадает на дашборд, откуда переходит к ведению операций, настройке бюджетов и просмотру отчётов. Формы реализованы в виде модальных окон с валидацией

на стороне клиента и предпросмотром результата (например, изменения балансов при создании операции).

2.5. Детальное проектирование (уровень кода)

2.5.1. Диаграмма классов ключевого компонента

Для ядра предметной логики — учёта операций и проводки балансов — построена диаграмма классов (рисунок 2.7). Она отражает модели предметной области (Account, Transaction, Category), наследуемые от декларативной базы SQLAlchemy, и сервис проводок ledger.

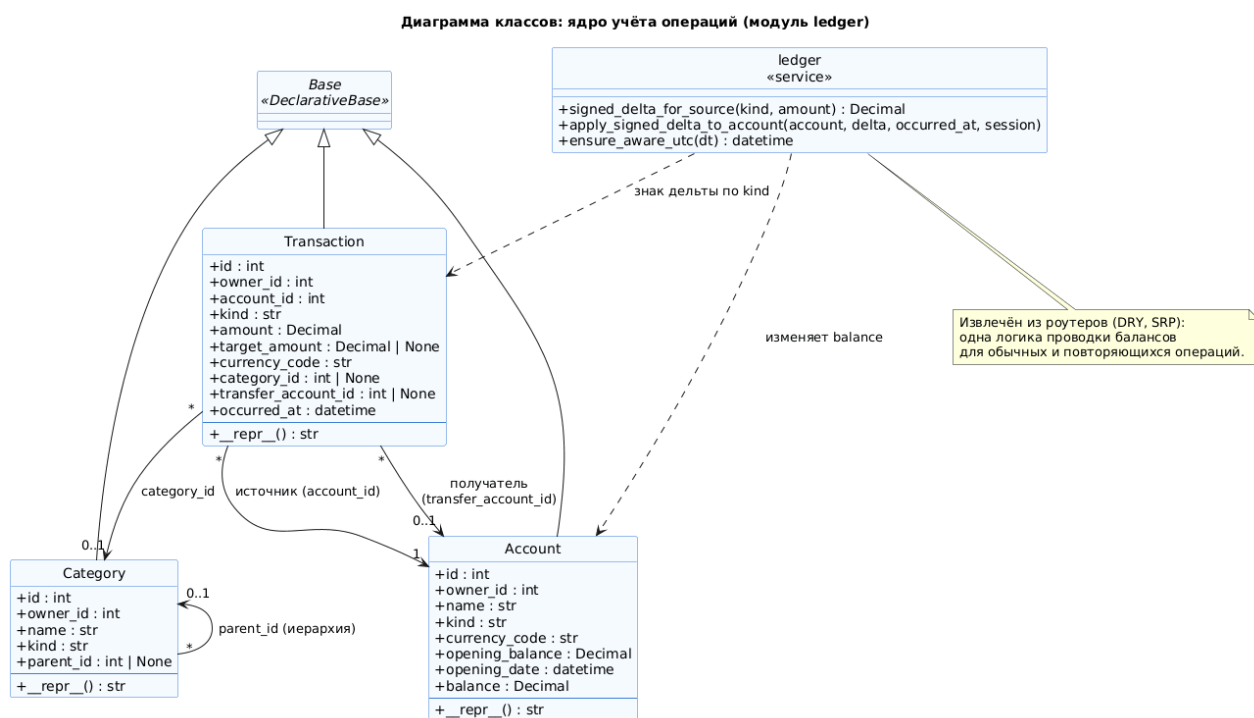


Рисунок 2.7 – Диаграмма классов ядра учёта операций (модуль ledger)

Логика проводки балансов вынесена из контроллеров в отдельный сервис ledger (принципы единственной ответственности и DRY): один и тот же код применяет изменения балансов как для обычных операций, так и для движка повторяющихся операций.

2.5.2. Диаграмма последовательности для ключевого сценария

Для наиболее сложного сценария — кросс-валютного перевода (прецедент UC-2) — построена диаграмма последовательности (рисунок 2.8), показывающая взаимодействие объектов во времени.

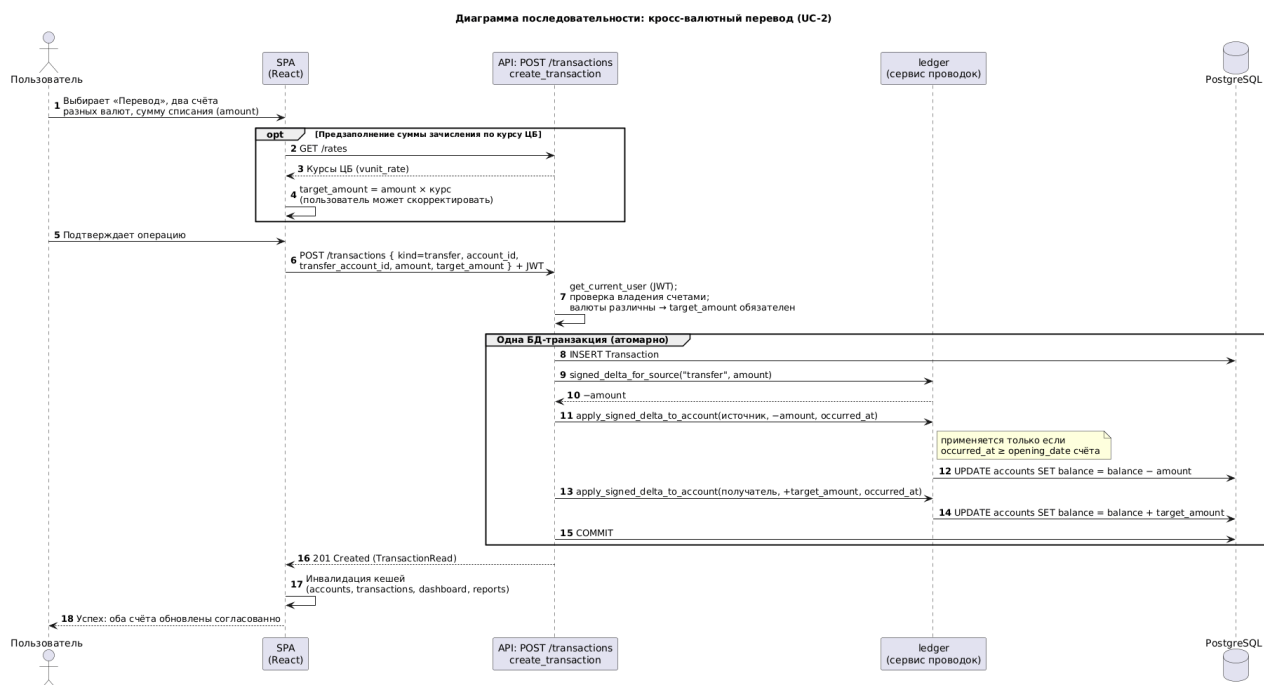


Рисунок 2.8 – Диаграмма последовательности «Кросс-валютный перевод»

Сценарий демонстрирует ключевое архитектурное решение — атомарность: создание операции и обновление балансов двух счетов выполняются в одной транзакции БД; при сбое любого шага база остаётся в согласованном состоянии.

2.5.3. Ключевые архитектурные решения

В процессе проектирования приняты следующие решения, существенные для корректности финансовой логики:

1. Модель «начальный остаток + движения». Счёт хранит начальный остаток на дату и кешированный текущий баланс; операции с датой ранее «даты остатка» сохраняются в истории, но не изменяют баланс (их эффект уже учтён в начальном остатке). Решение позволяет импортировать данные с произвольной даты (аналогично GnuCash, Beancount).

2. Положительная сумма и знак по типу операции. В БД сумма всегда положительна, а знак вычисляется по виду операции (доход — плюс, расход и перевод — минус). Это упрощает агрегаты (без CASE) и исключает ошибки знака.

3. Атомарность изменения балансов — все операции «создать/удалить операцию + обновить баланс(ы)» выполняются в одной транзакции БД [24]; для перевода атомарно обновляются два счёта.

4. Защита от IDOR через ответ 404. При обращении к чужому ресурсу возвращается 404 Not Found, что не раскрывает факт существования чужих ресурсов.

5. Cache-aside для курсов ЦБ. Курсы кешируются в БД; критерий свежести — момент последнего успешного обращения к ЦБ РФ, что исключает повторные запросы в выходные дни. При недоступности источника отдаётся последний известный курс.

2.5.4. Применение шаблонов проектирования (GoF и смежные)

При проектировании применён ряд шаблонов проектирования (детальные примеры кода — в разделе 3). Классические шаблоны «банды четырёх» (GoF) [22]:

- Одиночка (Singleton) — единственные на процесс экземпляры объекта конфигурации (settings) и хешера паролей (PasswordHasher);
- Стратегия (Strategy) — выбор алгоритма проводки по типу операции (функция signed_delta_for_source).

Корпоративные шаблоны уровня приложения (по М. Фаулеру [21]), дополняющие GoF:

- Cache-Aside (близок к структурному GoF-шаблону «Заместитель») — управление кешем курсов валют в сервисе cbr_rates;
- Внедрение зависимостей (Dependency Injection) — механизм Depends фреймворка FastAPI предоставляет сессию БД и текущего пользователя в обработчики;

– Объект передачи данных (DTO) — Pydantic-схемы запросов и ответов, отделённые от ORM-моделей, что предотвращает утечку внутренних полей (например, хеша пароля).

2.6. Обоснование выбора стека технологий

Обоснование выбора ключевых технологий приведено в таблице 2.4.

Таблица 2.4 – Обоснование выбора технологического стека

Технология	Назначение	Обоснование выбора
Python 3.13 + FastAPI	Backend, REST API	Асинхронность из коробки, автогенерация OpenAPI, валидация Pydantic, строгая типизация; легче Django, полнее «голового» Flask
SQLAlchemy 2.0 (async) + Alembic	ORM и миграции	Зрелый async-ORM, версионирование схемы миграциями
PostgreSQL 15	СУБД	ACID, NUMERIC, богатые ограничения целостности (см. 2.3.2)
React 19 + TypeScript	Frontend SPA	Крупнейшая экосистема, типобезопасность
Mantine 9	UI-компоненты	Готовые формы, модальные окна, DateTimePicker, тёмная тема
TanStack Query	Серверное состояние	Кеширование, инвалидация, отказ от ручного стора для серверных данных
Zustand	Клиентское состояние	Минимальный стор для JWT-токена
Docker + Docker Compose	Развёртывание	Воспроизводимое self-hosted-окружение одной командой

ВЫВОДЫ ПО РАЗДЕЛУ 2

В результате выполнения второго раздела спроектирована архитектура программного обеспечения FinTrack.

1. Обоснован выбор клиент-серверной трёхуровневой архитектуры (SPA + REST API + СУБД) как наилучшим образом удовлетворяющей требованиям проекта (открытый API, разделение слоёв, простота развёртывания, персональный масштаб).

2. Архитектура смоделирована по методологии C4 на трёх уровнях (контекст, контейнеры, компоненты); спроектирован REST API из 39 эндпоинтов с документацией OpenAPI 3.0.

3. Спроектирована база данных: логическая модель нормализована до 3НФ с двумя обоснованными денормализациями; обоснован выбор PostgreSQL; определена физическая модель (типы, индексы, ограничения целостности).

4. Выполнено детальное проектирование на уровне кода: построены диаграммы классов и последовательности для ключевых сценариев, зафиксированы пять ключевых архитектурных решений и применённые шаблоны проектирования (Singleton, Cache-Aside, Strategy, DI, DTO).

5. Обоснован выбор технологического стека.

Спроектированная архитектура является достаточной основой для перехода к этапу конструирования (раздел 3).

РАЗДЕЛ 3. КОНСТРУИРОВАНИЕ (РЕАЛИЗАЦИЯ) ПРОГРАММНОГО ПРОДУКТА

В настоящем разделе описывается, как архитектурные решения раздела 2 воплощены в коде: организация процесса разработки, структура проекта, реализация ключевых компонентов, соблюдение принципов качественного кода (SOLID, GoF, Clean Code), интеграция с внешними сервисами, обеспечение безопасности и управление версиями. Полные листинги вынесены в приложение; в тексте приводятся только ключевые фрагменты. Исходный код проекта доступен в публичном репозитории (ссылка приведена в подразделе 3.1.3).

3.1. Организация процесса разработки

3.1.1. Выбор и обоснование методологии разработки

В условиях индивидуальной разработки применён гибкий итеративно-инкрементальный подход, близкий к Kanban: работа велась небольшими законченными инкрементами («одна функция — один цикл реализация → проверка → коммит»), без жёстких спринтов; организация работ опиралась на процессы жизненного цикла ПО по ГОСТ Р ИСО/МЭК 12207 [7]. Каждый инкремент доводился до работоспособного и протестированного состояния, что позволяло поддерживать продукт постоянно готовым к демонстрации.

3.1.2. Инструменты управления задачами

Для планирования и отслеживания прогресса использовался список задач (бэклог функциональности) с приоритизацией по близости к ядру предметной области (деньги и балансы — в первую очередь). Учёт прогресса вёлся в привязке к коммитам репозитория.

3.1.3. Организация репозитория и системы контроля версий

Исходный код ведётся в системе контроля версий Git, репозиторий размещён на платформе GitHub в публичном доступе по адресу:

<https://github.com/cupke/financemanagement>. Репозиторий создан в начале работы; на протяжении всей разработки выполнялись регулярные коммиты с осмысленными сообщениями. Файл README.md в корне содержит название и назначение проекта, стек технологий и инструкцию по установке и запуску.

3.1.4. Стратегия ветвления

Для одиночной разработки выбрана модель, близкая к GitHub Flow: стабильной ветвью является `main`, а обособленные доработки велись в короткоживущих ветках с последующим слиянием в `main` через `merge`-коммит (с флагом `--no-ff`, что сохраняет в истории факт ветвления). Такой подход минимизирует накладные расходы на управление ветками, оправданные при командной работе, но избыточные для одного разработчика, и при этом сохраняет читаемую историю изменений. Характерный фрагмент истории репозитория, иллюстрирующий принятую стратегию ветвления, приведён на рисунке 3.1.

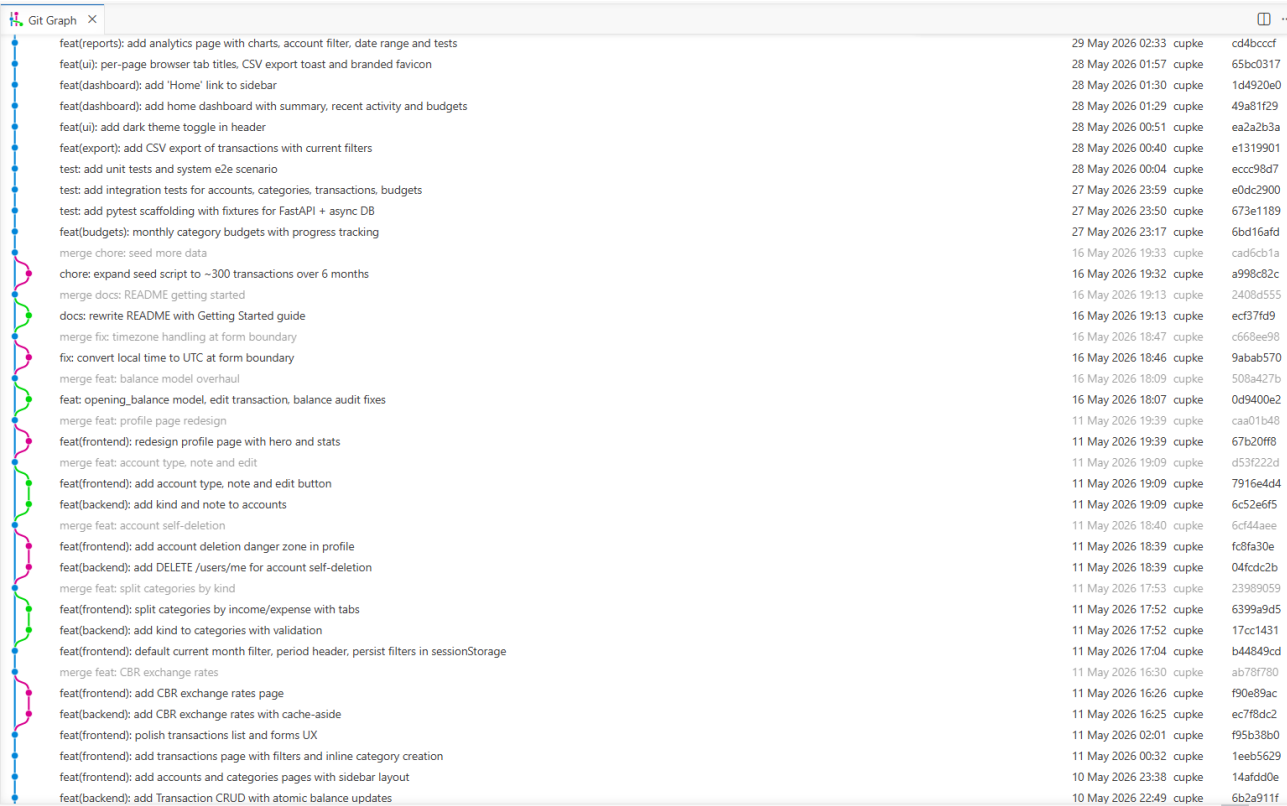


Рисунок 3.1 – Фрагмент истории репозитория: ветвление и слияние веток через `--no-ff` (GitHub Flow)

3.1.5. Правила оформления коммитов

Сообщения коммитов оформлены по соглашению Conventional Commits [37]: префикс типа (feat, fix, chore, docs, refactor), краткое описание и, при необходимости, тело с перечнем изменений. Примеры реальных сообщений:

- feat(categories): rename and move categories from the UI
- fix(recurring): serialize concurrent /run to prevent double-posting
- fix(api): tighten money precision, reports balance line, category cycles

Единый формат сообщений делает историю самодокументируемой и облегчает навигацию по изменениям.

3.2. Соответствие реализации спроектированной архитектуре

3.2.1. Сопоставление компонентов архитектуры и модулей реализации

Компоненты, выделенные на диаграмме компонентов (раздел 2.2.3), напрямую соответствуют модулям кода: REST-контроллеры — пакету app/api/v1/, сервисный слой — пакету app/services/ (ledger, recurrence, cbr_rates, email, tokens), слой данных — пакету app/db/models/, криптоутилиты — модулю app/security.py, конфигурация — app/config.py.

3.2.2. Организация файловой структуры проекта

Проект разделён на серверную и клиентскую части. Структура backend организована по слоям:

```
backend/app/
├── api/
│   ├── deps.py           # общие зависимости (get_current_user)
│   └── v1/               # REST-контроллеры (роутеры) по ресурсам
├── services/            # бизнес-логика (ledger, recurrence и др.)
├── db/
│   ├── models/          # ORM-модели (8 сущностей)
│   ├── base.py           # декларативная база SQLAlchemy
│   └── session.py        # фабрика асинхронных сессий
├── schemas/             # Pydantic-схемы (DTO запросов/ответов)
├── security.py           # Argon2id, JWT
├── config.py             # настройки (Pydantic Settings)
└── main.py              # сборка приложения, подключение роутеров
```

Клиентская часть структурирована по типам артефактов: `api/` (обёртки HTTP-вызовов), `pages/` (страницы), `components/` (переиспользуемые компоненты и модальные формы), `stores/` (состояние), `lib/` (утилиты).

3.2.3. Конфигурационные файлы и управление настройками

Конфигурация приложения вынесена в переменные окружения и считывается через `Pydantic Settings` (`app/config.py`), что обеспечивает типобезопасную валидацию настроек на старте: при отсутствии обязательной переменной (например, `JWT_SECRET_KEY` или `DATABASE_URL`) приложение завершится с понятной ошибкой ещё до приёма запросов. Шаблон переменных приведён в файле `backend/.env.example`.

3.3. Реализация ключевых компонентов

3.3.1. Сервис проводки балансов (`ledger`)

Назначение. Компонент `ledger` инкапсулирует единую логику изменения балансов счетов и используется как обычными операциями, так и движком повторяющихся операций (реализация ФТ-04, ФТ-06). Знак изменения баланса вычисляется по типу операции:

```
def signed_delta_for_source(kind: str, amount: Decimal) -> Decimal:
    if kind == "income":
        return amount          # доход — плюс
    return -amount             # расход и перевод — минус
```

Применение изменения к балансу выполняется атомарным SQL-обновлением (а не присваиванием в Python), что защищает от потери обновления при конкурентных запросах, и только если дата операции не раньше «даты остатка» счёта:

```
async def apply_signed_delta_to_account(account, delta, occurred_at,
session):
    if ensure_aware_utc(occurred_at) < account.opening_date:
        return # ретро-операция: её эффект уже учтён в начальном остатке
    await session.execute(
        update(Account).where(Account.id == account.id)
        .values(balance=Account.balance + delta)
    )
```

3.3.2. Сервис курсов валют (cbr_rates)

Назначение. Компонент `cbr_rates` реализует получение официальных курсов ЦБ РФ по шаблону `cache-aside` (реализация ФТ-10). Признаком свежести кеша служит момент последнего успешного обращения к ЦБ, а не дата курса — это исключает повторные запросы в выходные дни, когда ЦБ отдаёт курс последнего рабочего дня:

```
last_fetch = await
session.scalar(select(func.max(ExchangeRate.fetched_at)))
if last_fetch is not None and last_fetch >= today_start:
    return await _load_latest_rates(session) # отдаём из кеша
try:
    feed = await fetch_cbr_feed() # промах кеша → идём в
    ЦБ
except (httpx.HTTPError, ValueError, ET.ParseError):
    fallback = await _load_latest_rates(session) # ЦБ недоступен →
    последний кеш
    if fallback:
        return fallback
    raise
await _save_rates(session, feed)
```

Значения курсов парсятся из XML в кодировке `windows-1251` и преобразуются в тип `Decimal` (а не `float`), что исключает погрешности при пересчёте валют.

3.3.3. Движок повторяющихся операций (recurrence)

Назначение. Компонент `recurrence` содержит чистую арифметику дат расписания (при её разработке использованы общие приёмы построения алгоритмов [23]). Логика алгоритмов представлена листингами кода (приложение А), а не блок-схемами по ГОСТ 19.701 [8]: листинги выбраны как более выразительная форма для объектно-ориентированной реализации. Эндпоинт `/gun` материализует все назревшие операции (реализация ФТ-09). Чистота функций расписания (отсутствие обращений к БД) делает их тривиально тестируемыми. При конкурентном вызове `/gun` применяется блокировка на уровне БД (`pg_advisory_xact_lock` по идентификатору пользователя), сериализующая прогоны одного пользователя и исключающая двойное начисление.

3.3.4. Слой доступа к данным

Доступ к данным реализован средствами асинхронного ORM SQLAlchemy 2.0. Репозиторный подход воплощён через сессию и функции-геттеры с проверкой владельца (например, `_get_owned_account_or_404`), что одновременно решает задачу авторизации на уровне данных (см. 3.6.1).

3.4. Демонстрация применения принципов качественного кода

3.4.1. Принципы SOLID

Архитектура и код придерживаются принципов SOLID [20]; ниже для каждого принципа приведён пример из реализации FinTrack.

- S — Единственная ответственность (Single Responsibility). Логика проводки балансов вынесена из контроллеров в отдельный сервис ledger (листинг A.2, приложение A); модуль security отвечает только за криптографию (хеш паролей, JWT). Каждый роутер отвечает за один ресурс.

- O — Открытость/закрытость (Open/Closed). Бизнес-правила изолированы в сервисах: изменение правил проводки или расписания затрагивает один модуль, а не контроллеры; добавление нового типа счёта (перечисление AccountKind) не требует изменения логики проводки, так как она зависит от типа операции, а не счёта.

- L — Подстановка Лисков (Liskov Substitution). Принцип соблюден в иерархии Pydantic-схем: производная схема BudgetWithProgress наследует базовую BudgetRead, расширяя её вычисляемыми полями (потрачено, процент использования, статус, имя категории) и не сужая контракт базовой схемы, поэтому везде, где ожидается BudgetRead, корректно работает и BudgetWithProgress. Аналогично однотипные хелперы проверки владельца (`_get_owned_account_or_404`, `_get_owned_category_or_404` и др.) реализуют единый контракт «вернуть свою сущность или 404», что обеспечивает их взаимозаменяемость для вызывающего кода.

- I — Разделение интерфейсов (Interface Segregation). Для каждой операции с сущностью определены отдельные Pydantic-схемы (...Create, ...Read,

...Update) вместо одной «толстой» модели. Клиент зависит только от нужных полей; в частности, схема UserRead структурно не содержит password_hash.

– D — Инверсия зависимостей (Dependency Inversion). Обработчики запросов зависят от абстракций, поставляемых механизмом внедрения зависимостей FastAPI (Depends): сессия БД и текущий пользователь передаются извне, а не создаются внутри обработчика:

```
async def create_account(payload: AccountCreate,
                        current_user: User = Depends(get_current_user),
                        session: AsyncSession = Depends(get_session)):
    ...
```

3.4.2. Применение шаблонов проектирования (GoF и смежные)

Перечень применённых шаблонов и решаемые ими задачи приведены в разделе 2.5.4; в коде они реализованы следующим образом: Стратегия — выбор алгоритма проводки в сервисе ledger (signed_delta_for_source, раздел 3.3.1); Cache-Aside — кеширование курсов валют в cbr_rates (раздел 3.3.2); Внедрение зависимостей — механизм Depends фреймворка FastAPI (раздел 3.4.1, принцип D); Одиночка — единственные на процесс экземпляры settings и PasswordHasher; Объект передачи данных (DTO) — Pydantic-схемы, отделённые от ORM-моделей.

3.4.3. Принципы чистого кода (Clean Code)

При разработке соблюдались принципы чистого кода [18, 19]: осмысленные имена сущностей и функций, небольшие функции с единственной задачей, самодокументируемый код с комментариями только там, где требуется пояснить *почему* принято решение (а не *что* делает строка), сплошная типизация (Python type hints, TypeScript). Единый стиль кода поддерживается статическим анализатором ruff (backend) и ESLint (frontend); типобезопасность фронтенда проверяется компилятором TypeScript (tsc). На момент сдачи все проверки проходят без замечаний.

3.5. Интеграция с внешними сервисами и библиотеками

3.5.1. Интеграция с сервисом курсов ЦБ РФ

Интеграция реализована поверх HTTP-клиента `httpx`: сервис `cbr_rates` запрашивает ежедневный XML-фид `https://www.cbr.ru/scripts/XML_daily.asp` [40] (протокол HTTPS, формат XML), без авторизации (открытые данные). Обработка ошибок: при сетевой ошибке, таймауте (10 с) или некорректном XML система не падает, а возвращает последний известный кеш курсов; при полностью пустой базе запрос завершается кодом 503. Пример обработки — в 3.3.2.

3.5.2. Использование сторонних библиотек

Ключевые библиотеки `backend` и их назначение: `fastapi` (веб-фреймворк), `sqlalchemy` (ORM), `asyncpg` (драйвер PostgreSQL), `alembic` (миграции), `pydantic` (валидация), `argon2-cffi` (хеширование паролей), `PyJWT` (токены), `httpx` (HTTP-клиент). На фронтенде: `react`, `@mantine/*` (UI), `@tanstack/react-query` (серверное состояние) [31], `axios` (HTTP) [32], `zod` (валидация форм), `zustand` (состояние). Версии зафиксированы в `requirements.txt` и `package.json`.

3.6. Обеспечение безопасности и обработка ошибок

3.6.1. Аутентификация и авторизация

Аутентификация. Пароли хранятся в виде хеша `Argon2id` [34] (библиотека `argon2-cffi`, параметры по умолчанию соответствуют рекомендациям OWASP [35]). Вход выдаёт JWT [33] (алгоритм HS256), который клиент передаёт в заголовке `Authorization: Bearer:`

```
payload = {"sub": str(subject), "iat": now,
           "exp": now + timedelta(minutes=expires_minutes)}
return jwt.encode(payload, settings.jwt_secret_key, algorithm="HS256")
```

Авторизация. Зависимость `get_current_user` извлекает и проверяет токен, возвращая пользователя; защищённые маршруты подключают её через `Depends`. Изоляция данных между пользователями обеспечивается фильтрацией по владельцу: при обращении к чужому ресурсу возвращается 404 (защита от

IDOR без раскрытия факта существования ресурса). Ролевая модель (администратор/модератор) сознательно не вводится — инструмент персональный, единственная роль — владелец данных. Реализация приведена в листинге A.3 (приложение A).

3.6.2. Валидация входных данных

Валидация выполняется в два рубежа: на клиенте (схемы zod в формах) — для быстрой обратной связи, и на сервере (Pydantic) — как авторитетная проверка. Бизнес-инварианты дополнительно закреплены ограничениями БД (CHECK, UNIQUE) как «последней линией обороны». Защита от типовых уязвимостей: от SQL-инъекций — параметризованные запросы ORM (строки не конкатенируются); от XSS — экранирование вывода средствами React; от CSRF — передача токена в заголовке, а не в cookie.

3.6.3. Логирование

Используется стандартный модуль logging. Логируются значимые события уровня WARNING и выше: недоступность сервиса ЦБ РФ (с переходом на кеш), ошибки отправки писем. Сообщения не раскрывают чувствительных данных.

3.6.4. Обработка исключительных ситуаций

Ошибки бизнес-уровня возвращаются как HTTPException с понятными, но не раскрывающими внутреннюю структуру сообщениями и корректными кодами: 400 (ошибка валидации входа), 401 (не аутентифицирован), 404 (ресурс не найден/чужой), 409 (конфликт уникальности), 422 (ошибка схемы), 503 (внешний сервис недоступен). Конфликты уникальности БД (IntegrityError) перехватываются и преобразуются в 409 вместо аварийного 500. Отправка сопровождающих писем (подтверждение почты при регистрации, сброс пароля) выполняется по принципу best-effort: сбой почты не прерывает основную операцию. Исключение — явный запрос пользователя на повторную отправку

письма: в этом случае при недоступности SMTP возвращается код 502, чтобы не показывать ложный статус «письмо отправлено».

3.7. Версионность и управление конфигурацией

3.7.1. Версионирование продукта

Версионирование выполняется по схеме семантического версионирования (SemVer, MAJOR.MINOR.PATCH); текущая версия продукта — 0.1.0 (указана в метаданных приложения). Историю изменений на текущем этапе отражает Git-лог с сообщениями в формате Conventional Commits; ведение отдельного файла CHANGELOG.md отнесено к направлениям развития.

3.7.2. Анализ истории репозитория

Репозиторий содержит более 60 коммитов, отражающих последовательную разработку функциональности и последующий цикл аудита качества с исправлениями. Все сообщения оформлены по Conventional Commits (примеры — в 3.1.5), что делает граф истории (рисунок 3.1) читаемым и пригодным для аудита изменений.

3.7.3. Управление зависимостями

Версии зависимостей зафиксированы: backend — в requirements.txt, frontend — в package.json (и package-lock.json). Это обеспечивает воспроизводимость сборки.

3.7.4. Управление конфигурацией окружения

Настройки для разных окружений задаются переменными окружения (файл .env, не включаемый в репозиторий; шаблон — .env.example). Воспроизводимость окружения обеспечивается контейнеризацией: docker-compose.yml поднимает СУБД и backend одной командой, что важно для self-hosted-развёртывания (подробнее — в разделе 4).

ВЫВОДЫ ПО РАЗДЕЛУ 3

В результате выполнения третьего раздела реализован программный продукт в соответствии с архитектурой, спроектированной в разделе 2.

1. Процесс разработки организован по гибкому инкрементальному подходу с ведением репозитория Git (GitHub), стратегией ветвления, близкой к GitHub Flow (слияние через --no-ff), и сообщениями коммитов в формате Conventional Commits.

2. Реализованы ключевые компоненты: сервис проводки балансов ledger, сервис курсов cbr_rates (cache-aside), движок повторяющихся операций, слой доступа к данным; их реализация соответствует компонентам диаграммы уровня 3 (компоненты, раздел 2) и покрывает функциональные требования раздела 1.

3. Продемонстрировано соблюдение принципов SOLID (с конкретным примером для каждого принципа), применение шаблонов проектирования (Singleton, Cache-Aside, Strategy, DI, DTO) и принципов чистого кода (контроль стиля — ruff, ESLint, tsc).

4. Реализована интеграция с внешним сервисом курсов ЦБ РФ с устойчивой обработкой ошибок; обеспечена безопасность (Argon2id, JWT, защита от IDOR, валидация на двух рубежах, защита от SQL-инъекций/XSS/CSRF) и единообразная обработка исключений.

5. Налажены версионирование (SemVer), управление зависимостями и конфигурацией окружения, контейнеризация для воспроизводимого развёртывания.

Реализованная функциональность полностью покрывает функциональные требования раздела 1; продукт готов к этапу тестирования и оценки качества (раздел 4). Руководство по установке и эксплуатации приложения приведено в приложении Б.

РАЗДЕЛ 4. ТЕСТИРОВАНИЕ, ОБЕСПЕЧЕНИЕ КАЧЕСТВА И СОПРОВОЖДЕНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

В настоящем разделе описана стратегия тестирования FinTrack, результаты тестовых активностей, анализ выявленных дефектов и мероприятия по подготовке продукта к сопровождению.

4.1. Стратегия тестирования программного продукта

4.1.1. Цели и задачи тестирования

Цели тестирования: подтвердить соответствие реализации функциональным и нефункциональным требованиям (раздел 1), выявить и устранить дефекты, оценить качество и готовность продукта к эксплуатации. Особое внимание уделено корректности денежной логики (балансы, переводы, точность сумм), как наиболее критичной для финансового приложения.

4.1.2. Критерии начала и окончания тестирования

Критерий начала — реализованный и собираемый компонент с зафиксированным интерфейсом. Критерий окончания — успешное прохождение всех автоматических тестов на целевой СУБД, отсутствие нерешённых дефектов критической и значительной критичности, достижение целевого уровня покрытия кода тестами.

4.1.3. Выбор уровней тестирования

Применены три уровня: модульное (отдельные функции и классы в изоляции), интеграционное (взаимодействие контроллеров, сервисов и БД) и системное (сквозные пользовательские сценарии end-to-end). Приёмочное тестирование выполнялось неформально — ручной проверкой соответствия пользовательским ожиданиям.

4.1.4. Выбор типов тестирования

Проведены: функциональное (позитивные и негативные сценарии), тестирование безопасности (аутентификация, авторизация, изоляция данных), нагрузочное (оценка времени отклика под нагрузкой), UI/UX и тестирование совместимости (ручная проверка в браузерах), регрессионное (защита от повторного появления исправленных дефектов).

4.1.5. Инструментарий тестирования

- pytest + pytest-asyncio — фреймворк модульного и интеграционного тестирования;
- httpx (ASGI-транспорт) — вызовы API без поднятия сетевого сервера;
- pytest-cov — измерение покрытия кода;
- Locust [39] — нагрузочное тестирование;
- ruff, ESLint, компилятор TypeScript — статический анализ кода.

4.1.6. Организация тестовой среды

Тесты выполняются на отдельной тестовой базе данных PostgreSQL; схема создаётся перед прогоном и удаляется после. Внешние зависимости изолируются заглушками: отправка писем подменяется тестовым перехватчиком (monkeypatch), что позволяет проверять факт и содержимое писем без реального SMTP. Структура тестового проекта зеркалирует структуру основного кода.

4.2. Модульное тестирование

4.2.1. Фреймворк и организация

Для модульного тестирования выбран pytest — стандарт де-факто в экосистеме Python, с лаконичным синтаксисом утверждений и поддержкой асинхронных тестов через pytest-asyncio. Тесты сгруппированы по компонентам в каталоге backend/tests/; используется соглашение об именовании test_<условие>_<ожидаемый результат>.

4.2.2. Объекты модульного тестирования

Модульными тестами покрыты чистые компоненты бизнес-логики, не требующие БД:

- криптоутилиты (security): хеширование и проверка пароля, выпуск и декодирование JWT (test_unit_security);
- расчёты бюджетов (test_unit_budgets);
- арифметика расписания повторяющихся операций — функция next_occurrence с проверкой переноса дня в конце месяца и отсутствия «сползания» даты.

Пример модульного теста (проверка переноса конца месяца):

```
def test_next_occurrence_clamps_month_end():
    jan31 = datetime(2026, 1, 31, 9, 0, tzinfo=timezone.utc)
    feb = next_occurrence(jan31, "monthly", 1)
    assert (feb.year, feb.month, feb.day) == (2026, 2, 28)
```

4.2.3. Анализ покрытия кода

Покрытие измерено инструментом pytest-cov. Достигнутый общий уровень покрытия серверного кода составил 61 % (см. рисунок 4.1). Покрытие распределено закономерно: наиболее полно покрыто ядро предметной логики — модели данных (93–97 %), Pydantic-схемы (94–100 %), сервис проводки балансов ledger (94 %), модуль безопасности security (92 %), конфигурация (100 %), арифметика расписания recurrence (85 %). Более низкое покрытие приходится на вспомогательный код и обработку редких ветвей: отправку писем (email), часть путей сервиса курсов cbr_rates и ветви обработки ошибок в контроллерах. Такое распределение приоритетно: критичные для корректности компоненты (деньги, балансы, безопасность) покрыты наиболее полно, а непокрытые участки относятся к редко исполняемым или внешне зависимым ветвям и не влияют на надёжность основной функциональности.

===== tests coverage =====				
coverage: platform linux, python 3.13.13-final-0				
Name	Stmts	Miss	Cover	Missing
app/__init__.py	4	1	75%	15
app/api/__init__.py	0	0	100%	
app/api/deps.py	27	8	70%	52-53, 57, 61-62, 65-68
app/api/v1/__init__.py	0	0	100%	
app/api/v1/accounts.py	88	51	42%	66-76, 94, 108, 128-194, 223-262, 315-329, 345-352
app/api/v1/auth.py	92	49	47%	78-107, 128-142, 163-176, 190, 195-205, 226-234, 256-271, 296, 302
app/api/v1/budgets.py	118	62	47%	61-91, 142-211, 227-233, 247-249, 264-269, 300, 330-347
app/api/v1/categories.py	83	45	46%	44-45, 62-70, 100, 130-186, 207-209, 221-228, 243-248
app/api/v1/dashboard.py	63	25	60%	78-80, 93-100, 112-135, 139-156
app/api/v1/export.py	47	24	49%	55, 57, 59, 61, 63, 69-127
app/api/v1/rates.py	23	8	65%	41-53, 72-77
app/api/v1/recurring_transactions.py	143	77	46%	59-126, 139-149, 213-215, 227-229, 235-237, 261, 282-283, 324-393, 407-409, 421-426, 435-440
app/api/v1/reports.py	196	112	43%	107, 117, 127, 131-143, 156-207, 212-348, 357, 367-376
app/api/v1/transactions.py	141	100	29%	77, 91-215, 249, 251, 253, 255, 258, 271, 300-398, 423-452, 483-488, 504-509, 518-523
app/api/v1/users.py	15	1	93%	55
app/config.py	22	0	100%	
app/db/__init__.py	0	0	100%	
app/db/base.py	3	0	100%	
app/db/models/__init__.py	9	0	100%	
app/db/models/account.py	23	1	96%	121
app/db/models/budget.py	19	1	95%	98
app/db/models/category.py	18	1	94%	104
app/db/models/email_token.py	17	1	94%	75
app/db/models/exchange_rate.py	20	1	95%	79
app/db/models/recurring_transaction.py	32	1	97%	199
app/db/models/transaction.py	25	1	96%	169
app/db/models/user.py	14	1	93%	55
app/db/session.py	8	2	75%	45-46
app/main.py	45	4	91%	70, 106-111
app/schemas/__init__.py	0	0	100%	
app/schemas/account.py	32	0	100%	
app/schemas/budget.py	27	0	100%	
app/schemas/category.py	20	0	100%	
app/schemas/rate.py	16	0	100%	
app/schemas/recurring_transaction.py	66	4	94%	61, 63, 68, 72
app/schemas/token.py	4	0	100%	
app/schemas/transaction.py	48	1	98%	93
app/schemas/user.py	35	0	100%	
app/security.py	24	2	92%	46-48
app/services/__init__.py	0	0	100%	
app/services/cbr_rates.py	100	56	44%	74, 84-126, 131-141, 151-179, 189, 197-199, 230-243
app/services/email.py	39	29	26%	28-45, 53-64, 69-77, 82-90
app/services/ledger.py	17	1	94%	33
app/services/recurrence.py	20	3	85%	69-71
app/services/tokens.py	26	13	50%	50-58, 77-101
TOTAL	1769	686	61%	
===== 122 passed in 23.26s =====				

Рисунок 4.1 — Отчёт о покрытии кода тестами (pytest-cov)

4.3. Интеграционное, системное и нагрузочное тестирование

4.3.1. Интеграционное тестирование

Интеграционные тесты проверяют совместную работу контроллеров, сервисов и базы данных: HTTP-запрос проходит полный путь (роутер → сервис → ORM → тестовая БД), а результат сверяется как по коду ответа и телу, так и по фактическому состоянию БД (например, балансам счетов после операции). Вызовы выполняются через httpx с ASGI-транспортом — без поднятия сетевого сервера, что делает тесты быстрыми и детерминированными.

Пример проверяемого инварианта: после кросс-валютного перевода баланс источника уменьшается на сумму списания, а баланс получателя увеличивается на введённую сумму зачисления; при удалении перевода оба баланса возвращаются к исходным значениям.

4.3.2. Системное тестирование

Системное тестирование охватывает сквозные пользовательские сценарии (end-to-end). Представительные тест-кейсы приведены в таблице 4.1.

Таблица 4.1 – Системные тест-кейсы (фрагмент)

ID	Сценарий	Ожидаемый результат	Статус
TC-01	Регистрация → вход → получение профиля	Выдан JWT, профиль доступен	Пройден
TC-02	Создание счёта и расхода	Баланс счёта уменьшен на сумму расхода	Пройден
TC-03	Кросс-валютный перевод между счетами разных валют	Оба счёта обновлены согласованно	Пройден
TC-04	Превышение лимита бюджета	Статус бюджета — «превышен»	Пройден
TC-05	Запуск движка повторяющихся операций (/run)	Созданы назревшие операции; повторный вызов не дублирует	Пройден
TC-06	Обращение к чужому ресурсу (IDOR)	Ответ 404, данные не раскрыты	Пройден
TC-07	Экспорт операций в CSV с фильтром	Выгружены отфильтрованные операции	Пройден
TC-08 (негатив)	Создание операции с суммой ≤ 0	Ответ 422, операция не создана	Пройден

Полный сквозной сценарий «жизненный путь пользователя» автоматизирован в тесте test_system_e2e.

4.3.3. Нагрузочное тестирование

Нагрузочное тестирование выполнено инструментом Locust (см. сценарий в листинге A.5 приложения A — loadtest/locustfile.py). Сценарий моделирует read-heavy нагрузку: каждый виртуальный пользователь регистрируется, входит, создаёт счёт и операции, после чего циклически обращается к основным эндпоинтам чтения (список операций, дашборд, отчёты, счета, курсы) с весами, отражающими частоту обращений в реальном интерфейсе.

Профиль нагрузки: 20 одновременных пользователей, длительность — около 3,5 минут, всего обработано ≈ 1946 запросов, установившаяся пропускная способность — около 9 запросов/с. Доля ошибок за весь прогон — 0 %. Динамика времени отклика и пропускной способности приведена на рисунке 4.2, результаты по эндпоинтам — в таблице 4.2.

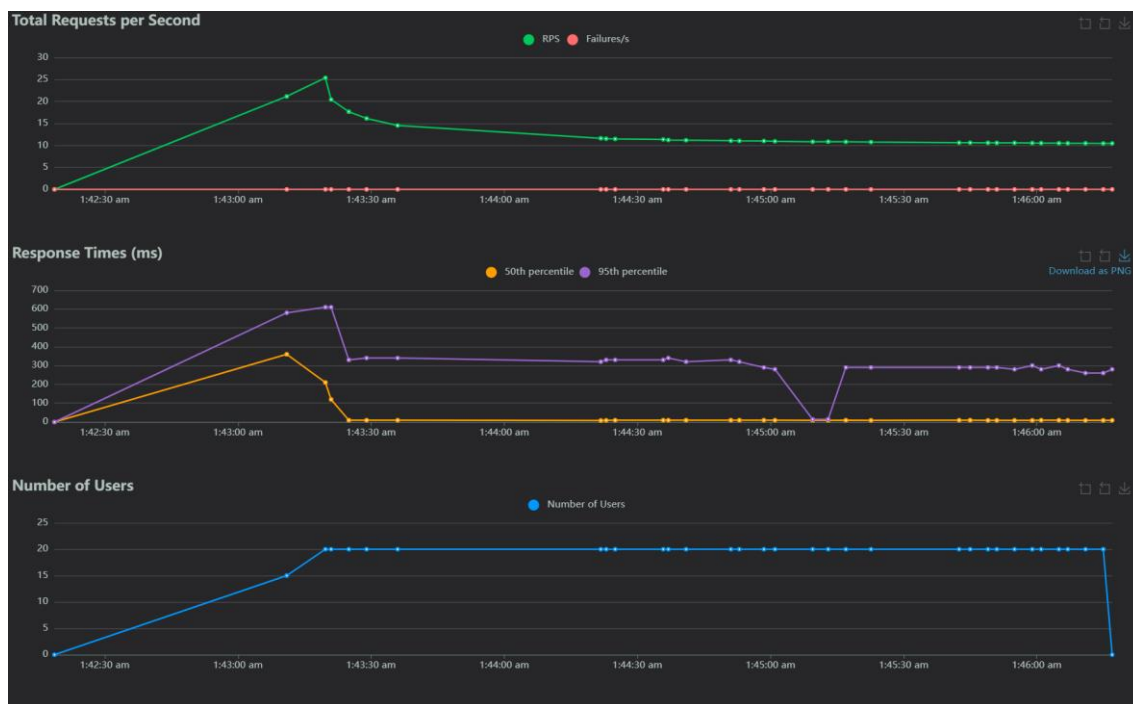


Рисунок 4.2 – Время отклика, пропускная способность и число пользователей под нагрузкой (Locust)

Таблица 4.2 – Результаты нагрузочного тестирования (по эндпоинтам)

Эндпоинт	Медиана, мс	95-й перцентиль, мс	Максимум, мс	Ошибки, %
GET /transactions	9	13	262	0
GET /dashboard/summary	9	16	85	0
GET /reports/overview	12	25	129	0
GET /accounts	8	14	475	0
GET /rates	320	380	1295	0

Основные операции чтения (список операций, дашборд, отчёты, счета) отвечают за единицы и первые десятки миллисекунд (медиана 8–12 мс, 95-й перцентиль ≤ 25 мс). Наиболее «тяжёлый» из read-эндпоинтов — GET /rates (медиана 320 мс): он отдаёт объёмный набор курсов и при промахе кеша обращается к внешнему фиду ЦБ, однако и он укладывается в доли секунды.

Операции записи ожидаемо медленнее (например, регистрация — около 550 мс), что обусловлено намеренно ресурсоёмким хешированием паролей Argon2id и не относится к горячему пути чтения.

Вывод: при нагрузке в 20 одновременных пользователей время отклика на чтение операций (95-й перцентиль 13 мс) и формирование отчётов/дашборда (95-й перцентиль 16–25 мс) с большим запасом укладываются в установленные пороги НФТ-01 (≤ 200 мс) и НФТ-02 (≤ 300 мс) при нулевой доле ошибок, что подтверждает выполнение требований к производительности для персонального масштаба эксплуатации.

4.3.4. UI/UX и тестирование совместимости

UI/UX-тестирование выполнено вручную: проверены корректность отображения и работа основных сценариев (создание операций, навигация по разделам, формы с валидацией, тёмная тема). Совместимость проверена в современных браузерах (таблица 4.3).

Таблица 4.3 – Результаты ручного тестирования совместимости

Браузер	Ключевые сценарии	Результат
Google Chrome (Chromium)	Вход, операции, отчёты, тёмная тема	Без замечаний
Mozilla Firefox	Вход, операции, отчёты, тёмная тема	Без замечаний
Microsoft Edge (Chromium)	Вход, операции, отчёты	Без замечаний

4.3.5. Тестирование безопасности

Проверены механизмы безопасности: корректность аутентификации (вход с верным/неверным паролем), изоляция данных между пользователями (обращение к чужому ресурсу возвращает 404), одноразовость и срок действия токенов подтверждения/сброса, политика паролей. Защита от SQL-инъекций обеспечивается параметризованными запросами ORM (проверено отсутствием конкатенации пользовательского ввода в запросы), от XSS — экранированием вывода во фронтенде. Перечисленные проверки автоматизированы в составе

интеграционных тестов (test_auth, test_auth_email, test_auth_hardening, проверки IDOR в тестах ресурсов).

4.4. Результаты тестирования и анализ дефектов

4.4.1. Сводные результаты тестирования

Разработано 122 автоматических теста: 16 модульных, 105 интеграционных и 1 системный сквозной. На целевой СУБД PostgreSQL проходят 100 % тестов (122 из 122). Общее покрытие кода — 61 %, при этом ядро предметной логики покрыто на 90–100 % (раздел 4.2.3). Статический анализ (ruff, ESLint, проверка типов TypeScript) проходит без замечаний.

4.4.2. Классификация дефектов

Для оценки качества проведён сквозной аудит реализации по 11 аспектам (деньги и балансы, переводы, безопасность, интеграции, повторяющиеся операции, отчёты, категории, схема БД, фронтенд, качество кода, инфраструктура). Выявленные дефекты классифицированы по критичности: значительные, незначительные, косметические.

4.4.3. Таблица найденных и исправленных дефектов

Результаты аудита и их устранение сведены в таблицу 4.4.

Таблица 4.4 – Найденные и устранённые дефекты

ID	Описание	Компонент	Критичность	Статус
D-1	Двойное начисление при конкурентном вызове /run	recurring (движок)	Значительный	Исправлен (блокировка pg_advisory_xact_lock)
D-2	Тайминг-оракул на /login (перебор email по времени отклика)	auth	Незначительный	Исправлен (проверка против фиктивного хеша)
D-3	HTTP 500 при гонке регистрации одинакового email	auth	Незначительный	Исправлен (IntegrityError → 409)

ID	Описание	Компонент	Критичность	Статус
D-4	Неатомарное гашение одноразового токена при гонке	tokens	Незначительный	Исправлен (UPDATE ... WHERE used_at IS NULL)
D-5	Отсутствие ограничения знаков после запятой у суммы	schemas	Незначительный	Исправлен (decimal_places=2)
D-6	Двойной учёт ретро-операций в линии баланса отчётов	reports	Незначительный	Исправлен (фильтр по opening_date)
D-7	Возможность создать цикл в иерархии категорий	categories	Незначительный	Исправлен (проверка предков при PATCH)
D-8	Неиспользуемый («мёртвый») код во фронтенд-API	frontend	Косметический	Исправлен (удалён)
D-9	Ложный статус «письмо отправлено» при сбое SMTP	auth/email	Косметический	Исправлен (для явного запроса повторной отправки возвращается 502)
D-10	Раскрытие занятого email при регистрации (эnumерация)	auth	Незначительный	Принят (осознанное решение, обосновано)
D-11	Передача токена в URL писем	email	Незначительный	Принят (стандартная практика, смягчена хешем и TTL)

4.4.4. Динамика обнаружения и исправления дефектов

Все дефекты выявлены в ходе единого цикла аудита и устранены за четыре последовательных коммита (по направлениям: конкурентность, безопасность, корректность/валидация, чистка), сопровождаемых регрессионными тестами. Критических дефектов в денежной логике не обнаружено.

4.4.5. Неисправленные дефекты и ограничения

Два пункта (D-10, D-11) сознательно не исправлялись: они являются не дефектами в строгом смысле, а компромиссами «удобство/безопасность» либо стандартными практиками, риск которых смягчён (одноразовость и короткий

срок жизни токенов, хранение только хеша токена). Их влияние на работоспособность системы отсутствует.

4.4.6. Выводы о качестве программного продукта

По результатам тестирования продукт обладает требуемым уровнем качества и стабильности: автоматические тесты проходят полностью, нагрузочное тестирование подтверждает требования к производительности, критические дефекты отсутствуют, выявленные в ходе аудита дефекты устранены. Продукт готов к опытной эксплуатации.

4.5. Подготовка к сопровождению (развёртывание и документация)

4.5.1. Требования к окружению эксплуатации

Программные требования: Docker и Docker Compose; для запуска без контейнеров — Python 3.13 и PostgreSQL 15. Аппаратные требования минимальны и соответствуют типовому VPS или домашнему серверу (для персонального масштаба).

4.5.2. Процесс развёртывания

Развёртывание автоматизировано: команда `docker compose up -d` поднимает СУБД и backend; применение миграций схемы выполняется командой `docker compose exec backend alembic upgrade head`. Перед первым запуском задаётся обязательная переменная `JWT_SECRET_KEY` (шаблон — `.env.example`). Автоматизация сборки и прогона тестов в CI/CD-пайплайне (например, GitHub Actions) отнесена к направлениям дальнейшего развития.

4.5.3. Пользовательская документация

Базовая пользовательская документация (назначение, установка, запуск) приведена в файле `README.md` репозитория; развёрнутое руководство пользователя со снимками экрана выносится в приложение (Приложение Б).

4.5.4. Техническая документация

API самодокументируется спецификацией OpenAPI 3.0 с интерактивным интерфейсом Swagger UI (/docs). Инструкция по сборке и конфигурированию приведена в README; архитектура описана в разделе 2.

4.5.5. План сопровождения

Предполагаемые виды работ по сопровождению: исправление ошибок, адаптация к изменениям внешнего фида ЦБ РФ, добавление новой функциональности. Рекомендуется регулярное резервное копирование тома данных PostgreSQL; обновление версий — по схеме SemVer.

4.5.6. Готовность продукта к внедрению

На основании результатов тестирования и аудита продукт признан готовым к опытной эксплуатации в режиме self-hosted. Ограничения этапа внедрения связаны с заявленными границами MVP (отсутствие многопользовательского режима, банковского импорта), не влияющими на основную функциональность.

ВЫВОДЫ ПО РАЗДЕЛУ 4

В результате выполнения четвёртого раздела проведено комплексное тестирование продукта.

1. Сформирована и реализована стратегия тестирования, охватывающая модульный, интеграционный и системный уровни, а также функциональное, нагрузочное, UI/UX и регрессионное тестирование.

2. Разработано 122 автоматических теста трёх уровней (раздел 4.4.1); на целевой СУБД проходят 100 %, общее покрытие кода — 61 % (ядро предметной логики — 90–100 %).

3. Нагрузочное тестирование (Locust, 20 одновременных пользователей, 0 % ошибок) подтвердило соответствие требованиям к производительности:

медианное время отклика операций чтения — единицы–десятки миллисекунд (НФТ-01, НФТ-02).

4. По результатам сквозного аудита выявлено и классифицировано 11 дефектов; девять устранены с добавлением регрессионных тестов, два сознательно приняты как обоснованные компромиссы. Критических дефектов в денежной логике не обнаружено.

5. Подготовлены материалы по развёртыванию (Docker, миграции) и документации (README, OpenAPI/Swagger); продукт признан готовым к опытной эксплуатации.

Результаты тестирования подтверждают выполнение функциональных и нефункциональных требований раздела 1.

РАЗДЕЛ 5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА

В настоящем разделе выполнено технико-экономическое обоснование разработки веб-приложения FinTrack: рассчитана трудоёмкость работ, определена себестоимость создания продукта и оценена экономическая эффективность его применения.

5.1. Планирование ресурсов и расчёт трудоёмкости разработки

5.1.1. Состав работ и этапов разработки

Разработка выполнялась в соответствии с жизненным циклом ПО и включала этапы: анализ требований и проектирование; написание исходного кода (кодирование); тестирование и отладку; подготовку документации; внедрение (начальный этап).

5.1.2. Метод оценки трудоёмкости

Для оценки трудоёмкости применён расчётно-аналоговый метод, основанный на условном числе операторов программы. Метод позволяет выразить трудоёмкость через объём разработанного кода и поправочные коэффициенты сложности, новизны и квалификации исполнителя, что удобно для индивидуальной разработки с известным объёмом исходного кода.

5.1.3. Расчёт трудоёмкости

Общая трудоёмкость разработки (в чел.-ч) определяется как сумма затрат труда по этапам по формуле (5.1):

$$T = T_o + T_{и} + T_a + T_{п} + T_{отл} + T_{д}, \quad (5.1)$$

где T_o — затраты на описание задачи; $T_{и}$ — на исследование предметной области; T_a — на разработку алгоритмов; $T_{п}$ — на программирование; $T_{отл}$ — на отладку; $T_{д}$ — на подготовку документации.

Большинство составляющих определяются через условное число операторов программы по формуле (5.2):

$$D = N \cdot c \cdot (1 + p), \quad (5.2)$$

где N — число операторов; c — коэффициент сложности; p — коэффициент новизны.

Исходные данные приняты на основе анализа кодовой базы проекта (около 13,7 тыс. строк исходного кода: backend, frontend и миграции) и характера задачи:

– $N = 2000$ условных операторов — оценка авторской логики без учёта комментариев, пустых строк, операторов импорта и автогенерируемых миграций (доля которых в проекте значительна ввиду подробного комментирования);

– $c = 1,25$ — повышенная сложность (финансовые расчёты, мультивалютность, безопасность, асинхронное взаимодействие);

– $p = 0,1$ — совершенно новая программа.

$$D = 2000 \cdot 1,25 \cdot 1,1 = 2750 \text{ условных операторов.} \quad (5.3)$$

Составляющие трудоёмкости рассчитаны по типовым формулам метода при следующих коэффициентах: коэффициент квалификации исполнителя $k = 0,8$ (стаж до 2 лет); коэффициент недостаточности описания $B = 1,2$; производительности по этапам: исследование $s_u = 80$, алгоритмизация и программирование $s_a = s_{\Pi} = 23$, отладка $s_{отл} = 4,5$, документирование $s_d = 18$ операторов/чел.-ч. Результаты приведены в таблице 5.1.

Таблица 5.1 – Расчёт составляющих трудоёмкости

Составляющая	Формула	Значение, чел.-ч
Описание задачи T_o	принято	40,0
Исследование предметной области T_{Π}	$D \cdot B / (s_u \cdot k)$	51,6
Разработка алгоритмов T_a	$D / (s_a \cdot k)$	149,5
Программирование T_{Π}	$D / (s_{\Pi} \cdot k)$	149,5
Отладка $T_{отл}$	$D / (s_{отл} \cdot k)$	763,9
Документирование T_d	$T_{др} + 0,75 \cdot T_{др}$	334,2
Итого (до корректировки)		1488,7

С учётом уровня языка программирования (язык высокого уровня, $k_{кор} = 0,8$) итоговая трудоёмкость по формуле (5.4):

$$T = 1488,7 \cdot 0,8 \approx 1191 \text{ чел.-ч.} \quad (5.4)$$

5.1.4. Итоговая трудоёмкость

Итоговая трудоёмкость разработки составляет 1191 чел.-ч (≈ 7 чел.-мес). С учётом частичного совмещения этапов календарная длительность разработки — около 6 месяцев.

5.1.5. Календарное планирование

Распределение этапов во времени представлено диаграммой Ганта (рисунок 5.1).

Календарный план разработки FinTrack

Диаграмма Ганта · общая трудоёмкость 1191 чел.-ч (≈ 6 месяцев)



Рисунок 5.1 – Календарный план разработки (диаграмма Ганта)

5.2. Расчёт себестоимости разработки

5.2.1. Состав статей калькуляции

Себестоимость создания продукта рассчитана по статьям: основная и дополнительная заработная плата; отчисления на социальные нужды; затраты на электроэнергию; амортизация вычислительной техники; материальные затраты; накладные расходы.

5.2.2. Затраты на оплату труда

Разработку вёл один исполнитель. При часовой тарифной ставке $C = 500$ Р/ч (соответствует рыночной ставке разработчика уровня middle в РФ) основная заработная плата по формуле (5.5):

$$ЗП_{\text{осн}} = C \cdot T = 500 \cdot 1191 = 595\,500 \text{ Р.} \quad (5.5)$$

Дополнительная заработная плата (15 % от основной) по формуле (5.6):

$$ЗП_{\text{доп}} = 0,15 \cdot 595\,500 = 89\,325 \text{ Р.} \quad (5.6)$$

$$\text{Фонд оплаты труда: } ФОТ = 595\,500 + 89\,325 = 684\,825 \text{ Р.} \quad (5.7)$$

5.2.3. Отчисления на социальные нужды

При нормативе страховых взносов 30 % по формуле (5.8):

$$О_{\text{соц}} = 0,30 \cdot \text{ФОТ} = 0,30 \cdot 684\,825 = 205\,448 \text{ Р.} \quad (5.8)$$

5.2.4. Затраты на электроэнергию

При мощности ПК $P_{\text{в}} = 0,25$ кВт, времени работы, равном трудоёмкости ($T = 1191$ ч), и тарифе $\text{Ц}_{\text{э}} = 6$ Р/кВт·ч по формуле (5.9):

$$\Xi = P_{\text{в}} \cdot T \cdot \text{Ц}_{\text{э}} = 0,25 \cdot 1191 \cdot 6 \approx 1787 \text{ Р.} \quad (5.9)$$

5.2.5. Амортизация вычислительной техники

При стоимости ПК $C_{\text{об}} = 80\,000$ Р, годовой норме амортизации $H_{\text{а}} = 20$ % и времени использования $T_{\text{исп}} = 7$ мес по формуле (5.10):

$$A = (C_{\text{об}} \cdot H_{\text{а}} \cdot T_{\text{исп}}) / (100 \cdot 12) = (80\,000 \cdot 20 \cdot 7) / 1200 \approx 9333 \text{ Р.} \quad (5.10)$$

5.2.6. Материальные затраты

Расходы на материалы (носители, расходные материалы) приняты $M = 2000$ Р.

5.2.7. Накладные расходы

При нормативе 50 % от основной заработной платы по формуле (5.11):

$$H = 0,50 \cdot 595\,500 = 297\,750 \text{ Р.} \quad (5.11)$$

5.2.8. Смета затрат на разработку

Полная себестоимость разработки сведена в таблице 5.2.

Таблица 5.2 – Смета затрат на разработку программного продукта

Статья затрат	Сумма, Р
Основная заработная плата	595 500
Дополнительная заработная плата (15 %)	89 325
Отчисления на социальные нужды (30 %)	205 448
Затраты на электроэнергию	1 787
Амортизация вычислительной техники	9 333
Материальные затраты	2 000
Накладные расходы (50 %)	297 750
Полная себестоимость `С разр`	1 201 143

Поскольку продукт разрабатывался индивидуально и распространяется свободно (открытый исходный код), оптовая и договорная цена не рассчитываются; капиталовложения при внедрении принимаются равными себестоимости разработки: $K = 1\,201\,143 \text{ Р.}$

5.3. Оценка экономической эффективности проекта

5.3.1. Модель экономического эффекта

FinTrack — бесплатный self-hosted-продукт, поэтому эффект оценивается как экономия пользователей от отказа от платной подписки на коммерческий аналог. Среднемесячная стоимость подписки на сопоставимый сервис (ZenMoney, CoinKeeper) составляет около 600 Р/мес, то есть 7200 Р/год на одного пользователя.

5.3.2. Годовой экономический эффект

С учётом постепенного роста пользовательской базы свободно распространяемого продукта принят прогноз: 50, 100, 150 и 200 пользователей по годам расчётного периода. Эксплуатационные затраты минимальны (приложение разворачивается на уже имеющемся оборудовании пользователя), поэтому годовой эффект Π_k принимается равным совокупной годовой экономии пользователей (таблица 5.3).

5.3.3. Показатели эффективности инвестиций

Расчётный период $n = 4$ года (до морального устаревания), норма дисконта $E = 0,2$. Чистый дисконтированный доход по формуле (5.12):

$$\text{ЧДД} = \sum \Pi_k / (1+E)^k - K. \quad (5.12)$$

Расчёт приведён в таблице 5.3.

Таблица 5.3 – Расчёт чистого дисконтированного дохода

Год k	Пользователей	Эффект P_k , Р	Коэф. дисконт. $1/(1+E)^k$	Дисконт. эффект, Р
1	50	360 000	0,833	299 880
2	100	720 000	0,694	499 680
3	150	1 080 000	0,579	625 320
4	200	1 440 000	0,482	694 080
Σ дисконтированных эффектов				2 118 960
Капиталовложения К				1 201 143
Чистый дисконтированный доход (ЧДД)				917 817

ЧДД положителен — проект экономически эффективен.

Индекс доходности: $PI = 2\,118\,960 / 1\,201\,143 = 1,76 (> 1)$ — проект эффективен.

Внутренняя норма доходности: значение нормы дисконта, при котором ЧДД = 0, составляет ВНД $\approx 48\%$, что значительно превышает принятую норму дисконта (20 %), — проект обладает запасом финансовой прочности.

Срок окупаемости определён методом накопления дохода (таблица 5.4): срок наступает в году, когда накопленный эффект впервые покрывает капиталовложения $K = 1\,201\,143$ Р.

Таблица 5.4 – Накопление экономического эффекта по годам, Р

Год k	Эффект (недиск.)	Накопл. (недиск.)	Эффект (диск.)	Накопл. (диск.)
1	360 000	360 000	299 880	299 880
2	720 000	1 080 000	499 680	799 560
3	1 080 000	2 160 000	625 320	1 424 880
4	1 440 000	3 600 000	694 080	2 118 960

Капиталовложения покрываются в течение третьего года:

– простой срок окупаемости $\approx 2 + (1\,201\,143 - 1\,080\,000)/1\,080\,000 \approx 2,1$ года;

– дисконтированный $\approx 2 + (1\,201\,143 - 799\,560)/625\,320 \approx 2,6$ года.

Оба срока меньше расчётного периода (4 года), что подтверждает экономическую целесообразность проекта. Следует отметить: поскольку продукт распространяется бесплатно, окупаемость носит характер совокупной

(общественной) — затраты на разработку перекрываются суммарной экономией пользовательской базы, а не возвратом средств разработчику.

5.4. Основные технико-экономические показатели проекта

Сводные показатели приведены в таблице 5.5.

Таблица 5.5 – Основные технико-экономические показатели проекта

Показатель	Ед. изм.	Значение
Трудоёмкость разработки	чел.-ч	1191
Длительность разработки (календарная)	мес.	≈ 6
Полная себестоимость разработки	Р	1 201 143
Годовой эффект (к 4-му году)	Р	1 440 000
Чистый дисконтированный доход (4 года)	Р	917 817
Индекс доходности (PI)	—	1,76
Внутренняя норма доходности (ВНД)	%	≈ 48
Срок окупаемости (простой)	год	≈ 2,1
Срок окупаемости (дисконтированный)	год	≈ 2,6

5.5. Обоснование экономической целесообразности

Сравнение «разработка против покупки» (make-or-buy): использование коммерческого аналога обошлось бы пользователям в стоимость подписки (около 7200 Р/год каждому) бессрочно, тогда как FinTrack предоставляет сопоставимую функциональность бесплатно при хранении данных под контролем пользователя. Помимо прямого экономического эффекта, проект обладает качественными эффектами: повышение приватности и соответствие 152-ФЗ, независимость от зарубежной инфраструктуры, возможность доработки сообществом благодаря открытому исходному коду и открытому API.

ВЫВОДЫ ПО РАЗДЕЛУ 5

В пятом разделе выполнено технико-экономическое обоснование разработки.

1. Рассчитана трудоёмкость разработки расчётно-аналоговым методом — 1191 чел.-ч (≈ 6 месяцев календарной длительности); построен календарный план (диаграмма Ганта).

2. Определена полная себестоимость создания продукта — 1 201 143 Р.

3. Оценка экономической эффективности по модели экономии на подписке на коммерческий аналог показала: чистый дисконтированный доход за 4 года — 917 817 Р (> 0), индекс доходности — 1,76 (> 1), внутренняя норма доходности — $\approx 48\%$ ($> 20\%$), срок окупаемости — около 2,1 года.

4. Проект экономически обоснован и целесообразен; помимо прямого эффекта, он обладает социально значимыми качественными эффектами (приватность, соответствие 152-ФЗ, открытость кода).

РАЗДЕЛ 6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА ПРИ ЭКСПЛУАТАЦИИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Эксплуатация веб-приложения FinTrack осуществляется пользователем (оператором) на персональном компьютере. В настоящем разделе проанализированы опасные и вредные факторы, воздействующие на человека при работе с ПК, и определены мероприятия по обеспечению безопасности труда, электробезопасности и пожарной безопасности на рабочем месте.

6.1. Анализ опасных и вредных факторов на рабочем месте

Классификация опасных и вредных производственных факторов выполнена в соответствии с ГОСТ 12.0.003-2015 «ССБТ. Опасные и вредные производственные факторы. Классификация» [10]. При работе оператора с персональным компьютером действуют следующие факторы.

Физические факторы:

- повышенная нагрузка на органы зрения вследствие длительной работы с экраном (мерцание, блики, недостаточная или избыточная яркость);
- электромагнитные поля и излучения от средств вычислительной техники;
- недостаточная или неравномерная освещённость рабочей зоны;
- неблагоприятные параметры микроклимата (температура, влажность, подвижность воздуха);
- повышенный уровень шума от систем охлаждения и периферии;
- опасность поражения электрическим током при работе с электрооборудованием (напряжение питания 220 В).

Психофизиологические факторы:

- статические физические перегрузки, связанные с длительным нахождением в фиксированной рабочей позе (нагрузка на опорно-двигательный аппарат, шею, кисти рук);
- нервно-психическое и эмоциональное напряжение, монотонность труда;
- перенапряжение зрительных анализаторов.

Наиболее значимыми для оператора ПК являются зрительное и статическое перенапряжение, а также опасность поражения электрическим током и пожароопасность электрооборудования.

6.2. Мероприятия по обеспечению безопасности труда

6.2.1. Организация рабочего места

Организация рабочего места оператора ПК выполняется согласно ГОСТ 12.2.032-78 «ССБТ. Рабочее место при выполнении работ сидя» [11] и требованиям СанПиН 1.2.3685-21 [16]. Основные требования:

- расстояние от глаз до экрана монитора — 600–700 мм;
- высота рабочей поверхности стола — 680–800 мм; под столом обеспечивается пространство для ног;
- кресло — подъёмно-поворотное, с регулировкой высоты сиденья и угла наклона спинки, с подлокотниками;
- экран располагается так, чтобы исключить блики от источников света; верхняя кромка экрана — на уровне или чуть ниже линии глаз;
- клавиатура размещается на расстоянии 100–300 мм от края стола для опоры предплечий.

6.2.2. Микроклимат

Оптимальные параметры микроклимата на рабочем месте оператора (категория работ Ia — лёгкие, выполняемые сидя) приведены в таблице 6.1 согласно СанПиН 1.2.3685-21.

Таблица 6.1 – Оптимальные параметры микроклимата

Период года	Температура воздуха, °С	Относительная влажность, %	Скорость движения воздуха, м/с
Холодный	22–24	40–60	0,1
Тёплый	23–25	40–60	0,1

Поддержание параметров обеспечивается системами отопления, вентиляции и кондиционирования, а также регулярным проветриванием помещения.

6.2.3. Расчёт искусственного освещения

Нормируемая освещённость рабочих мест с ПК согласно СП 52.13330.2016 «Естественное и искусственное освещение» [15] составляет $E = 300$ лк. Необходимое число светильников рассчитано методом коэффициента использования светового потока по формуле (6.1):

$$N = (E \cdot S \cdot k \cdot z) / (\Phi \cdot \eta), \quad (6.1)$$

где $E = 300$ лк — нормируемая освещённость; $S = 15 \text{ м}^2$ — площадь помещения; $k = 1,4$ — коэффициент запаса; $z = 1,1$ — коэффициент неравномерности освещения; $\Phi = 3600$ лм — световой поток одного светодиодного светильника (36 Вт); $\eta = 0,5$ — коэффициент использования светового потока (при индексе помещения ≈ 1 и коэффициентах отражения потолка/стен/пола 70/50/30 %).

$$N = (300 \cdot 15 \cdot 1,4 \cdot 1,1) / (3600 \cdot 0,5) = 6930 / 1800 \approx 4 \text{ светильника.} \quad (6.2)$$

Таким образом, для обеспечения нормируемой освещённости в помещении площадью 15 м^2 достаточно четырёх светодиодных светильников мощностью 36 Вт. Рабочее место располагается так, чтобы естественный свет падал сбоку (преимущественно слева), а искусственное освещение дополняло общее без бликов на экране.

6.2.4. Режим труда и отдыха

Для снижения зрительного и статического перенапряжения устанавливается рациональный режим труда и отдыха в соответствии с СанПиН 1.2.3685-21 [16] и Трудовым кодексом РФ [4] (ст. 108, 109): регламентированные перерывы по 10–15 минут через каждые 45–60 минут работы; суммарное время перерывов при 8-часовой смене — 50–90 минут в зависимости от категории и интенсивности работы. Во время перерывов рекомендуется выполнять упражнения для глаз, шеи и кистей рук, а также проветривать помещение. Уровень шума на рабочем месте не должен превышать 50 дБА.

6.3. Электробезопасность и пожарная безопасность

6.3.1. Электробезопасность

По степени опасности поражения электрическим током помещение с ПК относится к классу без повышенной опасности (сухое, отапливаемое, с непроводящими полами, без токопроводящей пыли). Электрооборудование питается от сети переменного тока напряжением 220 В частотой 50 Гц. Мероприятия по электробезопасности согласно ГОСТ 12.1.019-2017 [12] и ГОСТ 12.1.030-81 [13]:

- защитное заземление (зануление) корпусов электрооборудования;
- применение устройств защитного отключения (УЗО) и автоматических выключателей;
- исправность изоляции проводов и розеток, отсутствие перегрузки сети;
- запрет эксплуатации оборудования с повреждённой изоляцией и самостоятельного ремонта под напряжением;
- подключение техники через сетевые фильтры с защитой от перенапряжения.

6.3.2. Пожарная безопасность

Обеспечение пожарной безопасности регламентируется Федеральным законом № 123-ФЗ «Технический регламент о требованиях пожарной безопасности» [3] и ГОСТ 12.1.004-91 [14]. По пожарной и взрывопожарной опасности помещение с ПК относится к категории В (пожароопасное) согласно СП 12.13130.2009 [17]. Основные причины возможного возгорания — короткое замыкание, перегрузка электросети, перегрев оборудования. Мероприятия:

- оснащение помещения углекислотными (ОУ) или порошковыми (ОП) огнетушителями, пригодными для тушения электроустановок под напряжением;
- установка автоматической пожарной сигнализации;
- соблюдение правил эксплуатации электрооборудования, недопущение перегрузки сети;

- обеспечение свободных путей эвакуации и наличие плана эвакуации;
- проведение инструктажей по пожарной безопасности.

ВЫВОДЫ ПО РАЗДЕЛУ 6

В шестом разделе рассмотрены вопросы безопасности жизнедеятельности и охраны труда при эксплуатации программного обеспечения.

1. Проанализированы опасные и вредные факторы на рабочем месте оператора ПК (по ГОСТ 12.0.003-2015); наиболее значимы зрительное и статическое перенапряжение, опасность поражения током и пожароопасность.

2. Определены мероприятия по организации рабочего места, обеспечению микроклимата и режима труда и отдыха; выполнен расчёт искусственного освещения, показавший достаточность четырёх светодиодных светильников для помещения 15 м².

3. Рассмотрены меры электробезопасности (заземление, УЗО, исправность изоляции) и пожарной безопасности (огнетушители для электроустановок, сигнализация, пути эвакуации) в соответствии с действующими нормативными документами.

Соблюдение перечисленных требований обеспечивает безопасные и комфортные условия труда при эксплуатации программного обеспечения и снижает воздействие вредных факторов на здоровье пользователя.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы разработано веб-приложение FinTrack для управления личными финансами с поддержкой мультивалютного учёта по курсам ЦБ РФ и self-hosted-развёртыванием. Поставленная цель достигнута: реализован сквозной пользовательский сценарий — от регистрации и создания счетов до ведения операций, бюджетирования, автоповтора, анализа в отчётах и экспорта данных.

Все задачи, сформулированные во введении, выполнены:

1. проведён анализ предметной области и сравнительный обзор аналогов (CoinKeeper, ZenMoney, Money Lover, GnuCash), обосновавший актуальность собственной разработки;
2. сформированы и задокументированы 15 функциональных и 11 нефункциональных требований, построена модель вариантов использования;
3. спроектирована клиент-серверная архитектура с применением методологии C4 и спроектирован REST API из 39 эндпоинтов (OpenAPI 3.0);
4. спроектирована модель данных из 8 сущностей (нормализация до 3НФ, 14 миграций), обоснован выбор СУБД PostgreSQL;
5. реализована серверная часть на FastAPI с асинхронным доступом к данным;
6. реализована клиентская часть (SPA) на React + TypeScript;
7. реализована интеграция с сервисом курсов валют ЦБ РФ (паттерн cache-aside с устойчивостью к недоступности источника);
8. обеспечена безопасность (Argon2id, JWT, изоляция данных с защитой от IDOR) и соответствие 152-ФЗ (минимизация ПДн, право на забвение);
9. проведено тестирование: разработано 122 автоматических теста (модульные, интеграционные, системный), общее покрытие кода — 61 % (ядро предметной логики — 90–100 %); нагрузочное тестирование подтвердило соответствие требованиям к производительности;
10. выполнено технико-экономическое обоснование;

11. рассмотрены вопросы безопасности жизнедеятельности при эксплуатации ПО.

Полученные результаты. Создан работоспособный программный продукт, реализующий мультивалютные счета, категоризированный учёт операций трёх видов, кросс-валютные переводы, бюджеты, повторяющиеся операции, сводные отчёты, дашборд и экспорт данных. Корректность денежной логики (атомарность изменения балансов, согласованность переводов, точность расчётов) подтверждена тестированием и сквозным аудитом; критических дефектов не выявлено. Технико-экономическое обоснование показало экономическую целесообразность разработки: при себестоимости создания продукта 1 201 143 Р чистый дисконтированный доход за расчётный период составляет 917 817 Р, индекс доходности — 1,76, внутренняя норма доходности — $\approx 48\%$, срок окупаемости — около 2,1 года.

Практическая значимость. Полученное self-hosted-приложение пригодно для практического персонального и семейного учёта финансов: данные остаются под контролем пользователя (приватность, соответствие 152-ФЗ), а открытые исходный код и REST API допускают дальнейшую доработку и интеграцию.

Направления дальнейшего развития. Ряд возможностей сознательно вынесен за границы текущей версии и образует план развития продукта:

- импорт банковских выписок (CSV/OFX) и интеграция с банковскими API (open banking) для снятия ручного ввода как основного источника данных;
- многопользовательский режим с ролями и разделением учёта для полноценного семейного сценария;
- мобильный клиент (PWA или нативный);
- усиление безопасности: двухфакторная аутентификация, refresh-токены и инвалидация сессий, ограничение частоты запросов (rate-limiting);
- настраиваемая базовая валюта пользователя, системные категории «по умолчанию»;
- прогнозная аналитика расходов;

– организация CI/CD-конвейера с автоматическим прогоном тестов.

Перечисленные направления являются обоснованными границами минимально жизнеспособного продукта, а не невыполненными требованиями. Цель работы достигнута, поставленные задачи решены в полном объеме, а разработанный продукт готов к опытной эксплуатации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Нормативные правовые акты и стандарты

1. О персональных данных : Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 2024 г.). — Доступ из справ.-правовой системы «КонсультантПлюс».
2. Об информации, информационных технологиях и о защите информации : Федеральный закон от 27.07.2006 № 149-ФЗ. — Доступ из справ.-правовой системы «КонсультантПлюс».
3. Технический регламент о требованиях пожарной безопасности : Федеральный закон от 22.07.2008 № 123-ФЗ. — Доступ из справ.-правовой системы «КонсультантПлюс».
4. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ. — Доступ из справ.-правовой системы «КонсультантПлюс».
5. Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных : Постановление Правительства РФ от 01.11.2012 № 1119. — Доступ из справ.-правовой системы «КонсультантПлюс».
6. Об утверждении Правил формирования и ведения единого реестра российских программ для электронных вычислительных машин и баз данных : Постановление Правительства РФ от 16.11.2015 № 1236. — Доступ из справ.-правовой системы «КонсультантПлюс».
7. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. — Москва : Стандартинформ, 2011.
8. ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. — Москва : Изд-во стандартов, 1991.
9. IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications. — New York : IEEE, 1998.

10. ГОСТ 12.0.003-2015. Система стандартов безопасности труда. Опасные и вредные производственные факторы. Классификация. — Москва : Стандартинформ, 2016.
11. ГОСТ 12.2.032-78. Система стандартов безопасности труда. Рабочее место при выполнении работ сидя. Общие эргономические требования. — Москва : Изд-во стандартов, 1979.
12. ГОСТ 12.1.019-2017. Система стандартов безопасности труда. Электробезопасность. Общие требования и номенклатура видов защиты. — Москва : Стандартинформ, 2017.
13. ГОСТ 12.1.030-81. Система стандартов безопасности труда. Электробезопасность. Защитное заземление, зануление. — Москва : Изд-во стандартов, 1982.
14. ГОСТ 12.1.004-91. Система стандартов безопасности труда. Пожарная безопасность. Общие требования. — Москва : Стандартинформ, 2006.
15. СП 52.13330.2016. Естественное и искусственное освещение. — Москва : Минстрой России, 2016.
16. СанПиН 1.2.3685-21. Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания. — Москва : Роспотребнадзор, 2021.
17. СП 12.13130.2009. Определение категорий помещений, зданий и наружных установок по взрывопожарной и пожарной опасности. — Москва : ВНИИПО, 2009.

Литература

18. Макконнелл, С. Совершенный код. Мастер-класс / С. Макконнелл. — Москва : Русская редакция, 2017. — 896 с.
19. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. — Санкт-Петербург : Питер, 2019. — 464 с.
20. Мартин, Р. Чистая архитектура: искусство разработки программного обеспечения / Р. Мартин. — Санкт-Петербург : Питер, 2018. — 352 с.

21. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — Москва : Вильямс, 2016. — 544 с.

22. Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. — Санкт-Петербург : Питер, 2020. — 368 с.

23. Седжвик, Р. Алгоритмы на Python / Р. Седжвик, К. Уэйн. — Москва : Вильямс, 2017. — 672 с.

24. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — Санкт-Петербург : Питер, 2018. — 640 с.

Электронные ресурсы

25. FastAPI: documentation [Электронный ресурс]. — URL: <https://fastapi.tiangolo.com> (дата обращения: 31.05.2026).

26. SQLAlchemy 2.0: documentation [Электронный ресурс]. — URL: <https://docs.sqlalchemy.org> (дата обращения: 31.05.2026).

27. PostgreSQL 15: documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/15/> (дата обращения: 31.05.2026).

28. React: documentation [Электронный ресурс]. — URL: <https://react.dev> (дата обращения: 31.05.2026).

29. TypeScript: documentation [Электронный ресурс]. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 31.05.2026).

30. Mantine: React components library: documentation [Электронный ресурс]. — URL: <https://mantine.dev> (дата обращения: 31.05.2026).

31. TanStack Query: documentation [Электронный ресурс]. — URL: <https://tanstack.com/query> (дата обращения: 31.05.2026).

32. Axios: documentation [Электронный ресурс]. — URL: <https://axios-http.com> (дата обращения: 31.05.2026).

33. Jones, M. JSON Web Token (JWT) : RFC 7519 / M. Jones, J. Bradley, N. Sakimura [Электронный ресурс]. — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 31.05.2026).
34. Biryukov, A. Argon2 Memory-Hard Function for Password Hashing : RFC 9106 / A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson [Электронный ресурс]. — URL: <https://datatracker.ietf.org/doc/html/rfc9106> (дата обращения: 31.05.2026).
35. OWASP Top Ten: the standard awareness document for web application security [Электронный ресурс]. — URL: <https://owasp.org/www-project-top-ten/> (дата обращения: 31.05.2026).
36. OpenAPI Specification [Электронный ресурс]. — URL: <https://spec.openapis.org/oas/latest.html> (дата обращения: 31.05.2026).
37. Conventional Commits 1.0.0 [Электронный ресурс]. — URL: <https://www.conventionalcommits.org> (дата обращения: 31.05.2026).
38. Brown, S. The C4 model for visualising software architecture [Электронный ресурс]. — URL: <https://c4model.com> (дата обращения: 31.05.2026).
39. Locust: an open source load testing tool: documentation [Электронный ресурс]. — URL: <https://docs.locust.io> (дата обращения: 31.05.2026).
40. Банк России. Технические ресурсы. Получение данных, используя XML (ежедневные курсы валют) [Электронный ресурс]. — URL: <https://www.cbr.ru/development/SXML/> (дата обращения: 31.05.2026).

ПРИЛОЖЕНИЕ А. ЛИСТИНГИ ПРОГРАММНОГО КОДА

В приложении приведены листинги ключевых программных модулей системы FinTrack, на которые даются ссылки в тексте пояснительной записки, а также DDL-скрипт схемы базы данных.

Листинг А.1 – DDL-скрипт схемы базы данных (PostgreSQL)

```
CREATE TABLE accounts (  
    id integer NOT NULL,  
    owner_id integer NOT NULL,  
    name character varying(100) NOT NULL,  
    balance numeric(15,2) DEFAULT '0'::numeric NOT NULL,  
    currency_code character varying(3) DEFAULT 'RUB'::character varying  
NOT NULL,  
    created_at timestamp with time zone DEFAULT now() NOT NULL,  
    updated_at timestamp with time zone DEFAULT now() NOT NULL,  
    kind character varying(8) DEFAULT 'other'::character varying NOT  
NULL,  
    note character varying(500),  
    opening_balance numeric(15,2) DEFAULT '0'::numeric NOT NULL,  
    opening_date timestamp with time zone DEFAULT now() NOT NULL  
);  
  
CREATE SEQUENCE accounts_id_seq  
    AS integer  
    START WITH 1  
    INCREMENT BY 1  
    NO MINVALUE  
    NO MAXVALUE  
    CACHE 1;  
  
ALTER SEQUENCE accounts_id_seq OWNED BY accounts.id;  
  
CREATE TABLE alembic_version (  
    version_num character varying(32) NOT NULL  
);  
  
CREATE TABLE budgets (  
    id integer NOT NULL,  
    owner_id integer NOT NULL,  
    category_id integer NOT NULL,  
    amount numeric(15,2) NOT NULL,  
    created_at timestamp with time zone DEFAULT now() NOT NULL,  
    updated_at timestamp with time zone DEFAULT now() NOT NULL,  
    period_year integer NOT NULL,  
    period_month integer NOT NULL,  
    CONSTRAINT ck_budgets_amount_positive CHECK ((amount >  
(0)::numeric)),  
    CONSTRAINT ck_budgets_period_month_range CHECK (((period_month >= 1)  
AND (period_month <= 12)))  
);
```

```

CREATE SEQUENCE budgets_id_seq
  AS integer
  START WITH 1
  INCREMENT BY 1
  NO MINVALUE
  NO MAXVALUE
  CACHE 1;

ALTER SEQUENCE budgets_id_seq OWNED BY budgets.id;

CREATE TABLE categories (
  id integer NOT NULL,
  owner_id integer NOT NULL,
  name character varying(100) NOT NULL,
  parent_id integer,
  created_at timestamp with time zone DEFAULT now() NOT NULL,
  updated_at timestamp with time zone DEFAULT now() NOT NULL,
  kind character varying(7) DEFAULT 'expense'::character varying NOT
NULL
);

CREATE SEQUENCE categories_id_seq
  AS integer
  START WITH 1
  INCREMENT BY 1
  NO MINVALUE
  NO MAXVALUE
  CACHE 1;

ALTER SEQUENCE categories_id_seq OWNED BY categories.id;

CREATE TABLE email_tokens (
  id integer NOT NULL,
  owner_id integer NOT NULL,
  token_hash character varying(64) NOT NULL,
  purpose character varying(14) NOT NULL,
  expires_at timestamp with time zone NOT NULL,
  used_at timestamp with time zone,
  created_at timestamp with time zone DEFAULT now() NOT NULL
);

CREATE SEQUENCE email_tokens_id_seq
  AS integer
  START WITH 1
  INCREMENT BY 1
  NO MINVALUE
  NO MAXVALUE
  CACHE 1;

ALTER SEQUENCE email_tokens_id_seq OWNED BY email_tokens.id;

CREATE TABLE exchange_rates (
  id integer NOT NULL,
  char_code character varying(3) NOT NULL,

```



```

    num_code character varying(3) NOT NULL,
    name character varying(100) NOT NULL,
    nominal integer NOT NULL,
    value numeric(20,4) NOT NULL,
    vunit_rate numeric(20,10) NOT NULL,
    rate_date date NOT NULL,
    fetched_at timestamp with time zone DEFAULT now() NOT NULL
);

CREATE SEQUENCE exchange_rates_id_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;

ALTER SEQUENCE exchange_rates_id_seq OWNED BY exchange_rates.id;

CREATE TABLE recurring_transactions (
    id integer NOT NULL,
    owner_id integer NOT NULL,
    name character varying(100) NOT NULL,
    kind character varying(8) NOT NULL,
    account_id integer NOT NULL,
    amount numeric(15,2) NOT NULL,
    currency_code character varying(3) NOT NULL,
    category_id integer,
    transfer_account_id integer,
    note character varying(500),
    frequency character varying(7) NOT NULL,
    "interval" integer NOT NULL,
    start_at timestamp with time zone NOT NULL,
    end_at timestamp with time zone,
    next_run_at timestamp with time zone NOT NULL,
    last_run_at timestamp with time zone,
    is_active boolean DEFAULT true NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT ck_recurring_amount_positive CHECK ((amount >
(0)::numeric)),
    CONSTRAINT ck_recurring_interval_positive CHECK (("interval" >= 1)),
    CONSTRAINT ck_recurring_non_transfer_shape CHECK (((kind)::text =
'transfer'::text) OR (transfer_account_id IS NULL)),
    CONSTRAINT ck_recurring_transfer_shape CHECK (((kind)::text <>
'transfer'::text) OR ((transfer_account_id IS NOT NULL) AND (category_id
IS NULL) AND (account_id <> transfer_account_id)))
);

CREATE SEQUENCE recurring_transactions_id_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;

```

```

ALTER          SEQUENCE      recurring_transactions_id_seq      OWNED      BY
recurring_transactions.id;

CREATE TABLE transactions (
    id integer NOT NULL,
    owner_id integer NOT NULL,
    account_id integer NOT NULL,
    kind character varying(8) NOT NULL,
    amount numeric(15,2) NOT NULL,
    currency_code character varying(3) NOT NULL,
    category_id integer,
    transfer_account_id integer,
    occurred_at timestamp with time zone NOT NULL,
    note character varying(500),
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    target_amount numeric(15,2),
    CONSTRAINT ck_transactions_amount_positive CHECK ((amount >
(0)::numeric)),
    CONSTRAINT ck_transactions_non_transfer_shape CHECK (((kind)::text =
'transfer'::text) OR (transfer_account_id IS NULL)),
    CONSTRAINT ck_transactions_target_amount_positive CHECK
(((target_amount IS NULL) OR (target_amount > (0)::numeric))),
    CONSTRAINT ck_transactions_target_amount_transfer_only CHECK
(((target_amount IS NULL) OR ((kind)::text = 'transfer'::text))),
    CONSTRAINT ck_transactions_transfer_shape CHECK (((kind)::text <>
'transfer'::text) OR ((transfer_account_id IS NOT NULL) AND (category_id
IS NULL) AND (account_id <> transfer_account_id)))
);

CREATE SEQUENCE transactions_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;

ALTER SEQUENCE transactions_id_seq OWNED BY transactions.id;

CREATE TABLE users (
    id integer NOT NULL,
    email character varying(320) NOT NULL,
    password_hash character varying(200) NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    email_verified boolean DEFAULT false NOT NULL
);

CREATE SEQUENCE users_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;

```

```

ALTER SEQUENCE users_id_seq OWNED BY users.id;

ALTER TABLE ONLY accounts ALTER COLUMN id SET DEFAULT
nextval('accounts_id_seq'::regclass);

ALTER TABLE ONLY budgets ALTER COLUMN id SET DEFAULT
nextval('budgets_id_seq'::regclass);

ALTER TABLE ONLY categories ALTER COLUMN id SET DEFAULT
nextval('categories_id_seq'::regclass);

ALTER TABLE ONLY email_tokens ALTER COLUMN id SET DEFAULT
nextval('email_tokens_id_seq'::regclass);

ALTER TABLE ONLY exchange_rates ALTER COLUMN id SET DEFAULT
nextval('exchange_rates_id_seq'::regclass);

ALTER TABLE ONLY recurring_transactions ALTER COLUMN id SET DEFAULT
nextval('recurring_transactions_id_seq'::regclass);

ALTER TABLE ONLY transactions ALTER COLUMN id SET DEFAULT
nextval('transactions_id_seq'::regclass);

ALTER TABLE ONLY users ALTER COLUMN id SET DEFAULT
nextval('users_id_seq'::regclass);

ALTER TABLE ONLY accounts
    ADD CONSTRAINT accounts_pkey PRIMARY KEY (id);

ALTER TABLE ONLY alembic_version
    ADD CONSTRAINT alembic_version_pkc PRIMARY KEY (version_num);

ALTER TABLE ONLY budgets
    ADD CONSTRAINT budgets_pkey PRIMARY KEY (id);

ALTER TABLE ONLY categories
    ADD CONSTRAINT categories_pkey PRIMARY KEY (id);

ALTER TABLE ONLY email_tokens
    ADD CONSTRAINT email_tokens_pkey PRIMARY KEY (id);

ALTER TABLE ONLY exchange_rates
    ADD CONSTRAINT exchange_rates_pkey PRIMARY KEY (id);

ALTER TABLE ONLY recurring_transactions
    ADD CONSTRAINT recurring_transactions_pkey PRIMARY KEY (id);

ALTER TABLE ONLY transactions
    ADD CONSTRAINT transactions_pkey PRIMARY KEY (id);

ALTER TABLE ONLY accounts
    ADD CONSTRAINT uq_accounts_owner_name UNIQUE (owner_id, name);

```

```

ALTER TABLE ONLY budgets
    ADD CONSTRAINT uq_budgets_owner_category_period UNIQUE (owner_id,
category_id, period_year, period_month);

ALTER TABLE ONLY email_tokens
    ADD CONSTRAINT uq_email_tokens_token_hash UNIQUE (token_hash);

ALTER TABLE ONLY exchange_rates
    ADD CONSTRAINT uq_exchange_rates_code_date UNIQUE (char_code,
rate_date);

ALTER TABLE ONLY users
    ADD CONSTRAINT users_pkey PRIMARY KEY (id);

CREATE INDEX ix_accounts_owner_id ON accounts USING btree (owner_id);

CREATE INDEX ix_budgets_category_id ON budgets USING btree (category_id);

CREATE INDEX ix_budgets_owner_id ON budgets USING btree (owner_id);

CREATE INDEX ix_categories_kind ON categories USING btree (kind);

CREATE INDEX ix_categories_owner_id ON categories USING btree (owner_id);

CREATE INDEX ix_categories_parent_id ON categories USING btree
(parent_id);

CREATE INDEX ix_email_tokens_owner_id ON email_tokens USING btree
(owner_id);

CREATE INDEX ix_email_tokens_token_hash ON email_tokens USING btree
(token_hash);

CREATE INDEX ix_exchange_rates_char_code ON exchange_rates USING btree
(char_code);

CREATE INDEX ix_exchange_rates_rate_date ON exchange_rates USING btree
(rate_date);

CREATE INDEX ix_recurring_transactions_account_id ON
recurring_transactions USING btree (account_id);

CREATE INDEX ix_recurring_transactions_category_id ON
recurring_transactions USING btree (category_id);

CREATE INDEX ix_recurring_transactions_next_run_at ON
recurring_transactions USING btree (next_run_at);

CREATE INDEX ix_recurring_transactions_owner_id ON recurring_transactions
USING btree (owner_id);

CREATE INDEX ix_transactions_account_id ON transactions USING btree
(account_id);

```

```

CREATE INDEX ix_transactions_category_id ON transactions USING btree
(category_id);

CREATE INDEX ix_transactions_occurred_at ON transactions USING btree
(occurred_at);

CREATE INDEX ix_transactions_owner_id ON transactions USING btree
(owner_id);

CREATE UNIQUE INDEX ix_users_email ON users USING btree (email);

CREATE UNIQUE INDEX uq_categories_owner_parent_name ON categories USING
btree (owner_id, parent_id, name) NULLS NOT DISTINCT;

ALTER TABLE ONLY accounts
    ADD CONSTRAINT accounts_owner_id_fkey FOREIGN KEY (owner_id)
REFERENCES users(id) ON DELETE CASCADE;

ALTER TABLE ONLY budgets
    ADD CONSTRAINT budgets_category_id_fkey FOREIGN KEY (category_id)
REFERENCES categories(id) ON DELETE CASCADE;

ALTER TABLE ONLY budgets
    ADD CONSTRAINT budgets_owner_id_fkey FOREIGN KEY (owner_id)
REFERENCES users(id) ON DELETE CASCADE;

ALTER TABLE ONLY categories
    ADD CONSTRAINT categories_owner_id_fkey FOREIGN KEY (owner_id)
REFERENCES users(id) ON DELETE CASCADE;

ALTER TABLE ONLY categories
    ADD CONSTRAINT categories_parent_id_fkey FOREIGN KEY (parent_id)
REFERENCES categories(id) ON DELETE CASCADE;

ALTER TABLE ONLY email_tokens
    ADD CONSTRAINT email_tokens_owner_id_fkey FOREIGN KEY (owner_id)
REFERENCES users(id) ON DELETE CASCADE;

ALTER TABLE ONLY recurring_transactions
    ADD CONSTRAINT recurring_transactions_account_id_fkey FOREIGN KEY
(account_id) REFERENCES accounts(id) ON DELETE CASCADE;

ALTER TABLE ONLY recurring_transactions
    ADD CONSTRAINT recurring_transactions_category_id_fkey FOREIGN KEY
(category_id) REFERENCES categories(id) ON DELETE SET NULL;

ALTER TABLE ONLY recurring_transactions
    ADD CONSTRAINT recurring_transactions_owner_id_fkey FOREIGN KEY
(owner_id) REFERENCES users(id) ON DELETE CASCADE;

ALTER TABLE ONLY recurring_transactions

```

```

        ADD CONSTRAINT recurring_transactions_transfer_account_id_fkey
FOREIGN KEY (transfer_account_id) REFERENCES accounts(id) ON DELETE
CASCADE;

```

```

ALTER TABLE ONLY transactions

```

```

        ADD CONSTRAINT transactions_account_id_fkey FOREIGN KEY (account_id)
REFERENCES accounts(id) ON DELETE CASCADE;

```

```

ALTER TABLE ONLY transactions

```

```

        ADD CONSTRAINT transactions_category_id_fkey FOREIGN KEY
(category_id) REFERENCES categories(id) ON DELETE SET NULL;

```

```

ALTER TABLE ONLY transactions

```

```

        ADD CONSTRAINT transactions_owner_id_fkey FOREIGN KEY (owner_id)
REFERENCES users(id) ON DELETE CASCADE;

```

```

ALTER TABLE ONLY transactions

```

```

        ADD CONSTRAINT transactions_transfer_account_id_fkey FOREIGN KEY
(transfer_account_id) REFERENCES accounts(id) ON DELETE CASCADE;

```

Листинг А.2 – Сервис проводки операций (app/services/ledger.py)

```

"""Единая логика проводки сумм по балансам счетов.

```

Извлечено из app.api.v1.transactions, чтобы и обычные операции (роутер /transactions), и движок повторяющихся операций (роутер /recurring-transactions) применяли изменения баланса ОДИНАКОВО – через один источник истины, без копиасты (принципы DRY и Single Responsibility).

Модель «opening_balance + движения» (см. vkr/02_design.md): транзакция влияет на balance счёта ТОЛЬКО если её occurred_at >= account.opening_date.

Транзакции «до opening_date» сохраняются в истории, но баланс не двигают –

их эффект уже зашит в opening_balance.

```

"""

```

```

from datetime import datetime, timezone
from decimal import Decimal

```

```

from sqlalchemy import update
from sqlalchemy.ext.asyncio import AsyncSession

```

```

from app.db.models.account import Account

```

```

def ensure_aware_utc(dt: datetime) -> datetime:

```

```

    """Если datetime naive – считаем его UTC (так же интерпретирует
Postgres
при INSERT в TIMESTAMPTZ-колонку).

```

Mantine 9 присылает naive ISO-строки ("2026-04-16T17:08:00", без Z и без

offset). Pydantic парсит их в naive datetime, а колонки в БД – TIMESTAMPTZ

```

        (SQLAlchemy отдаёт их aware). Прямое сравнение naive < aware в Python
даёт
        TypeError. Приведение naive к UTC-aware совпадает с реальной
семантикой
        хранения в Postgres.
        """
        if dt.tzinfo is None:
            return dt.replace(tzinfo=timezone.utc)
        return dt

def signed_delta_for_source(kind: str, amount: Decimal) -> Decimal:
    """Знаковое изменение баланса счёта-источника при СОЗДАНИИ
транзакции.

    income -> +amount (деньги пришли).
    expense -> -amount (деньги ушли).
    transfer -> -amount (на источнике уменьшается; счёт-получатель
обновляется отдельным вызовом в роутере).

    При DELETE применяется противоположный знак (см. delete_transaction).
    """
    if kind == "income":
        return amount
    return -amount

async def apply_signed_delta_to_account(
    account: Account,
    delta: Decimal,
    occurred_at: datetime,
    session: AsyncSession,
) -> None:
    """Применить delta к account.balance, только если occurred_at >=
opening_date.

    UPDATE через session.execute (а не присваивание в Python) атомарен на
уровне строки в Postgres – это защищает от lost update при двух
параллельных запросах к одному счёту.
    """
    if ensure_aware_utc(occurred_at) < account.opening_date:
        return
    await session.execute(
        update(Account)
        .where(Account.id == account.id)
        .values(balance=Account.balance + delta)
    )

```

Листинг А.3 – Модуль безопасности: хеширование паролей (Argon2id) и JWT (app/security.py)

```

"""Криптографические утилиты: хеширование паролей и JWT-токены.

```

Этот модуль изолирует все операции с криптографией в одном месте:
 – хеширование/проверка паролей (Argon2id),

- выпуск/декодирование JWT access-токенов.

Все функции – чистые (без обращений к БД), что упрощает их тестирование unit-тестами (требование МУ ВКР, раздел 4 «Тестирование»).

```
"""
```

```
from datetime import datetime, timedelta, timezone
from typing import Any
```

```
import jwt
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError
```

```
from app.config import settings
```

```
# Один экземпляр PasswordHasher на процесс (паттерн Singleton).
# Параметры по умолчанию у argon2-cffi соответствуют рекомендациям OWASP
2023:
# time_cost=3, memory_cost=64 MiB, parallelism=4.
_password_hasher = PasswordHasher()
```

```
def hash_password(plain_password: str) -> str:
    """Захэшировать пароль алгоритмом Argon2id.
```

```
    Возвращает строку вида `argon2id$v=19$m=65536,t=3,p=4$...$...`,
    в которой уже зашита соль, параметры и сам хэш – отдельно соль
    хранить
```

```
    не нужно. Это и пишем в `users.password_hash`.
    """
```

```
    return _password_hasher.hash(plain_password)
```

```
def verify_password(plain_password: str, password_hash: str) -> bool:
    """Проверить, что пароль соответствует ранее сохранённому хэшу.
```

```
    Возвращает True, если совпадает, иначе False. Никогда не выбрасывает
    исключений наружу – это удобнее для вызывающего кода.
    """
```

```
    try:
```

```
        _password_hasher.verify(password_hash, plain_password)
```

```
    except VerifyMismatchError:
```

```
        return False
```

```
    except Exception:
```

```
        # Любая другая ошибка (повреждённый хэш и т.п.) – тоже неуспешная
        проверка.
```

```
        return False
```

```
    return True
```

```
def create_access_token(subject: str | int, expires_minutes: int | None =
None) -> str:
```

```
    """Выпустить JWT access-токен.
```


Параметр `subject` – то, что мы хотим зашифровать в токене как идентификатор

пользователя (обычно `user_id`). Кладём его в стандартное claim `sub`.

Также добавляем `exp` (срок истечения) и `iat` (момент выпуска) – это стандартные JWT-claims, которые библиотека PyJWT понимает автоматически

(например, при декодировании сама проверяет, не истёк ли токен).

```
"""
    expires_minutes = expires_minutes or
settings.jwt_access_token_expires_minutes
    now = datetime.now(timezone.utc)
    payload: dict[str, Any] = {
        "sub": str(subject),
        "iat": now,
        "exp": now + timedelta(minutes=expires_minutes),
    }
    return jwt.encode(payload, settings.jwt_secret_key,
algorithm=settings.jwt_algorithm)
```

```
def decode_access_token(token: str) -> dict[str, Any]:
```

```
    """Расшифровать и проверить JWT access-токен.
```

При успехе возвращает payload – словарь с claims (`sub`, `iat`, `exp`).

При проблемах (плохая подпись, истёк срок, мусор вместо токена) выбросит

`jwt.PyJWTError` или одного из его наследников. Вызывающий код (например, зависимость `get_current_user`) должен это перехватить и вернуть HTTP 401.

```
"""
    return jwt.decode(
        token,
        settings.jwt_secret_key,
        algorithms=[settings.jwt_algorithm],
    )
```

Листинг A.4 – Маршрутизатор операций REST API

(`app/api/v1/transactions.py`)

```
"""Роутер /transactions: CRUD финансовых операций.
```

Создание и удаление транзакции атомарно меняют balance связанного счёта (или двух счетов для перевода) в одной БД-транзакции. Если любая часть упадёт – БД остаётся консистентной (НФТ-07 раздела 1 ВКР).

Сознательные упрощения MVP:

- PATCH реализован ЧАСТИЧНО: правятся только «безопасные» поля, не влияющие на балансы – `category_id`, `occurred_at`, `note`. Изменение суммы / счёта / типа / счёта-получателя делается через DELETE старой операции + POST новой. Так мы по построению исключаем класс багов «рассинхрон баланса и истории», который возникал бы при полноценном UPDATE с пересчётом балансов «откатить старое →

применить новое» (особенно для переводов и при смене kind).

- Кросс-валютный перевод: если валюты счёта-источника и счёта-получателя различаются, списывается amount (в валюте источника), а зачисляется target_amount (в валюте получателя) — её вводит пользователь, т.к. банковский курс с комиссией отличается от курса ЦБ. Для одновалютного перевода target_amount не нужен (зачисляется тот же amount).

ВАЖНО (модель «opening_balance + движения», см. vkr/02_design.md): Транзакция влияет на balance счёта ТОЛЬКО если её occurred_at >= account.opening_date. Транзакции «до opening_date» сохраняются в истории, но не двигают balance — их эффект уже сидит в opening_balance.

Все изменения баланса в этом модуле обернуты в _apply_signed_delta_to_account, который инкапсулирует эту проверку.

"""

```
from datetime import datetime, timezone
from decimal import Decimal
from typing import Literal
```

```
from fastapi import APIRouter, Depends, HTTPException, Query, Response,
status
from sqlalchemy import select, update
from sqlalchemy.ext.asyncio import AsyncSession
```

```
from app.api.deps import get_current_user
from app.db.models.account import Account
from app.db.models.category import Category
from app.db.models.transaction import Transaction
from app.db.models.user import User
from app.db.session import get_session
from app.schemas.transaction import (
    TransactionCreate,
    TransactionRead,
    TransactionUpdate,
)
```

```
# Логика проводки баланса вынесена в общий сервис (app.services.ledger),
# чтобы её одинаково переиспользовали и обычные операции, и движок
# повторяющихся операций (DRY). Импортируем под прежними _именами,
# чтобы остальной код модуля остался без изменений.
```

```
from app.services.ledger import (
    apply_signed_delta_to_account as _apply_signed_delta_to_account,
    ensure_aware_utc as _ensure_aware_utc,
    signed_delta_for_source as _signed_delta_for_source,
)
```

```
router = APIRouter(prefix="/transactions", tags=["transactions"])
```

```
@router.post(
    "",
    response_model=TransactionRead,
    status_code=status.HTTP_201_CREATED,
    summary="Создать транзакцию (атомарно меняет балансы)",
)
```

```

    )
    async def create_transaction(
        payload: TransactionCreate,
        current_user: User = Depends(get_current_user),
        session: AsyncSession = Depends(get_session),
    ) -> Transaction:
        # 0. Дата операции не должна быть в будущем. Иначе balance
        #    показывал бы
        #    «будущее» состояние как текущее — это семантически
        #    некорректно.
        #    Планирование будущих платежей — отдельная фишка (бюджеты),
        #    а не наложение на текущий баланс.
        if
            _ensure_aware_utc(payload.occurred_at)
            >
            datetime.now(timezone.utc):
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Дата операции не может быть в будущем",
            )

        # 1. Источник — обязательно наш (защита от IDOR).
        source = await _get_owned_account_or_404(
            payload.account_id, current_user, session
        )

        # 1.5. Если клиент явно передал currency_code — он должен совпадать
        #    с валютой счёта. Иначе balance += amount даёт математическую
        #    бессмыслицу (рубли + доллары). Через UI этого не сделать,
        #    но API без проверки = silent corruption при
        #    импорте/интеграциях.
        if (
            payload.currency_code is not None
            and payload.currency_code.upper() != source.currency_code
        ):
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=(
                    f"Валюта операции ({payload.currency_code.upper()}) не
                    совпадает "
                    f"с валютой счёта ({source.currency_code}). Для разных
                    валют "
                    f"используйте отдельные счета."
                ),
            )

        # 2. Категория, если задана — обязательно наша, и её kind должен
        #    совпадать с kind транзакции (расходную транзакцию нельзя
        #    отнести
        #    к доходной категории). 400 (а не 404) — это валидация входа.
        if payload.category_id is not None:
            category = await session.get(Category, payload.category_id)
            if category is None or category.owner_id != current_user.id:
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail="Категория не найдена",
                )
            # Для transfer категория уже отвергается схемой и БД-
            # CHECK'ом.

```

```

        # Здесь явная проверка соответствия kind для income/expense.
        if payload.kind in ("income", "expense") and category.kind !=
payload.kind:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=(
                    f"Категория «{category.name}» предназначена для "
                    f"{category.kind}, а операция — {payload.kind}."
                ),
            )

        # 3. Для перевода — получатель обязательно наш. Если валюты
        совпадают —
        # зачисляется тот же amount (target_amount хранить не нужно).
        Если
        # различаются — это кросс-валютный перевод: зачисляется
        введённый
        # пользователем target_amount (в валюте получателя).
        target: Account | None = None
        # credited_amount — сколько зачислится на получателя;
        stored_target_amount
        # — что положим в колонку target_amount (NULL для одновалютных
        переводов).
        credited_amount: Decimal = payload.amount
        stored_target_amount: Decimal | None = None
        if payload.kind == "transfer":
            # transfer_account_id точно не None: проверено
            model_validator'ом схемы.
            assert payload.transfer_account_id is not None
            target = await _get_owned_account_or_404(
                payload.transfer_account_id, current_user, session
            )
            if target.currency_code == source.currency_code:
                # Одновалютный перевод. target_amount тут — лишнее поле;
                принимаем
                # только если он не противоречит amount, иначе — понятная
                400
                # (а не тихое игнорирование переданного значения).
                if (
                    payload.target_amount is not None
                    and payload.target_amount != payload.amount
                ):
                    raise HTTPException(
                        status_code=status.HTTP_400_BAD_REQUEST,
                        detail=(
                            "Для перевода в одной валюте сумма зачисления
                            равна "
                            "сумме списания — не указывайте её отдельно."
                        ),
                    )
                credited_amount = payload.amount
                stored_target_amount = None
            else:
                # Кросс-валютный перевод: сумма зачисления обязательна.
                if payload.target_amount is None:
                    raise HTTPException(
                        status_code=status.HTTP_400_BAD_REQUEST,
                        detail=(

```

```

        f"Перевод между счетами разных валют
({source.currency_code} "
        f"→ {target.currency_code}): укажите сумму
зачисления "
        f"в {target.currency_code} (поле
target_amount)."
    ),
    ),
    credited_amount = payload.target_amount
    stored_target_amount = payload.target_amount

# 4. Валюта операции – snapshot валюты счёта-источника, если клиент
# не передал явно. .upper() для нормализации (RUB / Rub / rub).
currency = (payload.currency_code or source.currency_code).upper()

# 5. Создание + пересчёт балансов в одной БД-транзакции. SQLAlchemy
# сессия уже в неявной транзакции с момента первого запроса; нам
# нужно лишь не вызывать commit между шагами и вызвать его в
конце.
transaction = Transaction(
    owner_id=current_user.id,
    account_id=payload.account_id,
    kind=payload.kind,
    amount=payload.amount,
    target_amount=stored_target_amount,
    currency_code=currency,
    category_id=payload.category_id,
    transfer_account_id=payload.transfer_account_id,
    occurred_at=payload.occurred_at,
    note=payload.note,
)
session.add(transaction)

# delta_source: знаковое изменение баланса счёта-источника при
создании.
delta_source = _signed_delta_for_source(payload.kind,
payload.amount)

# Применяем delta к balance, но только если occurred_at >=
opening_date
# источника. Для ретро-операций «до opening_date» balance не
меняется –
# их эффект уже учтён в opening_balance (см. vkr/02_design.md).
# UPDATE атомарен на уровне строки в Postgres – защита от lost
update.
await _apply_signed_delta_to_account(
    source, delta_source, payload.occurred_at, session
)

if payload.kind == "transfer":
    assert target is not None # установлен выше
    # Для перевода – независимая проверка для счёта-получателя:
    # его opening_date может отличаться от opening_date источника.
    # Зачисляем credited_amount: для одновалютного – это amount,
для
    # кросс-валютного – target_amount в валюте получателя.
    await _apply_signed_delta_to_account(

```

```

        target, credited_amount, payload.occurred_at, session
    )

    await session.commit()
    await session.refresh(transaction)
    return transaction

@router.get(
    "",
    response_model=list[TransactionRead],
    summary="Список транзакций с фильтрами",
)
async def list_transactions(
    account_id: int | None = Query(default=None, description="Фильтр по счёту"),
    category_id: int | None = Query(default=None, description="Фильтр по категории"),
    kind: Literal["income", "expense", "transfer"] | None = Query(default=None),
    from_date: datetime | None = Query(default=None, description="С (включительно)"),
    to_date: datetime | None = Query(default=None, description="По (включительно)"),
    limit: int = Query(default=50, ge=1, le=500),
    offset: int = Query(default=0, ge=0),
    current_user: User = Depends(get_current_user),
    session: AsyncSession = Depends(get_session),
) -> list[Transaction]:
    """Транзакции юзера с фильтрами и пагинацией. Свежие — первыми."""
    stmt = (
        select(Transaction)
        .where(Transaction.owner_id == current_user.id)
        .order_by(Transaction.occurred_at.desc(),
Transaction.id.desc())
        .limit(limit)
        .offset(offset)
    )
    # Note: фильтр account_id ловит только те операции, где счёт является
    # источником (account_id). Переводы-получатели сюда не попадут.
    # «Все операции, затронувшие счёт» — отдельный отчёт, добавим при
    # необходимости через OR transfer_account_id.
    if account_id is not None:
        stmt = stmt.where(Transaction.account_id == account_id)
    if category_id is not None:
        stmt = stmt.where(Transaction.category_id == category_id)
    if kind is not None:
        stmt = stmt.where(Transaction.kind == kind)
    if from_date is not None:
        stmt = stmt.where(Transaction.occurred_at >= from_date)
    if to_date is not None:
        stmt = stmt.where(Transaction.occurred_at <= to_date)

    result = await session.scalars(stmt)
    return list(result)

```

```

@router.get(
   ("/{transaction_id}",
    response_model=TransactionRead,
    summary="Получить транзакцию по id",
)
)
async def get_transaction(
    transaction_id: int,
    current_user: User = Depends(get_current_user),
    session: AsyncSession = Depends(get_session),
) -> Transaction:
    return await _get_owned_transaction_or_404(
        transaction_id, current_user, session
    )

@router.patch(
   ("/{transaction_id}",
    response_model=TransactionRead,
    summary="Частично обновить транзакцию (только безопасные поля)",
)
)
async def update_transaction(
    transaction_id: int,
    payload: TransactionUpdate,
    current_user: User = Depends(get_current_user),
    session: AsyncSession = Depends(get_session),
) -> Transaction:
    """Обновить категорию / дату / заметку транзакции.

```

Балансы счетов НЕ трогаются — это инвариант, защищающий от рассинхрона истории и сумм на счетах. Чтобы поправить amount, account, kind или transfer_account_id — клиент удаляет старую операцию (DELETE откатит балансы) и создаёт новую (POST применит новые балансы). Двухшаговый сценарий по построению атомарен на уровне каждого шага.

Используем model_fields_set, а не значения атрибутов, чтобы различать «поле не передано» (не трогаем) и «поле передано null» (например, снять категорию).

```

"""
transaction = await _get_owned_transaction_or_404(
    transaction_id, current_user, session
)

fields = payload.model_fields_set

if "category_id" in fields:
    if payload.category_id is not None:
        # Перевод не может иметь категорию (ЧЕКС на БД-уровне).
        # Здесь — понятный 400 вместо 500 от БД.
        if transaction.kind == "transfer":
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Перевод не может иметь категорию",
            )
        category = await session.get(Category, payload.category_id)

```

```

        if category is None or category.owner_id !=
current_user.id:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Категория не найдена",
            )
        # kind транзакции в PATCH не меняется, поэтому достаточно
        # сверить с текущим kind операции.
        if category.kind != transaction.kind:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=(
                    f"Категория «{category.name}» предназначена для
"
                    f"{category.kind}, а операция —
{transaction.kind}."
                ),
            )
        transaction.category_id = payload.category_id

    if "occurred_at" in fields:
        # occurred_at обнулять нельзя — поле NOT NULL.
        if payload.occurred_at is None:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="occurred_at не может быть null",
            )
        # И не может быть в будущем — то же правило, что в POST.
        if _ensure_aware_utc(payload.occurred_at) >
datetime.now(timezone.utc):
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Дата операции не может быть в будущем",
            )
        # Если новая дата пересекает границу opening_date счёта —
        # нужно скорректировать balance: убрать delta (если ушла «до»)
        # или добавить (если пришла «после»). Делаем ДО изменения
        # самого поля, чтобы передать в helper и старое, и новое
значения.
        old_occurred_at = transaction.occurred_at
        new_occurred_at = payload.occurred_at
        # Сравниваем по UTC-моменту, а не по «голому» значению с
tzinfo.
        # Иначе naive 17:08 и aware 17:08+00:00 считались бы разными
        # и мы зря триггерили бы _shift_balance.
        if _ensure_aware_utc(old_occurred_at) != _ensure_aware_utc(
            new_occurred_at
        ):
            source_account = await session.get(Account,
transaction.account_id)
            assert source_account is not None
            delta_source = _signed_delta_for_source(
                transaction.kind, transaction.amount
            )
            await _shift_balance_for_date_change(
                source_account,
                delta_source,
                old_occurred_at,

```



```

        new_occurred_at,
        session,
    )
    if transaction.kind == "transfer":
        target_account = await session.get(
            Account, transaction.transfer_account_id
        )
        assert target_account is not None
        # Получателю при переводе зачислялся credited amount:
        # target_amount для кросс-валютного, иначе amount.
        credited = (
            transaction.target_amount
            if transaction.target_amount is not None
            else transaction.amount
        )
        await _shift_balance_for_date_change(
            target_account,
            credited,
            old_occurred_at,
            new_occurred_at,
            session,
        )
        transaction.occurred_at = new_occurred_at

    if "note" in fields:
        # note nullable — null = снять заметку. Пустую строку
        # тоже трактуем как «снять», чтобы не хранить '' в БД.
        transaction.note = payload.note if payload.note else None

    await session.commit()
    await session.refresh(transaction)
    return transaction

@router.delete(
   ("/{transaction_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    summary="Удалить транзакцию (с откатом балансов)",
)
async def delete_transaction(
    transaction_id: int,
    current_user: User = Depends(get_current_user),
    session: AsyncSession = Depends(get_session),
) -> Response:
    """Удалить транзакцию и зеркально откатить эффект на балансах
    счетов.

    DELETE применяет противоположный delta к источнику; для перевода —
    дополнительно вычитает зачисленную сумму из получателя
    (target_amount
    для кросс-валютного, иначе amount). Всё в одной БД-транзакции.
    """
    transaction = await _get_owned_transaction_or_404(
        transaction_id, current_user, session
    )

```

```

# Загружаем счёт-источник как объект (а не апдейтим по id),
# чтобы проверить opening_date в _apply_signed_delta_to_account.
source_account = await session.get(Account, transaction.account_id)
assert source_account is not None # FK + IDOR-проверка выше
гарантируют

```

```

# Зеркальный эффект: применяем противоположный delta. Условие про
# opening_date то же: если транзакция была «до opening_date»,
# она не вкладывалась в balance — и убирать тоже нечего.
delta_source = -_signed_delta_for_source(transaction.kind,
transaction.amount)
await _apply_signed_delta_to_account(
    source_account, delta_source, transaction.occurred_at, session
)

```

```

if transaction.kind == "transfer":
    target_account = await session.get(Account,
transaction.transfer_account_id)
    assert target_account is not None
    # Откатываем именно ту сумму, что зачисляли получателю.
    credited = (
        transaction.target_amount
        if transaction.target_amount is not None
        else transaction.amount
    )
    await _apply_signed_delta_to_account(
        target_account,
        -credited,
        transaction.occurred_at,
        session,
    )

```

```

await session.delete(transaction)
await session.commit()
return Response(status_code=status.HTTP_204_NO_CONTENT)

```

#	—	Внутренние	хелперы
#			
#	_ensure_aware_utc,	_apply_signed_delta_to_account	и
#	_signed_delta_for_source		
#	переехали в app.services.ledger (импортированы выше под теми же именами) —		
#	чтобы движок повторяющихся операций использовал ту же логику без копипасты.		

```

async def _shift_balance_for_date_change(
    account: Account,
    delta: Decimal,
    old_occurred_at: datetime,
    new_occurred_at: datetime,
    session: AsyncSession,
) -> None:

```

```
"""Скорректировать account.balance при смене даты транзакции через
PATCH.
```

```
delta — знаковое изменение balance, которое транзакция применяет,
когда активна (т.е. occurred_at >= opening_date).
```

```
4 случая:
```

```
- был активен, остался активен → delta уже применён, ничего не
делаем.
- был неактивен, остался неактивен → delta не применялся, ничего не
делаем.
- был активен, стал неактивен → откатываем (применяем -delta).
- был неактивен, стал активен → применяем (применяем +delta).
```

```
Это инкрементальная альтернатива recompute_account_balance —
быстрее на больших объёмах транзакций (без полного scan'a).
```

```
"""
old_active      =      _ensure_aware_utc(old_occurred_at)      >=
account.opening_date
new_active      =      _ensure_aware_utc(new_occurred_at)      >=
account.opening_date
if old_active == new_active:
    return
sign = 1 if new_active else -1
await session.execute(
    update(Account)
    .where(Account.id == account.id)
    .values(balance=Account.balance + sign * delta)
)
```

```
async def _get_owned_account_or_404(
    account_id: int,
    current_user: User,
    session: AsyncSession,
) -> Account:
    """То же, что в accounts.py — продублировано здесь, чтобы не
создавать
cross-роутерные импорты ради одной хелпер-функции.
"""
    account = await session.get(Account, account_id)
    if account is None or account.owner_id != current_user.id:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Счёт не найден",
        )
    return account
```

```
async def _get_owned_transaction_or_404(
    transaction_id: int,
    current_user: User,
    session: AsyncSession,
) -> Transaction:
    transaction = await session.get(Transaction, transaction_id)
    if transaction is None or transaction.owner_id != current_user.id:
```

```

        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Транзакция не найдена",
        )
    return transaction

```

Листинг А.5 – Сценарий нагрузочного тестирования (loadtest/locustfile.py)

"""Нагрузочный тест FinTrack на Locust.

Сценарий моделирует типичную read-heavy нагрузку: каждый виртуальный пользователь регистрируется, входит, создаёт счёт и несколько операций (чтобы списки и отчёты были непустыми), после чего циклически читает основные эндпоинты — список операций, дашборд, отчёты, счета, курсы. Веса задач отражают частоту обращений в реальном UI.

Запуск и снятие графиков — см. [changes/loadtest-locust/INSTRUCTIONS.md](https://github.com/locustio/locust/blob/master/CONTRIBUTING.md).

```

"""
import uuid
from datetime import datetime, timezone

from locust import HttpUser, between, task

class FinTrackUser(HttpUser):
    # Пауза между запросами одного пользователя — имитация «думающего»
    # человека.
    wait_time = between(1, 3)

    def on_start(self) -> None:
        """Подготовка пользователя: регистрация, вход, посев данных."""
        email = f"load_{uuid.uuid4().hex[:12]}@example.com"
        password = "LoadTest123" # удовлетворяет политике: буква +
        # цифра, >= 8

        self.client.post(
            "/api/v1/auth/register",
            json={"email": email, "password": password},
            name="/auth/register",
        )
        resp = self.client.post(
            "/api/v1/auth/login",
            json={"email": email, "password": password},
            name="/auth/login",
        )
        token = resp.json().get("access_token") if resp.status_code ==
        200 else None
        if token:
            self.client.headers.update({"Authorization": f"Bearer
            {token}"})

        # Посев: один счёт и несколько расходов, чтобы read-эндпоинты
        # возвращали данные.

```

```

        acc = self.client.post(
            "/api/v1/accounts",
            json={"name": "Kapra", "opening_balance": "10000",
"currency_code": "RUB"},
            name="/accounts [POST]",
        )
        self.account_id = acc.json().get("id") if acc.status_code == 201
    else None
    if self.account_id is not None:
        now = datetime.now(timezone.utc).isoformat()
        for _ in range(5):
            self.client.post(
                "/api/v1/transactions",
                json={
                    "kind": "expense",
                    "account_id": self.account_id,
                    "amount": "100",
                    "occurred_at": now,
                },
                name="/transactions [POST]",
            )

# --- Основная read-heavy нагрузка (веса = относительная частота) ---

@task(5)
def list_transactions(self) -> None:
    self.client.get("/api/v1/transactions", name="/transactions")

@task(3)
def dashboard(self) -> None:
    self.client.get("/api/v1/dashboard/summary",
name="/dashboard/summary")

@task(2)
def reports(self) -> None:
    self.client.get("/api/v1/reports/overview",
name="/reports/overview")

@task(2)
def accounts(self) -> None:
    self.client.get("/api/v1/accounts", name="/accounts")

@task(1)
def rates(self) -> None:
    self.client.get("/api/v1/rates", name="/rates")

```

ПРИЛОЖЕНИЕ Б. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Б.1. Назначение и область применения

Веб-приложение FinTrack предназначено для учёта и анализа личных финансов: ведения счетов в разных валютах, регистрации доходов, расходов и переводов, категоризации операций, планирования бюджетов, автоматизации повторяющихся операций, а также просмотра сводных отчётов и дашборда. Приложение работает в браузере и разворачивается в режиме self-hosted (на сервере или компьютере пользователя), что обеспечивает хранение данных под его контролем.

Б.2. Системные требования и установка

Для развёртывания необходимы установленные Docker и Docker Compose. Порядок установки:

1. Получить исходный код проекта (клонировать репозиторий).
2. Создать файл `backend/.env` на основе `backend/.env.example` и задать обязательную переменную `JWT_SECRET_KEY` (длинная случайная строка). Пример генерации: `python -c "import secrets; print(secrets.token_urlsafe(48))"`.
3. Запустить инфраструктуру: `docker compose up -d`.
4. Применить миграции базы данных: `docker compose exec backend alembic upgrade head`.
5. Открыть приложение в браузере по адресу фронтенда (по умолчанию `http://127.0.0.1:60001`).

Б.3. Регистрация и вход

При первом обращении необходимо зарегистрироваться: указать адрес электронной почты и пароль (не менее 8 символов, должен содержать букву и цифру). После регистрации выполняется вход по тем же данным; при успешном входе открывается главная страница.

Внешний вид страницы входа приведён на рисунке В.1 (приложение В).

Подтверждение адреса электронной почты не обязательно для работы, но рекомендуется: его можно запросить на странице профиля (кнопка повторной отправки письма).

Б.4. Главная страница (дашборд)

На дашборде отображаются: общий капитал на всех счетах (в рублях по курсу ЦБ РФ), сумма расходов за текущий месяц, топ категорий расходов и прогресс бюджетов.

Внешний вид дашборда приведён на рисунке В.2 (приложение В).

Б.5. Счета

Раздел «Счета» позволяет создавать, редактировать и удалять счета. При создании задаются: название, тип (карта, наличные, накопительный, кредитная карта, электронный кошелёк, прочее), валюта, текущий остаток и его дата. Текущий баланс рассчитывается автоматически как остаток плюс все операции после «даты остатка».

Внешний вид страницы счетов приведён на рисунке В.3 (приложение В).

Б.6. Категории

Раздел «Категории» позволяет вести древовидную классификацию доходов и расходов: создавать категории и подкатегории, переименовывать их и переносить под другого родителя, удалять. Категории разделены на доходные и расходные. Внешний вид страницы категорий приведён на рисунке В.6 (приложение В).

Б.7. Операции

Раздел «История» предназначен для ведения операций трёх видов:

- доход — поступление средств на счёт;
- расход — списание со счёта с указанием категории;
- перевод — перемещение средств между своими счетами.

При создании операции выбираются тип, счёт, сумма, категория (для дохода/расхода), дата и заметка. Форма показывает предпросмотр изменения балансов.

Внешний вид формы создания операции приведён на рисунке В.5 (приложение В).

Кросс-валютный перевод. Если счёт-источник и счёт-получатель в разных валютах, дополнительно указывается сумма зачисления в валюте получателя; приложение предзаполняет её по курсу ЦБ РФ, а пользователь при необходимости корректирует (банковский курс отличается от курса ЦБ).

Фильтры и поиск. Историю можно фильтровать по счёту, категории, типу и периоду (есть быстрые периоды: сегодня, 7 дней, месяц и др.) и искать по тексту заметки. Текущая выборка может быть выгружена в файл CSV (кнопка экспорта). Внешний вид страницы истории операций приведён на рисунке В.4 (приложение В).

Б.8. Бюджеты

Раздел «Бюджеты» позволяет задать лимит расходов по категории на конкретный месяц. Приложение отслеживает сумму расходов (с пересчётом в рубль) и показывает прогресс и статус: «в норме», «предупреждение» ($\geq 70\%$) или «превышен» ($\geq 100\%$).

Внешний вид страницы бюджетов приведён на рисунке В.7 (приложение В).

Б.9. Повторяющиеся операции

Раздел «Регулярные» позволяет задать правила автоматического создания операций (например, зарплата, аренда, подписки): тип, счёт, сумма, частота (ежедневно/еженедельно/ежемесячно/ежегодно), интервал, дата начала и (необязательно) окончания. Назревшие операции создаются автоматически при входе в приложение, а также по кнопке «Выполнить сейчас». Правило можно поставить на паузу и возобновить. Внешний вид страницы регулярных операций приведён на рисунке В.8 (приложение В).

Б.10. Отчёты

Раздел «Отчёты» отображает за выбранный период: динамику доходов, расходов и капитала по времени, структуру расходов и доходов по категориям, распределение капитала по счетам. Возможна фильтрация по счёту; для одного счёта суммы показываются в его валюте, для всех счетов — в рублях.

Внешний вид страницы отчётов приведён на рисунке В.9 (приложение В).

Б.11. Курсы валют

Раздел «Курсы валют» показывает актуальные курсы ЦБ РФ, используемые для пересчёта мультивалютных сумм. Внешний вид страницы курсов валют приведён на рисунке В.10 (приложение В).

Б.12. Профиль и настройки

На странице профиля доступны: просмотр адреса электронной почты, смена пароля (требуется ввод текущего пароля), запрос письма для подтверждения почты, переключение тёмной/светлой темы, а также удаление учётной записи со всеми данными (реализация права на забвение; действие необратимо). Внешний вид страницы профиля приведён на рисунке В.11 (приложение В).

Б.13. Восстановление пароля

Если пароль забыт, на странице входа используется ссылка «Забыли пароль?»: на указанный адрес отправляется письмо со ссылкой для установки нового пароля (ссылка одноразовая и действует ограниченное время).

Б.14. Сообщения об ошибках и действия пользователя

Основные сообщения об ошибках и рекомендуемые действия пользователя приведены в таблице Б.1.

Таблица Б.1 – Сообщения об ошибках и действия пользователя

Ситуация	Реакция приложения	Действия пользователя
Некорректный ввод в форме (пустое поле, сумма ≤ 0 , слабый пароль)	Подсветка поля и пояснение	Исправить введённые данные
Неверный email или пароль при входе	Сообщение «Неверный email или пароль»	Проверить данные или восстановить пароль
Истёк срок сессии	Перенаправление на страницу входа	Войти повторно
Попытка создать дубликат (счёт/категория/бюджет с тем же именем/периодом)	Сообщение о конфликте	Изменить название или период
Перевод между счетами разных валют без суммы зачисления	Сообщение об ошибке	Указать сумму зачисления
Курсы ЦБ временно недоступны	Используется последний известный курс либо уведомление	Повторить позже

ПРИЛОЖЕНИЕ В. СНИМКИ ЭКРАНОВ ПРИЛОЖЕНИЯ

В приложении приведены снимки экранов работающего приложения FinTrack, иллюстрирующие реализованный пользовательский интерфейс и основные функциональные возможности системы. Снимки выполнены в десктопном браузере и отражают актуальное состояние интерфейса.

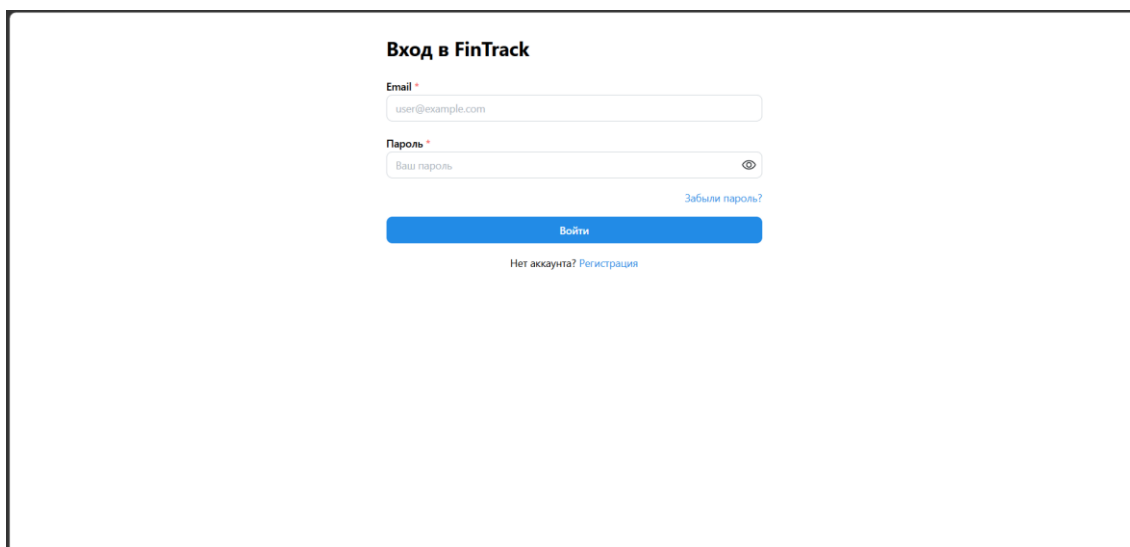


Рисунок В.1 – Страница входа в систему

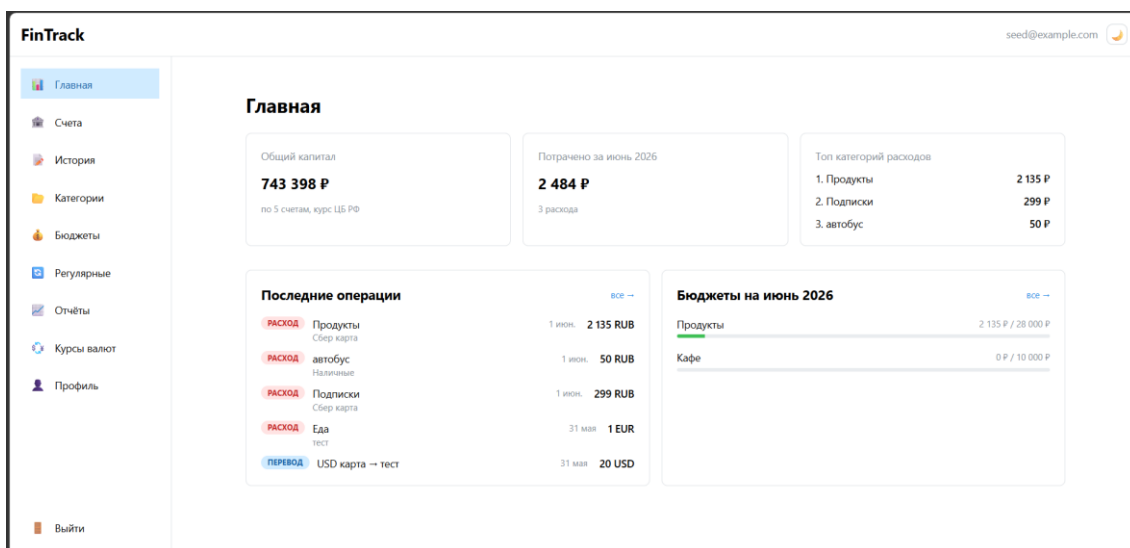


Рисунок В.2 – Главная страница (дашборд): общий капитал, расходы за месяц, топ категорий расходов, последние операции и бюджеты

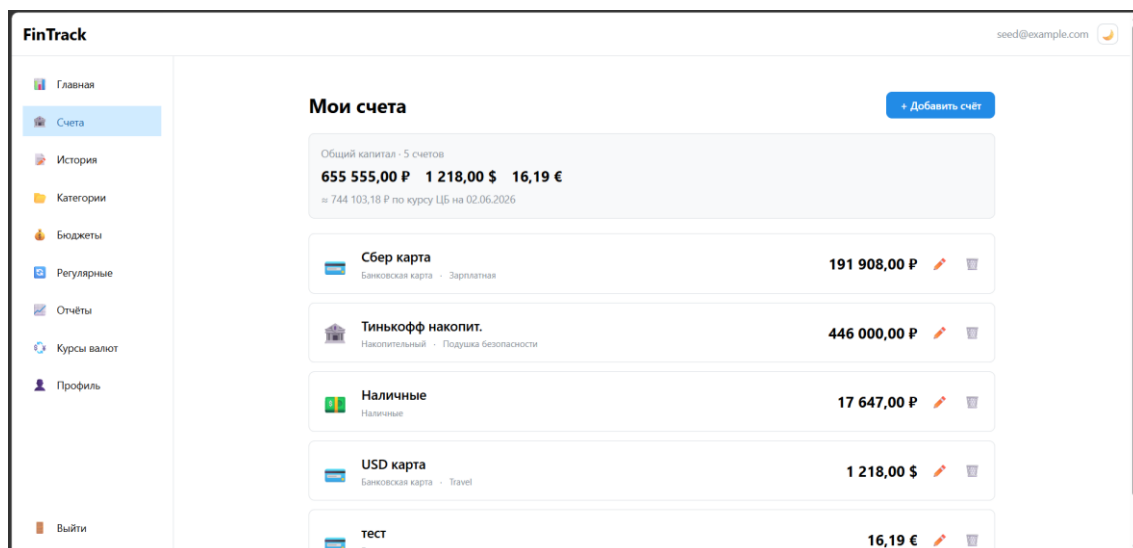


Рисунок В.3 – Страница «Счета»: список счетов в разных валютах и общий капитал в пересчёте по курсу ЦБ РФ

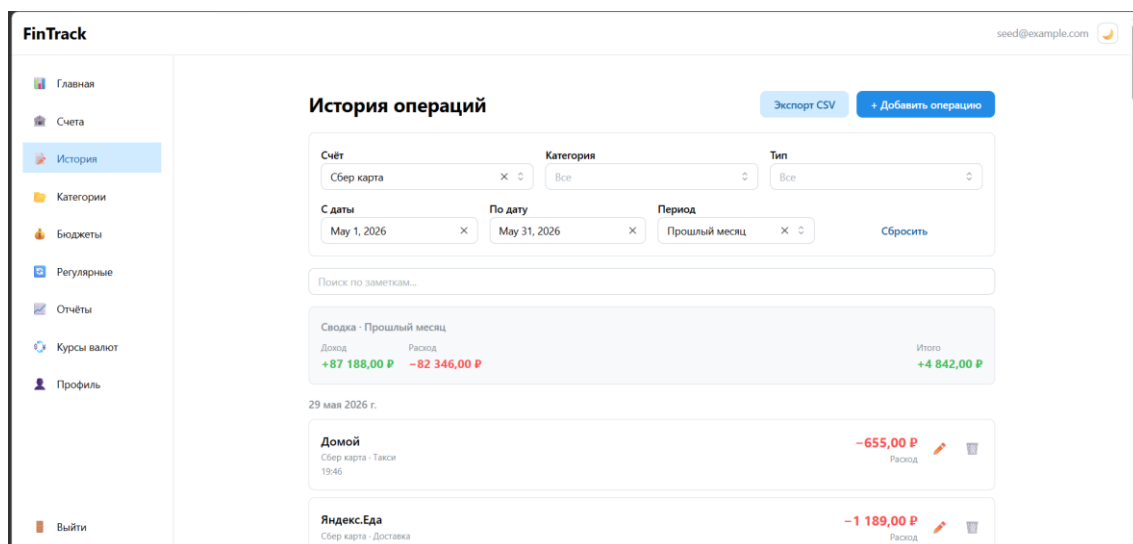


Рисунок В.4 – Страница «История операций»: фильтры по счёту, категории, типу и периоду, сводка и список операций по датам

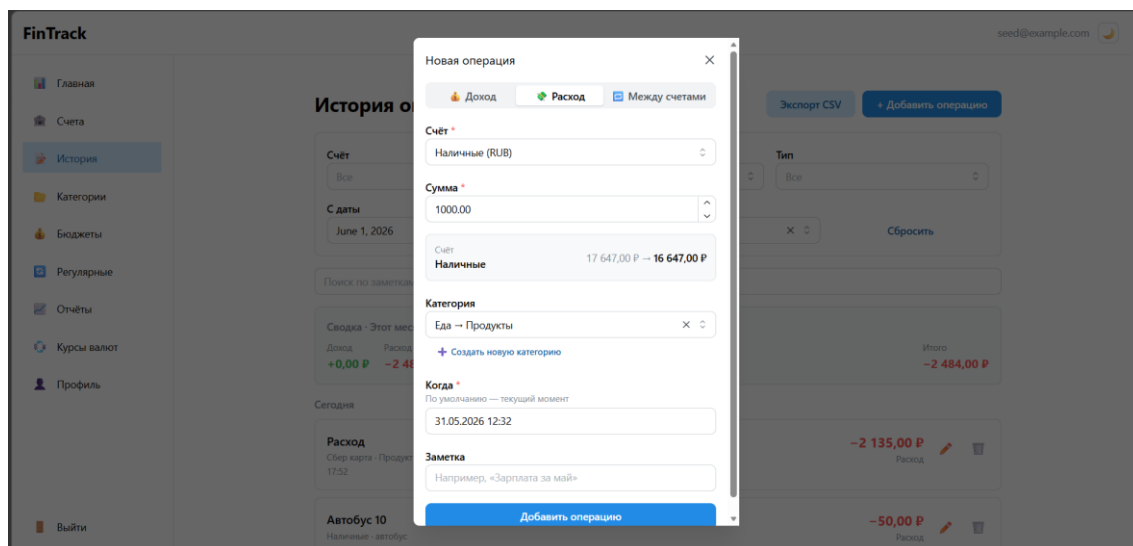


Рисунок В.5 – Форма создания операции с предпросмотром изменения баланса счёта

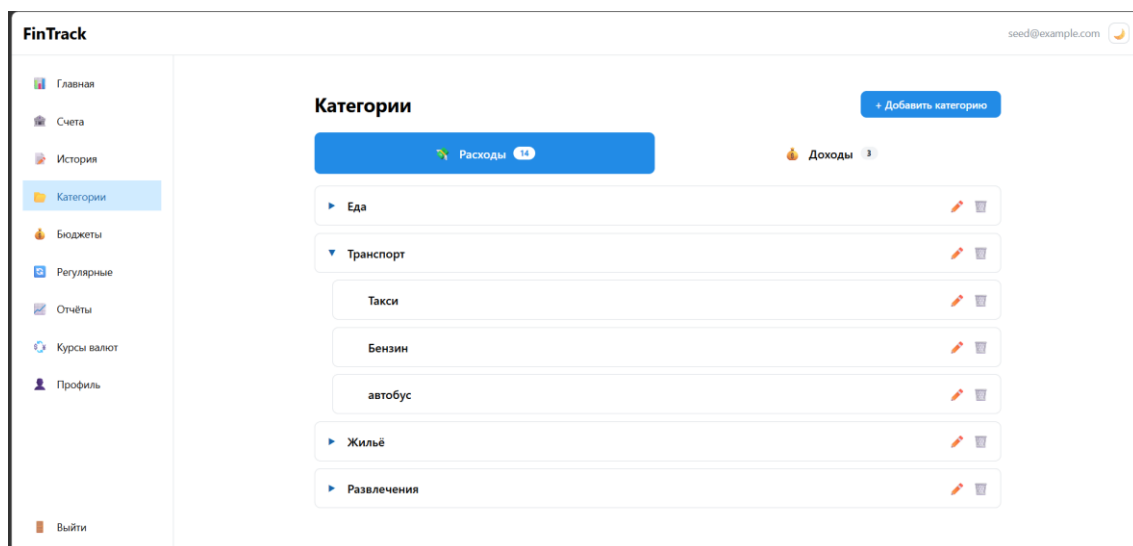


Рисунок В.6 – Страница «Категории»: иерархия категорий доходов и расходов с подкатегориями

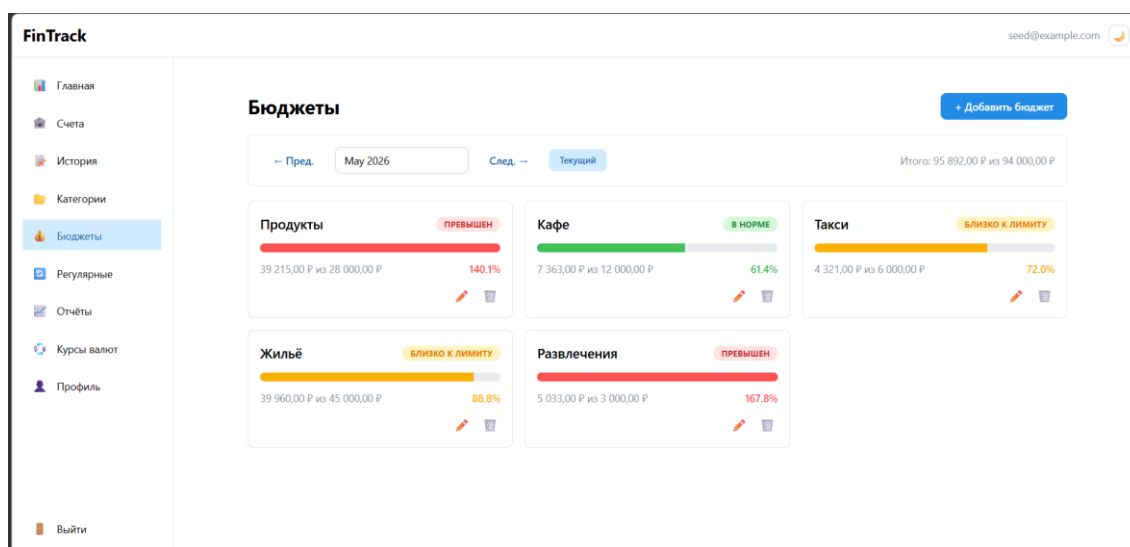


Рисунок В.7 – Страница «Бюджеты»: лимиты по категориям с индикаторами расхода и статусами «в норме», «близко к лимиту», «превышен»

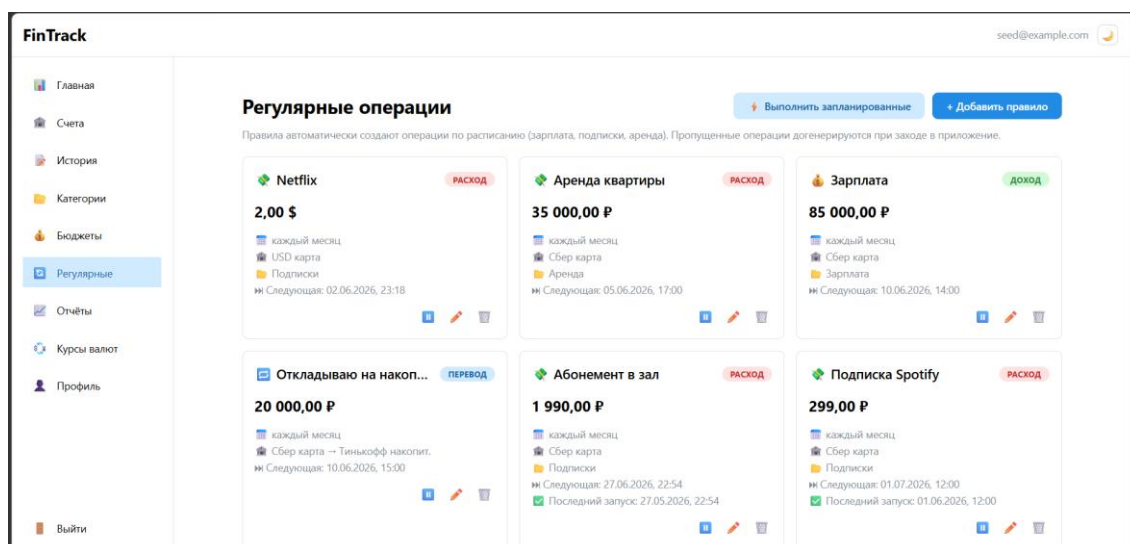


Рисунок В.8 – Страница «Регулярные операции»: правила автоматического создания повторяющихся операций по расписанию

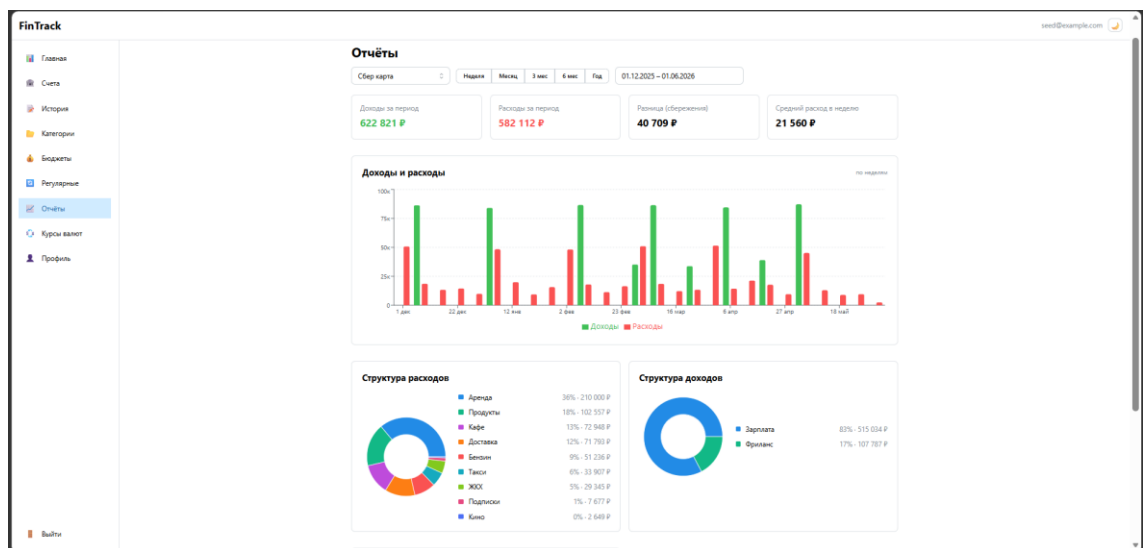


Рисунок В.9 – Страница «Отчёты»: динамика доходов и расходов и структура операций по категориям в виде диаграмм

FinTrack

ЦБ РФ НА 02 ИЮНЯ 2026 Г.
Обновлено 01.06.2026, 22:49

Курсы валют

Код	Валюта	Номинал	Курс (Р)	Курс за 1 единицу (Р)
AED	Дирхам ОАЭ	1	19,4835	19,4835
AMD	Армянский драм	100	19,4253	0,1943
AUD	Австралийский доллар	1	51,4324	51,4324
AZN	Азербайджанский манат	1	42,0901	42,0901
BDT	Так	100	58,2352	0,5824
BHD	Бахрейнский динар	1	190,2600	190,2600
BOB	Боливiano	1	10,3550	10,3550
BRL	Бразильский реал	1	14,1505	14,1505
BYN	Белорусский рубль	1	25,8333	25,8333
CAD	Канадский доллар	1	51,8577	51,8577
CHF	Швейцарский франк	1	91,3601	91,3601
CNY	Юань	1	10,5762	10,5762
CUP	Кубинский песо	10	29,8138	2,9814

Рисунок В.10 – Страница «Курсы валют»: официальные курсы ЦБ РФ с указанием номинала

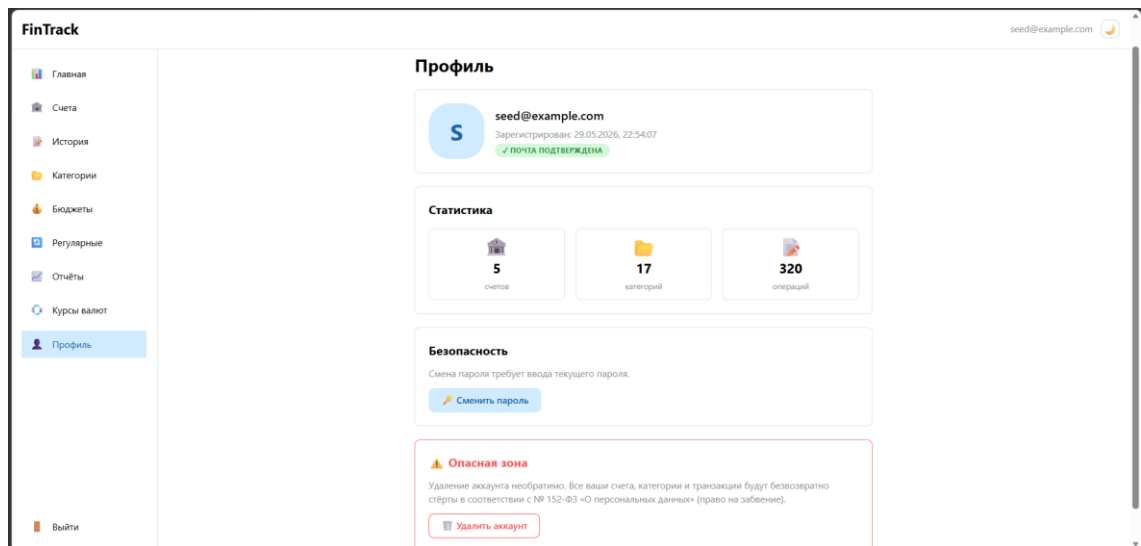


Рисунок В.11 – Страница «Профиль»: данные учётной записи, статистика, управление безопасностью и удаление аккаунта